
А. А. АНДРИАНОВА,
Л. Н. ИСМАГИЛОВ,
Т. М. МУХТАРОВА

АЛГОРИТМИЗАЦИЯ И ПРОГРАММИРОВАНИЕ ПРАКТИКУМ

Учебное пособие
Издание третье, стереотипное



• САНКТ-ПЕТЕРБУРГ • МОСКВА • КРАСНОДАР •
2024

УДК 004
ББК 32.973-018я723

А 65 Андрианова А. А. Алгоритмизация и программирование. Практикум : учебное пособие для СПО / А. А. Андрианова, Л. Н. Исмагилов, Т. М. Мухтарова. — 3-е изд., стер. — Санкт-Петербург : Лань, 2024. — 240 с. : ил. — Текст : непосредственный.

ISBN 978-5-507-50372-8

В пособии будут рассмотрены базовые приемы алгоритмизации, которые являются основой для решения более сложных задач программирования: проиллюстрированы основные понятия алгоритмизации (ветвление, циклы различных видов), базовые алгоритмы работы с основными структурами данных (массивами, символьными строками, матрицами). Отдельные главы учебного пособия посвящены структурированию программ (созданию пользовательских функций), а также работе с более сложными структурами данных (списками, деревьями, графами). Приведенные в учебном пособии примеры программ написаны на языках программирования C++ и C#, которые на данный момент являются одними из самых популярных универсальных языков программирования.

Соответствует современным требованиям Федерального государственного образовательного стандарта среднего профессионального образования и профессиональным квалификационным требованиям.

Данное учебное пособие предназначено для обучающихся по направлениям группы «Информатика и вычислительная техника» среднего профессионального образования, начинающих изучать программирование.

УДК 004
ББК 32.973-018я723

Обложка
П. И. ПОЛЯКОВА

© Издательство «Лань», 2024
© Коллектив авторов, 2024
© Издательство «Лань»,
художественное оформление, 2024

Введение	5
Глава 1. Простейшие алгоритмические конструкции	9
1.1. Блок условия	9
Домашнее задание	22
1.2. Цикл ПОКА (цикл с условием)	24
Домашнее задание	31
1.3. Цикл ДЛЯ (цикл с параметром)	33
Домашнее задание	41
Глава 2. Основные структуры данных.....	43
2.1. Массивы.....	43
Домашнее задание	66
2.2. Символьные строки.....	68
Домашнее задание	84
2.3. Двумерные массивы (матрицы).....	85
Домашнее задание	106
Глава 3. Структура программы	108
3.1. Функции.....	108
Домашнее задание	135
3.2. Рекурсивные функции.....	139
Домашнее задание	147

Глава 4. Линейные динамические структуры данных	150
4.1. Односвязные списки.....	150
Домашнее задание	166
4.2. Стеки и очереди	169
Домашнее задание	175
4.3. Двусвязные списки	181
Домашнее задание	188
 Глава 5. Нелинейные структуры данных	190
5.1. Деревья	190
Домашнее задание	206
5.2. Графы.....	207
Домашнее задание	233
 Литература.....	235

Алгоритм – это точно определенное описание способа решения задачи в виде конечной последовательности действий, ведущей от варьируемых начальных данных к искомому результату. Основное требование при составлении алгоритма заключается в его применимости для решения всех частных случаев решаемой задачи.

Алгоритмы реализуются на языках программирования. Все языки программирования являются искусственными языками, т. е. оперируют очень небольшим количеством команд, которые являются своеобразными «кирпичиками» программирования. В способности строить алгоритмы любой сложности из этих «кирпичиков» и заключается искусство программирования.

Помимо языков программирования существует еще целый ряд способов записи алгоритма, к числу которых относятся, в частности, блок-схемы и псевдокод. В данном учебном пособии для целей пояснения алгоритмов в первых двух главах пособия мы будем использовать блок-схемы, поскольку они являются наиболее наглядной и понятной формой записи алгоритма.

Блок-схема – это графическая интерпретация алгоритма. Каждый вид действия в ней обозначается с помощью некоторой геометрической фигуры (блока), внутри которой делается запись, конкретизирующая его смысл. Блоки блок-схемы связаны с помощью линий потока, определяющих ход выполнения вычислительного процесса. Поток начинается в специальном блоке начала алгоритма и заканчивается в блоке конца алгоритма. Обозначения основных блоков, используемых в блок-схеме, приведены в таблице 1.

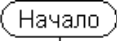
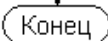
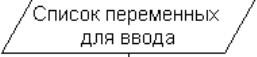
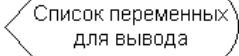
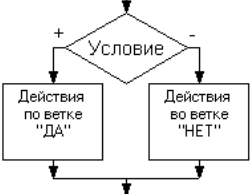
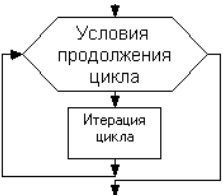
Учебное пособие состоит из пяти глав. Первая глава посвящена рассмотрению основных конструкций построения алгоритмов. Она разделена на три раздела: «Блок условия», «Цикл ПОКА (цикл с условием)», «Цикл ДЛЯ (цикл с параметром)».

Во второй главе приводятся основные алгоритмы работы с основными составными структурами данных. Эта глава также разделена на три раздела, посвященных работе с массивами, символьными строками и двумерными массивами (матрицами).

Третья глава посвящена принципам структурного программирования. Решение сложных задач обычно заключается в составлении большого алгоритма и соответственно в написании длинного программного кода его реализации. Для более эффективной разработки осуществляется разбиение этого алгоритма на небольшие и относительно независимые составные части, каждая из которых решает менее сложную отдельную подзадачу. Каждая из этих составных частей оформляется в

виде отдельной подпрограммы, которая называется функцией. Таким образом, третья глава учебного пособия посвящена разбору основных приемов декомпозиции программы и разработки функций. Она состоит из двух разделов – «Функции» и «Рекурсивные функции».

Таблица 1. Обозначения основных блоков блок-схемы

	Блок начала блок-схемы
	Блок конца блок-схемы
	Блок действия. Обычно обозначает присваивание значения некоторой переменной, например на основе вычисления ее значения
	Блок ввода значений. Значения вводятся в перечисленные в блоке переменные
	Блок вывода значений перечисленных в блоке переменных
	Условный блок, или блок ветвления. В зависимости от условия алгоритм разделяется на две ветви – ветвь выполнения условия (ветвь «ДА») и ветвь нарушения условия (ветвь «НЕТ»)
	Блок цикла. Это блок повторения некоторой последовательности действий (итерации цикла). В блоке цикла должно быть предусмотрено указание условия выхода из цикла

Четвертая глава учебного пособия посвящена использованию линейных динамических структур данных. Их главная особенность – нефиксированный размер данных, в отличие от массивов и матриц, при создании которых явно указывается их размер. Глава состоит из трех разделов, каждый из которых посвящен отдельным видам линейных динамических структур: односвязным спискам, стекам и очередям, двусвязным спискам. Для рассмотренных структур данных описываются основные операции работы с ними и приводится ряд приложений их использования.

Пятая глава учебного пособия посвящена построению и использованию нелинейных структур данных на примере деревьев и графов. Рассматриваются принципы создания, хранения и работы с ними. Глава состоит из двух разделов – «Деревья» и «Графы».

В каждом разделе пособия приведено теоретическое описание приемов работы с рассматриваемыми конструкциями, подробно разбирается несколько задач, а также приводится список задач для самостоятельного решения в виде домашнего задания (в некоторых случаях с методическими указаниями). Задачи имеют собственную нумерацию внутри разделов. Номера рисунков имеют нумерацию вида «Номер_главы.Номер_раздела.Номер_рисунка». Аналогичным образом строится нумерация приведенных листингов программного кода «Номер_главы.Номер_раздела.Номер_листинга».

При разборе каждой задачи приводится ее постановка, краткое обсуждение используемого алгоритма решения, блок-схема алгоритма для задач из первых двух глав, а также программный код с комментариями, написанный на двух языках программирования – C++ и C#. Программы были созданы и отлажены с помощью оболочки проектирования Microsoft Visual Studio 2017. Отметим, что в пособии намеренно не используется большинство объектно-ориентированных возможностей выбранных языков программирования, а также не рассматриваются подробно синтаксические особенности языков. Это сделано, чтобы акцентировать внимание читателя на алгоритмических аспектах решения задач.

Большинство описаний решаемых задач будет содержать программный код лишь на одном из выбранных языков программирования. Это обусловлено незначительными отличиями этих языков при реализации алгоритмов, которые в основном заключаются в способах организации ввода и вывода информации. Тем не менее мы не будем уделять одному языку больше внимания, чем другому, чтобы предоставить читателю примеры реализации сходных алгоритмов на разных языках программирования. Это позволит также провести сравнительную характеристику языков программирования, оценить их технологические особенности. В тех случаях, когда языки программирования имеют технологические

особенности, которые можно использовать при решении задачи, примеры программного кода будут приведены на обоих языках программирования.

При формировании блок-схем в первых двух главах мы будем пользоваться некоторыми обозначениями, принятыми для языка программирования C++. Некоторые из них используются также и в других языках программирования. Эти обозначения приведены далее в таблице 2.

Таблица 2. Обозначения некоторых операций и символов, используемые в блок-схемах

%	Операция вычисления остатка от деления
/	Операция деления, а также операция целочисленного деления в случае целых операндов
!=	Операция сравнения «не равно»
[]	Получение элемента массива по индексу
'\0'	EOI – признак конца строки
sqrt(x)	Функция вычисления квадратного корня из числа x

Глава 1. Простейшие алгоритмические конструкции

1.1. Блок условия

Достаточно часто возникает ситуация, когда результат работы алгоритма зависит от выполнения того или иного условия. В этом случае требуется выполнить одну из нескольких альтернативных последовательностей действий. Для задания возможных альтернатив используется блок условия (или, как его еще называют, блок ветвления, логический блок).

Основой этой конструкции является логическое условие. Если условие истинно, выполняются команды, следующие по ветви «Да» (+). В противном случае – команды из ветви «Нет» (–). Блок условия завершен, когда обе ветви соединяются для продолжения единого вычислительного процесса алгоритма.

Например, пусть даны две переменные a и b . Если $a < b$, переменная c должна принимать значение 1, иначе – значение 2. Фрагмент блок-схемы с таким условным блоком приведен на рис. 1.1.1.

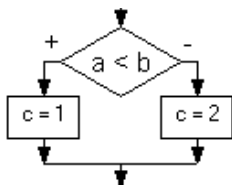


Рис. 1.1.1. Пример блока условия (логического блока)

Задача 1. Вычислить значение функции

$$y = \text{sign}(x) = \begin{cases} 1, & \text{если } x > 0 \\ -1, & \text{если } x < 0 \\ 0, & \text{если } x = 0 \end{cases}$$

Алгоритм решения данной задачи очень прост. Сначала требуется ввести значение переменной x , которая является аргументом функции. Далее следуют два блока условия для выделения трех вариантов вычисления функции. Результат вычислений распечатывается.

Блок-схема решения задачи 1 приведена на рис. 1.1.2, а программный код – в листинге 1.1.1.

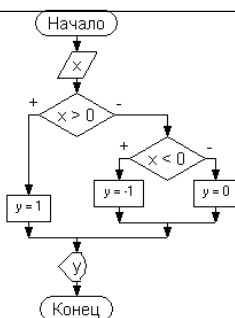


Рис. 1.1.2. Блок-схема для задачи вычисления функции $\text{sign}(x)$

```

#include <iostream>
#include <locale>
using namespace std;

int main()
{
    // включение русских букв
    setlocale(LC_STYPE, "rus");
    // ввод исходных данных задачи
    double x, y;
    cout << "Введите x: ";
    cin >> x;
    // вычисление функции sign(x)
    if (x > 0)
        y = 1;
    else
        if (x < 0)
            y = -1;
        else y = 0;
    cout << "y = sign(" << x << ") = " << y << endl;
    return 0;
}

```

Листинг 1.1.1. Программный код решения задачи 1 на языке C++

Нередко бывает необходимо осуществить выбор, заданный группой условий. Эти условия можно объединить с помощью логических операций. Существуют три основные логические операции – логическое НЕ, логическое ИЛИ и логическое И. С их помощью можно комбинировать несколько условий в блоке ветвления. Приоритет выполнения операций можно изменить с помощью круглых скобок. Следующие задачи демонстрируют применение этих логических операций в алгоритмах.

Задача 2. Определить, принадлежит ли точка заштрихованной области, изображенной на рис. 1.1.3.

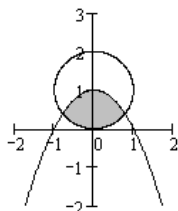


Рис. 1.1.3. Область из задачи 2

Заштрихованная область образуется пересечением двух множеств:

$$\{(x, y): x^2 + (y - 1)^2 \leq 1\} \text{ и } \{(x, y): y \leq 1 - x^2\}.$$

Для проверки, принадлежит ли точка одному множеству, требуется проверить, выполняется ли условие из определения этого множества. Для проверки принадлежности точки сразу двум множествам удобно использовать операцию «логическое И» (результат операции «условие1 И условие2» будет истинным, если истинными являются одновременно и условие1, и условие2). Поэтому условие проверки принадлежности точки (x, y) заштрихованной области записывается так:

$$x * x + (y - 1) * (y - 1) \leq 1 \text{ И } y \leq 1 - x * x.$$

Блок-схема решения задачи 2 представлена на рис.1.1.4, а программный код – в листинге 1.1.2.

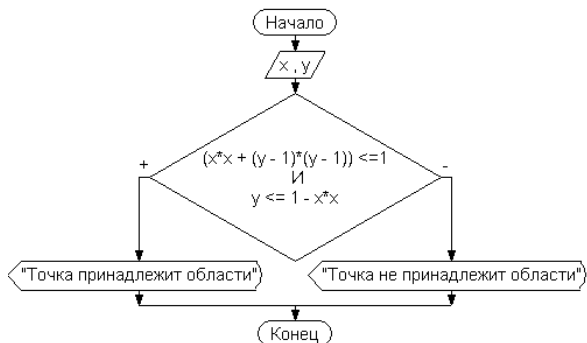


Рис. 1.1.4. Блок-схема решения задачи о принадлежности точки области, изображенной на рис. 1.1.3

```

#include <iostream>
#include <locale>

using namespace std;

int main()
{
    setlocale(LC_CTYPE, "rus");
    // ввод исходных данных задачи
    double x, y;
    cout << "Введите x:";
    cin >> x;
    cout << "Введите y:";
    cin >> y;
    // проверка принадлежности точки множеству
    if (x * x + (y - 1) * (y - 1) <= 1 &&
        y <= 1 - x * x)
        cout << "Точка принадлежит области" << endl;
    else
        cout << "Точка не принадлежит области" << endl;
    return 0;
}

```

Листинг 1.1.2. Программный код решения задачи 2 на языке C++

Задача 3. Определить, принадлежит ли точка заштрихованной области, изображенной на рис. 1.1.5.

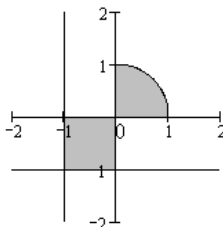


Рис. 1.1.5. Область из задачи 3

В данном случае множество состоит из объединения двух множеств:

$$\{(x, y): x^2 + y^2 \leq 1, x \geq 0, y \geq 0\} \text{ и } \{(x, y): -1 \leq x \leq 0, -1 \leq y \leq 0\}.$$

Точка принадлежит заштрихованной области, если она принадлежит хотя бы одному из этих двух множеств.

Для формулирования условия здесь подходит операция «логического ИЛИ». Результат операции «условие1 ИЛИ условие2» будет

истинным, если будет выполняться хотя бы одно из условий. Таким образом, условие в блоке ветвления имеет вид:

$$(x * x + y * y \leq 1 \text{ И } x \geq 0 \text{ И } y \geq 0) \text{ ИЛИ} \\ (x \geq -1 \text{ И } x \leq 0 \text{ И } y \geq -1 \text{ И } y \leq 0)$$

Блок-схема решения задачи 3 приведена на рис. 1.1.6, а программный код – в листинге 1.1.3.

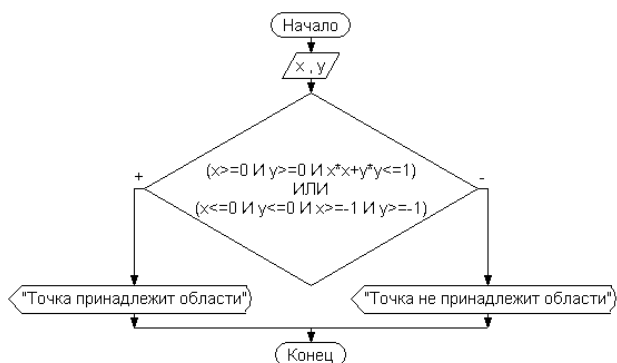


Рис. 1.1.6. Блок-схема решения задачи о принадлежности точки области, изображенной на рис. 1.1.5

```
#include <iostream>
#include <locale>
using namespace std;

int main(void)
{
    setlocale(LC_CTYPE, "rus");
    // ввод исходных данных задачи
    double x, y;
    cout << "Введите x:"; cin >> x;
    cout << "Введите y:"; cin >> y;
    // проверка принадлежности точки множеству
    if ((x >= 0 && y >= 0 && x * x + y * y <= 1) ||
        (x <= 0 && y <= 0 && x >= -1 && y >= -1))
        cout << "Точка принадлежит области" << endl;
    else
        cout << "Точка не принадлежит области" << endl;
    return 0;
}
```

Листинг 1.1.3. Программный код решения задачи 3 на языке C++

Задача 4. Определить, принадлежит ли точка заштрихованной области, изображенной на рис. 1.1.7.

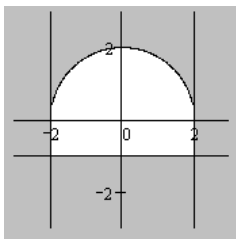


Рис. 1.1.7. Область из задачи 4

В данном случае проще описать множество точек, которые не входят в допустимое множество – это объединение множеств:

$$\{(x, y): x^2 + y^2 < 4, y \geq 0\} \text{ и } \{(x, y): -2 < x < 2, -1 < y \leq 0\}.$$

Поэтому здесь лучше использовать операцию «логического НЕ» (если условие1 ложно, то (НЕ условие1) истинно, и наоборот).

Блок-схема решения задачи 4 представлена на рис. 1.1.8, а программный код – в листинге 1.1.4.

```
#include <iostream>
#include <locale>
using namespace std;

int main(void)
{
    setlocale(LC_CTYPE, "rus");
    // ввод исходных данных задачи
    double x, y;
    cout << "Введите x:"; cin >> x;
    cout << "Введите y:"; cin >> y;
    // проверка принадлежности точки множеству
    if (!((x * x + y * y < 4 && y >= 0) ||
        (-2 < x && x < 2 && -1 < y && y <= 0)))
        cout << "Точка принадлежит области" << endl;
    else
        cout << "Точка не принадлежит области" << endl;
    return 0;
}
```

Листинг 1.1.4. Программный код решения задачи 4 на языке C++

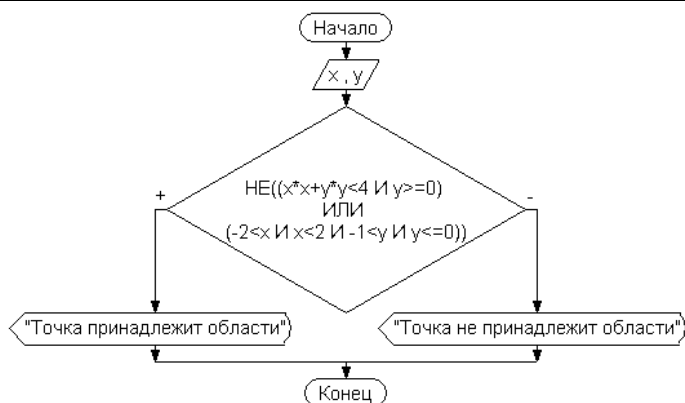


Рис. 1.1.8. Блок-схема решения задачи о принадлежности точки области, изображенной на рис. 1.1.7

Задача 5. Найти корни квадратного уравнения $ax^2 + bx + c = 0$.

Прежде чем приступать к поиску корней уравнения через вычисление дискриминанта, надо выяснить, не является ли нулем коэффициент a . При $a = 0$ уравнение становится линейным. Далее следует проверить на равенство нулю коэффициенты b и c . В зависимости от этого уравнение может не иметь решений ($b = 0, c \neq 0$), иметь бесконечное множество решений ($b = 0, c = 0$) или при $b \neq 0$ иметь единственное решение.

Если же $a \neq 0$, надо вычислить дискриминант и найти корни обычного квадратного уравнения.

Блок-схема решения задачи 5 представлена на рис. 1.1.9, а программный код – в листинге 1.1.5.

Программный код написан на языке программирования C#. Следует обратить внимание на другие синтаксические правила организации ввода и вывода в этом языке. Особенно это касается ввода информации, так как ввод осуществляется в виде текстовых строк (функция *ReadLine()*). В этом случае всегда надо осуществлять преобразование типа данных с помощью функции *Parse()*. Так, в следующем примере вводимые данные преобразуются к типу *double*:

```
double a = double.Parse(Console.ReadLine());
```

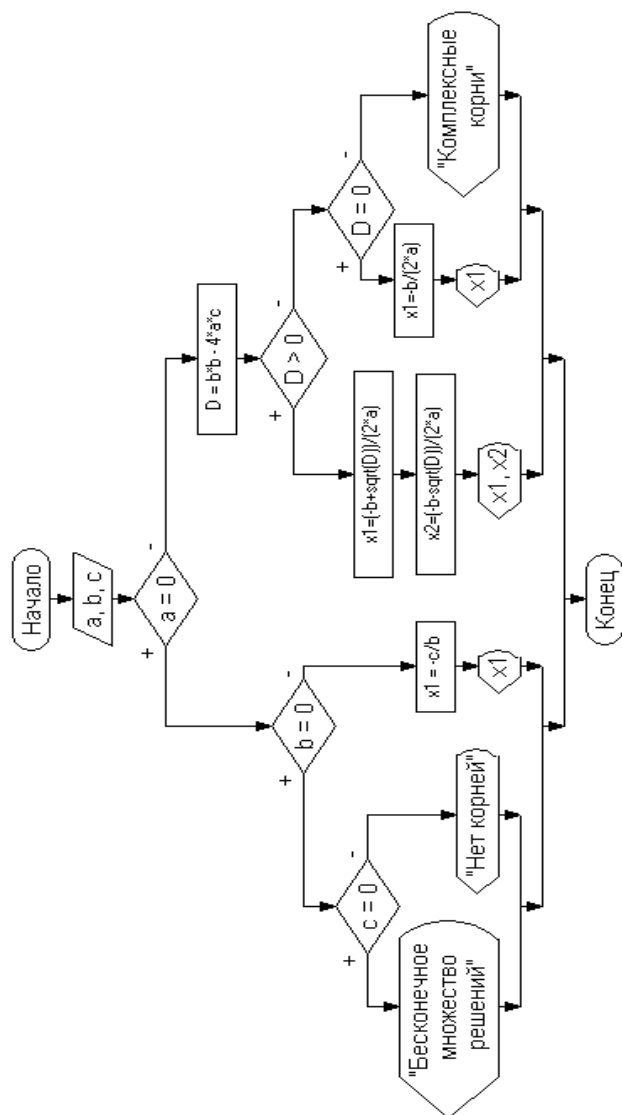


Рис. 1.1.9. Блок-схема решения задачи о корнях квадратного уравнения


```

using System;

namespace Example
{
    class Program
    {
        static void Main(string[] args)
        {
            double a, b, c;
            double D, x1, x2;
            Console.WriteLine("Введите коэффициенты уравнения:");
            a = double.Parse(Console.ReadLine());
            b = double.Parse(Console.ReadLine());
            c = double.Parse(Console.ReadLine());
            // нестандартные случаи задания коэффициентов уравнения
            if (a == 0)
                if (b == 0)
                    if (c == 0)
                        Console.WriteLine("Бесконечное множество решений");
                    else
                        Console.WriteLine("Нет корней");
                else
                {
                    x1 = -c / b;
                    Console.WriteLine("x={0}", x1);
                }
            else
            {
                // вычисление корней через дискриминант
                D = b * b - 4 * a * c;
                if (D > 0)
                {
                    x1 = (-b + Math.Sqrt(D)) / (2 * a);
                    x2 = (-b - Math.Sqrt(D)) / (2 * a);
                    Console.WriteLine("x1={0}, x2={1}", x1, x2);
                }
                else
                {
                    if (D == 0)
                    {
                        x1 = -b / (2 * a);
                        Console.WriteLine("x={0}", x1);
                    }
                    else
                        Console.WriteLine("Комплексные корни");
                }
            }
        }
    }
}

```

Листинг 1.1.5. Программный код решения задачи 5 на языке C#

Задача 6. Пусть пара чисел (x, y) задает координаты клетки шахматной доски. Даны координаты двух клеток. Определить, являются ли они клетками одного цвета.

После ввода координат клеток следует проверить их корректность. Шахматная доска имеет размер 8×8 . Следовательно, каждая из координат должна быть целым числом из интервала $[1; 8]$. Если хотя бы одна из координат окажется некорректной, надо сообщить пользователю о неправильном вводе.

Заметим, что белые поля шахматной доски имеют четную сумму координат, т. е. $x + y$ – четное число, а черные поля – нечетную сумму.

Это следует из того, что при переходе от одной клетки некоторого цвета к другой сумма координат изменится на число, кратное 2. Например, при движении по горизонтали (вертикали) – клетки того же цвета расположены левее (выше) на 2, 4 и т. д. или правее (ниже) на 2, 4, т. е. одна из координат меняется на четное число. При движении по диагонали либо сумма остается неизменной (одна координата увеличивается, другая – уменьшается на одно и то же число – движение по стрелке б) на рис. 1.1.10), либо сумма координат меняется на четное число (обе координаты меняются на одно и то же число, что означает неизменную разность координат – движение по стрелке а) на рис. 1.1.10).

Блок-схема решения задачи 6 представлена на рис. 1.1.11, а программный код – в листинге 1.1.6.

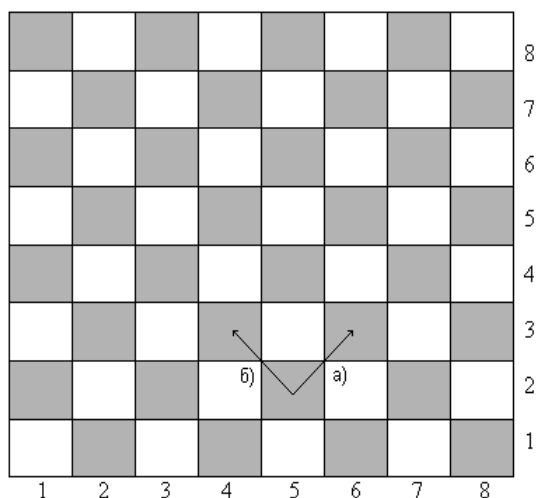


Рис. 1.1.10. Схема шахматной доски

```

#include <iostream>
#include <locale>
using namespace std;

int main(void)
{
    setlocale(LC_CTYPE, "rus");
    int x1, y1, x2, y2;
    int s1, s2;
    cout << "Введите координаты двух клеток шахматной
                                                    доски:";

    cin >> x1 >> y1 >> x2 >> y2;
    // проверка корректности ввода данных
    if (!(x1 >= 1 && x1 <= 8 && y1 >= 1 && y1 <= 8 &&
        x2 >= 1 && x2 <= 8 && y2 >= 1 && y2 <= 8))
    {
        cout << "Неправильно заданы координаты" << endl;
        return 0 ;
    }
    s1 = x1 + y1;
    s2 = x2 + y2;
    if (s1 % 2 == s2 % 2)
        cout << "Поля одного цвета" << endl;
    else
        cout << "Поля разных цветов" << endl;
    return 0;
}

```

Листинг 1.1.6. Программный код решения задачи 6 на языке C++

Задача 7. Заданы координаты двух клеток шахматной доски. На первой из них расположен белый слон, на второй – фигура противника. Определить, бьет ли слон фигуру противника.

По правилам игры в шахматы слон бьет фигуру противника, если они находятся на одной диагонали. Через клетку, на которой стоит слон, может проходить одна или две диагонали. Условия проверки принадлежности двух клеток одной и той же диагонали обоснованы при объяснении задачи 6 – если сумма или разность координат обеих фигур совпадают, то слон бьет фигуру противника.

Блок-схема решения этой задачи изображена на рис. 1.1.12, а программный код на языке C# – в листинге 1.1.7.

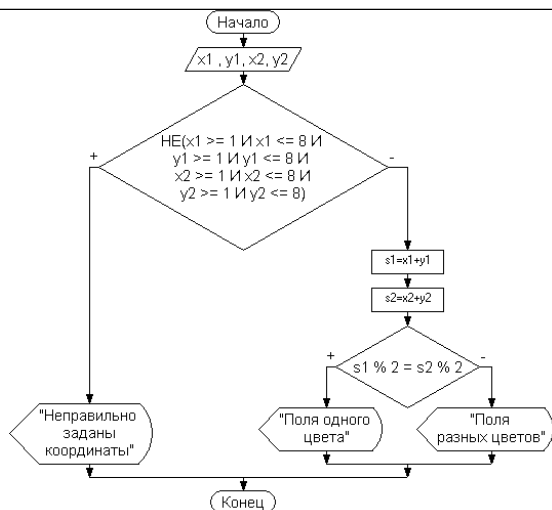


Рис. 1.1.11. Блок-схема решения задачи о клетках шахматной доски

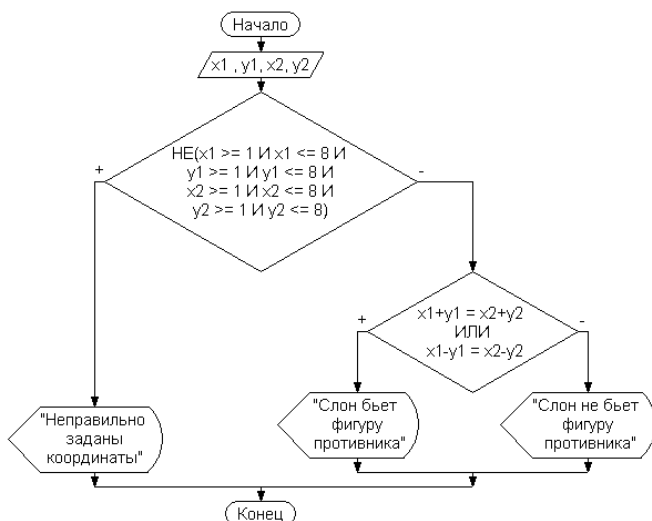


Рис. 1.1.12. Блок-схема решения задачи о слоне и фигуре противника

```

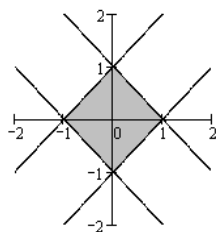
using System;

namespace Example
{
    class Program
    {
        static void Main(string[] args)
        {
            int x1, y1, x2, y2;
            Console.WriteLine("Введите координаты двух
                               клеток шахматной доски:");
            x1 = int.Parse(Console.ReadLine());
            y1 = int.Parse(Console.ReadLine());
            x2 = int.Parse(Console.ReadLine());
            y2 = int.Parse(Console.ReadLine());
            if (!(x1 >= 1 && x1 <= 8 && y1 >= 1 && y1 <= 8 &&
                  x2 >= 1 && x2 <= 8 && y2 >= 1 && y2 <= 8))
            {
                Console.WriteLine("Неправильно заданы
                                   координаты");
                return;
            }
            if (x1 + y1 == x2 + y2 || x1 - y1 == x2 - y2)
                Console.WriteLine("Слон бьет фигуру противника");
            else
                Console.WriteLine("Слон не бьет фигуру
                                   противника");
        }
    }
}

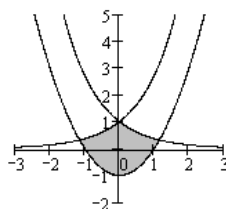
```

Листинг 1.1.7. Программный код решения задачи 7 на языке C#

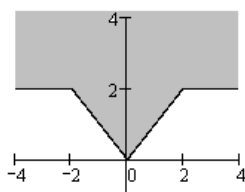
1. Определить, принадлежит ли точка (x, y) заштрихованной области, изображенной на рис. 1.1.13 (а, б, в, г, д).



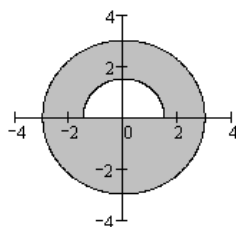
а)



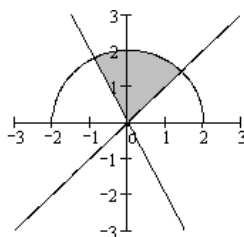
б)



в)



г)



д)

Рис. 1.1.13. Области из задачи 1 домашнего задания

2. Даны три целых числа. Вывести «истина», если только одно из них является четным.
3. Даны три целых числа. Получить максимальное из них.
4. Даны три целых числа. Вывести эти числа в порядке возрастания.

-
5. Заданы координаты двух клеток шахматной доски. На первой из них расположен белый конь, на второй – фигура противника. Определить, бьет ли конь фигуру противника.
 6. Заданы координаты двух клеток шахматной доски. На первой из них расположен белый ферзь, на второй – фигура противника. Определить, бьет ли ферзь фигуру противника.
 7. Даны три числа, обозначающие дату (день, месяц и год). Проверить корректность даты.
 8. Дана корректная дата в виде трех целых чисел. Получить дату следующего дня.
 9. Дано целое число от 1 до 999. Распечатать его вместе со словом «рубль» в правильном падеже. Например, 32 рубля, 51 рубль, 112 рублей.

1.2. Цикл ПОКА (цикл с условием)

Очень часто в алгоритмах приходится выполнять одну и ту же последовательность действий для варьируемых данных. В этом случае используется блок цикла, который обозначается в блок-схеме так:

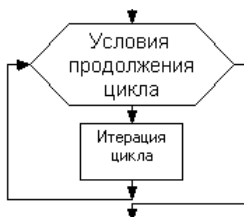


Рис. 1.2.1. Блок цикла

Цикл ПОКА – одна из разновидностей блока цикла. Вход в цикл осуществляется, если условие продолжения цикла, указанное внутри блока, принимает истинное значение. Далее выполняется тело цикла. После этого снова проверяется условие и так далее, пока условие остается истинным. Одно выполнение тела цикла называется итерацией цикла.

Примером использования цикла ПОКА может являться ситуация ввода пароля, изображенная на рис. 1.2.2, – пока пользователь не введет правильный пароль, он будет запрашиваться снова.

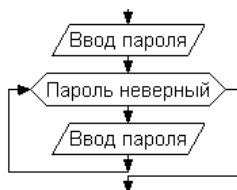


Рис. 1.2.2. Фрагмент блок-схемы «Ввод и проверка пароля»

Задача 1. Найти сумму цифр записи заданного целого числа.

Для решения задачи 1 требуется решить три подзадачи – получить последнюю цифру записи числа (вычислить остаток от деления числа на 10, например, $2345 \div 10 = 5$), перейти к числу, которое отличается от исходного отсутствием младшего разряда (выполнить целочисленное деление числа на 10, например $3456/10 = 345$), и определить, были ли просмотрены все разряды числа.

Алгоритм решения заключается в организации цикла, итерация которого выделяет последнюю цифру числа, добавляет ее к сумме и

станет равным нулю (после учета последней цифры $3/10 = 0$), цикл заканчивается.

Блок-схема решения задачи 1 приведена на рис. 1.2.3, а программный код – в листинге 1.2.1.

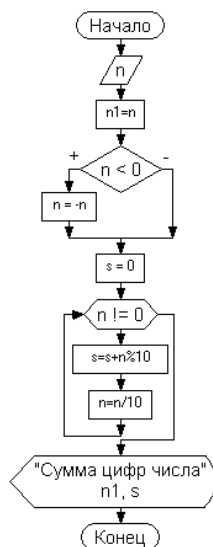


Рис. 1.2.3. Блок-схема задачи о сумме цифр
записи целого числа

```
#include <iostream>
#include <locale>

using namespace std;

int main(void)
{
    setlocale(LC_CTYPE, "rus");
    // ввод и подготовка исходных данных
    int n, n1, s;
    cout << "Введите n:";
    cin >> n;
    n1 = n;
    // если число отрицательное, рассматриваем его модуль
    if (n < 0)
        n = - n;
```

Листинг 1.2.1. Программный код решения задачи 1 на языке C++
(Начало)

```

// переменная для суммы цифр числа
// вначале сумма равна 0
s = 0;
// пока разряды не закончатся
while (n != 0)
{
    // выделение последней цифры и учет ее в сумме
    s = s + n % 10;
    // удаление последней цифры из записи числа n
    n = n / 10;
}
cout << "Сумма цифр числа " << n1 << " = " << s << endl;
return 0;
}

```

Листинг 1.2.1. Программный код решения задачи 1 на языке C++
(Окончание)

Задача 2. Найти значение числа π с заданной точностью ε из следующего соотношения:

$$\pi = 4 \left(1 - \frac{1}{3} + \frac{1}{5} - \dots + \frac{(-1)^{i-1}}{2i-1} + \dots \right).$$

Для вычисления π нужно найти сумму бесконечной последовательности, которая будет накапливаться в переменной pi . Очередной член ряда без учета знака заносится в переменную $drob$. Для его вычисления введем отдельную переменную для знаменателя дроби – $znam$, которая при переходе к очередному члену ряда увеличивает свое значение на 2. Для чередования знака очередного члена ряда вводится переменная z , которая принимает значение 1 или -1 и рассматривается как множитель члена ряда. Когда очередное слагаемое ряда станет меньше значения ε , заданная точность будет достигнута и вычисления можно остановить.

Блок-схема решения задачи 2 приведена на рис. 1.2.4, а программный код – в листинге 1.2.2.

```

using System;
namespace Example
{
    class Program
    {
        static void Main(string[] args)
        {
            double eps;
            double pi = 0, drob;
            Console.Write("Введите точность eps:");
            eps = double.Parse(Console.ReadLine());

```

Листинг 1.2.2. Программный код решения задачи 2 на языке C# (Начало)

```

// проверка корректности введенных данных
if (eps <= 0)
{
    Console.WriteLine("Точность должна быть
                        положительной");
    return;
}
int z = 1;          // знак очередного слагаемого
double znam = 1;    // знаменатель дроби
drob = 4;           // первое слагаемое
// вычисления продолжаем, пока очередной член ряда
// больше требуемой точности
while (drob > eps)
{
    // учет очередного слагаемого
    pi = pi + z * drob;
    // вычисление следующего слагаемого
    z = -z; znam = znam + 2;
    drob = 4 / znam;
}
Console.WriteLine("pi={0}", pi);
}
}

```

Листинг 1.2.2. Программный код решения задачи 2 на языке C#
(Окончание)

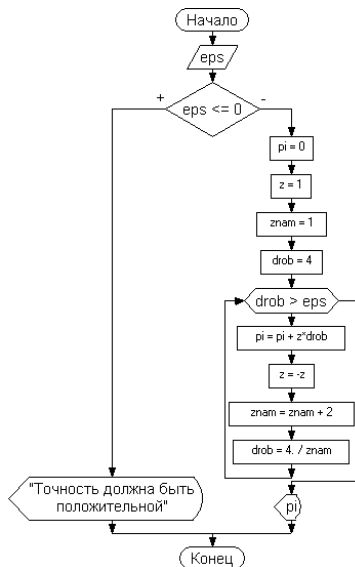


Рис. 1.2.4. Блок-схема задачи о вычислении числа π

Задача 3. Найти наибольший общий делитель (НОД) двух неотрицательных чисел по алгоритму Евклида.

Алгоритм Евклида заключается в следующем. Пусть m и n – одновременно неравные нулю целые неотрицательные числа ($m \geq n$).

$$\text{НОД}(m, n) = \begin{cases} m, & \text{если } n = 0 \\ \text{НОД}(n, \text{ost}), & \text{если } n \neq 0, \text{ где } \text{ost} = m \% n \end{cases}$$

Таким образом, для вычисления НОД необходимо свести задачу к поиску НОД для меньшего из чисел и остатка от деления большего числа на меньшее. Процесс продолжается до тех пор, пока остаток от деления большего числа на меньшее не станет равным нулю.

Блок-схема решения задачи 3 приведена на рис. 1.2.5, а программный код – в листинге 1.2.3.

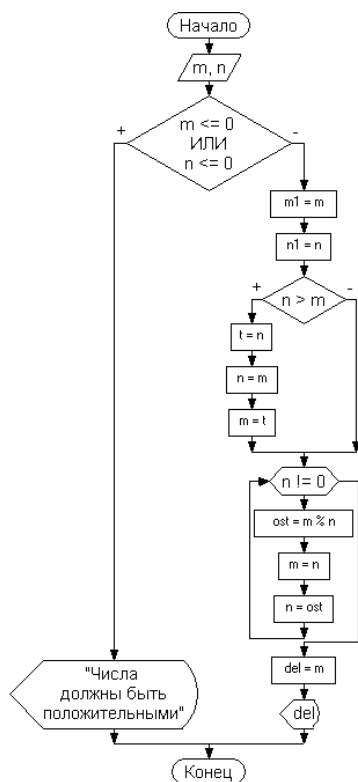


Рис. 1.2.5. Блок-схема задачи о вычислении НОД двух чисел

```

#include <iostream>
#include <locale>

using namespace std;

int main(void)
{
    setlocale(LC_CTYPE, "rus");
    int n, m, m1, n1;
    int t, ost, del;
    // ввод и проверка корректности исходных данных
    cout << "Введите два числа:";
    cin >> m >> n;
    if (m <= 0 || n <= 0)
    {
        cout << "Числа должны быть положительными" << endl;
        return 0;
    }
    m1 = m; n1 = n;
    // требуется обеспечить, чтобы m >= n
    if (n > m)
    {
        t = n;
        n = m;
        m = t;
    }
    // алгоритм Евклида
    while (n != 0)
    {
        ost = m % n;
        m = n;
        n = ost;
    }
    del = m;
    cout << "НОД двух чисел " << m1 << " и " << n1 <<
        " = " << del << endl;
    return 0;
}

```

Листинг 1.2.3. Программный код решения задачи 3 на языке C++

Задача 4. Найти минимальный элемент во входном потоке.

Сначала вводим количество элементов, которые будут извлекаться из входного потока. Далее используем прием, достаточно часто применяющийся для нахождения минимума или максимума. Пусть E – достаточно большое число (например, в его качестве можно использовать машинный максимум, для C++ – `INT_MAX`, для C# – свойство `MaxValue` типа данных) такое, что каждый из элементов входного потока не

превосходит его. Примем за текущий минимум (переменная *min*) значение *E*. Таким образом, первый же полученный элемент станет текущим минимумом. Далее, сравнивая последовательно все числа с текущим минимумом, меняем его значение при нахождении меньшего значения. В заключение распечатываем минимум.

Блок-схема решения задачи 4 приведена на рис. 1.2.6, а программный код – в листинге 1.2.4.

```
using System;

namespace Example
{
    class Program
    {
        static void Main(string[] args)
        {
            int n, a,
            min = int.MaxValue; // так можно получить
                                // максимальное целое число в C#
            Console.WriteLine("Введите количество чисел
                                последовательности:");
            n = int.Parse(Console.ReadLine());
            Console.WriteLine("Введите последовательность
                                чисел:");

            int i = 1;
            while (i <= n)
            {
                // ввод очередного числа из входного потока
                a = int.Parse(Console.ReadLine());
                // сравнение с текущим значением минимума
                if (min > a)
                    min = a;
                i++;
            }
            Console.WriteLine("min={0}", min);
        }
    }
}
```

Листинг 1.2.4. Программный код решения задачи 4 на языке C#

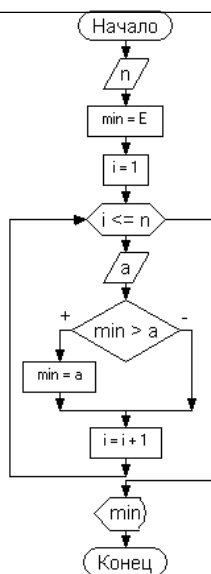


Рис. 1.2.6. Блок-схема задачи о минимальном элементе
во входном потоке

Домашнее задание

1. Найти методом деления отрезка пополам корни уравнения $f(x) = 0$ на отрезке $[a, b]$ с заданной точностью ϵ .
Кратко опишем суть метода деления отрезка пополам. Полагаем, что отрезок задан так, что $f(a)f(b) < 0$, функция $f(x)$ непрерывна и унимодальна на $[a, b]$. В этом случае гарантируется, что корень на отрезке $[a, b]$ будет существовать (поскольку на границах отрезка функция принимает разные знаки, то она обязательно пересечет ось абсцисс, т. е. обязательно найдется x из отрезка $[a, b]$ такое, что $f(x) = 0$). Метод деления отрезка пополам заключается в следующем. До тех пор пока длина отрезка не будет меньше заданной точности, делим отрезок на две равные части и выбираем для продолжения поиска ту из них, на концах которой функция принимает значения разных знаков. Для примера можно рассмотреть функцию $f(x) = (x + 1)^2 - 5$, $[a, b] = [-1; 5]$.

-
2. Дано целое число. Определить, входит ли заданная цифра в запись этого числа.
3. Из заданного целого числа сформировать новое, цифры в котором идут в обратном порядке.
4. Определить, является ли заданное целое число числом Фибоначчи. Числа Фибоначчи определяются по правилу: $F_1 = F_2 = 1, F_i = F_{i-1} + F_{i-2}$ для $i = 3, 4, 5 \dots$
5. Вычислить с заданной точностью ε сумму следующих рядов. Считается, что требуемая точность достигнута, если очередное слагаемое по модулю меньше ε .
- а) $y = e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots + \frac{x^{n-1}}{(n-1)!} + \dots;$
- б) $y = \cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \dots + \frac{(-1)^{n-1} x^{2n-2}}{(2n-2)!} + \dots;$
- в) $y = \operatorname{arctg} x = x - \frac{x^3}{3} + \frac{x^5}{5} - \dots + \frac{(-1)^{n-1} x^{2n-1}}{(2n-1)} + \dots;$
- г) $y = \ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \dots + \frac{(-1)^{n-1} x^n}{n} + \dots$

Другим видом цикла является цикл ДЛЯ. Его удобно использовать в тех случаях, когда точно известно количество итераций цикла. Тогда создается переменная-счетчик, которая будет изменять свое значение при переходе от одной итерации к другой, пока не достигнет обозначенной при определении цикла границы. Чаще всего эта переменная имеет определенный смысл: номер элемента массива, номер строки или столбца матрицы, степень числа и др.

При оформлении блок-схемы в блоке цикла указываются три части, которые разделяются запятыми. Первая часть служит для инициализации переменной-счетчика. Вторая задает ее граничное значение. Когда переменная-счетчик становится больше граничного значения, цикл завершается. Третья часть представляет собой приращение значения переменной-счетчика при переходе к следующей итерации цикла. Оно происходит в конце каждой итерации. Значение приращения называют также шагом цикла.

В качестве примера использования цикла ДЛЯ приведем фрагмент алгоритма решения следующей задачи – требуется вывести значения средних температур за год по месяцам. Фрагмент блок-схемы показан на рис. 1.3.1.



Рис. 1.3.1. Фрагмент блок-схемы с циклом ДЛЯ

Задача 1. Найти $n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot n$.

Алгоритм не требует особенных комментариев. Значение переменной n должно быть неотрицательным. Цикл с параметром используется для перебора всех сомножителей искомого произведения (от 2 до n). Блок-схема решения этой задачи представлена на рис. 1.3.2, а программный код – в листинге 1.3.1.

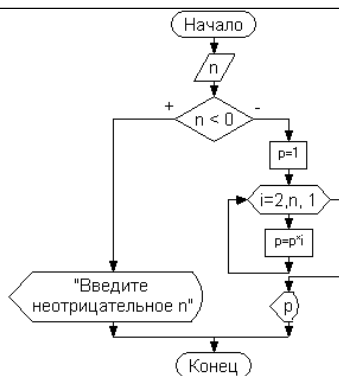


Рис. 1.3.2. Блок-схема решения задачи о вычислении $n!$

```

#include <iostream>
#include <locale>

using namespace std;

int main(void)
{
    setlocale(LC_CTYPE, "rus");
    // ввод n и проверка корректности значения
    int n;
    cout << "Введите n:";
    cin >> n;
    if (n < 0)
    {
        cout << "Введите неотрицательное n." << endl;
        return 0;
    }
    // организация цикла вычисления n!
    int p = 1;
    for (int i = 2; i <= n; i++)
        p = p * i;
    cout << n << "! = " << p << endl;
    return 0;
}

```

Листинг 1.3.1. Программный код решения задачи 1 на языке C++

Задача 2. Найти максимальное совершенное число, не превышающее заданного M . Число является совершенным, если оно равно сумме всех своих делителей, включая 1 и исключая само себя.

В данном случае приходится использовать один цикл внутри итерации другого. Такие конструкции алгоритма называются вложенными циклами.

При решении данной задачи во внешнем цикле просматриваются все натуральные числа, начиная с заданного значения M в обратном порядке. В этом случае шаг при переходе от итерации к итерации равен -1 . Во внутреннем цикле определяется, является ли число совершенным. Для этого находят все возможные делители числа и вычисляется их сумма.

Блок-схема решения задачи 2 показана на рис. 1.3.3, а программный код приведен в листинге 1.3.2.

```
using System;

namespace Example
{
    class Program
    {
        static void Main(string[] args)
        {
            int M, Sdel = 1;    // Sdel - переменная для хранения
                               // суммы делителей числа
            // ввод исходных данных и проверка их корректности
            Console.WriteLine("Введите M:");
            M = int.Parse(Console.ReadLine());
            if (M <= 0)
            {
                Console.WriteLine("Введите положительное M");
                return;
            }
            // цикл поиска максимального совершенного числа
            for (int k = M; k > 0; k--)
            {
                // цикл подсчета суммы делителей числа k
                Sdel = 1;
                for (int i = 2; i <= k / 2; i++)
                {
                    if (k % i == 0)
                        Sdel = Sdel + i;
                }
                if (Sdel == k && k != 1)
                {
                    Console.WriteLine("Совершенное число={0}", k);
                    k = 1;
                }
            }
            if (Sdel == 1)
                Console.WriteLine("Таких чисел нет");
        }
    }
}
```

Листинг 1.3.2. Программный код решения задачи 2 на языке C#

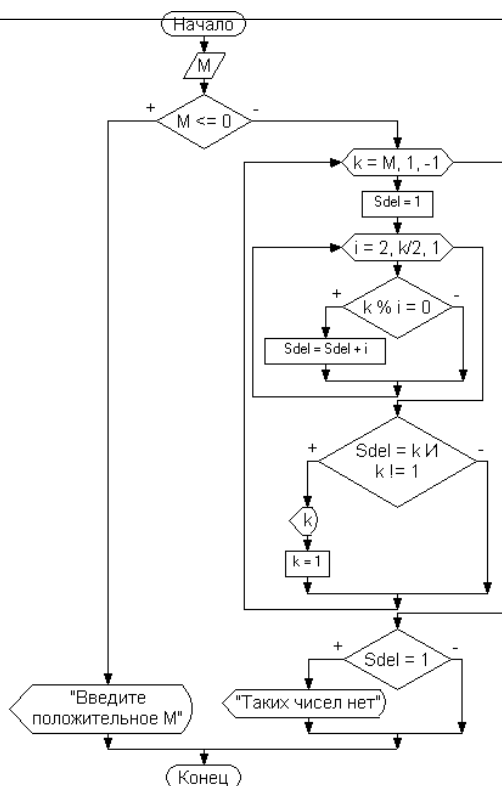


Рис. 1.3.3. Блок-схема решения задачи о совершенном числе

Задача 3. Цилиндр объема 1 имеет радиус основания r . Определить высоту для значений $r = 0.5, 1, 1.5, \dots 5$.

Алгоритм решения этой задачи очень прост. Нестандартным здесь является только шаг изменения переменной-счетчика – шаг равен 0.5. Поэтому тип переменной-счетчика надо задать как «число с плавающей точкой».

Блок-схема решения задачи 3 показана на рис. 1.3.4, а программный код – в листинге 1.3.3.

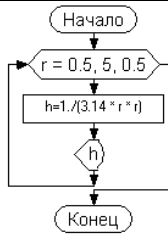


Рис. 1.3.4. Блок-схема решения задачи о высоте цилиндра

```

#include <iostream>
#include <locale>

using namespace std;

int main(void)
{
    setlocale(LC_CTYPE, "rus");
    for (double r = 0.5; r <= 5; r = r + 0.5)
    {
        double h = 1. / (3.14*r*r);
        cout << "r = " << r << "\t h = " << h << endl;
    }
    return 0;
}

```

Листинг 1.3.3. Программный код решения задачи 3 на языке C++

Задача 4. Распечатать все простые числа до 100.

Число 2 распечатываем без проверки, так как это единственное простое четное число. Все остальные нечетные числа проверяем на «простоту». Для проверки простоты числа k требуется исследовать в качестве делителей все нечетные числа от 3 до $k/2$. Если найдется хотя бы один делитель, то число k не является простым. Факт нахождения такого делителя можно определить с помощью приема, называемого «использованием флажка». Флажком является переменная, которая имеет всего два значения. Одно из них соответствует состоянию «истина», другое – «ложь». Значение переменной меняется при выполнении определенных условий (в данной задаче – при нахождении делителя числа k). Проверив все возможные делители, по значению переменной-флажка определяем, выполнилось ли требуемое условие.

Заметим, что для проверки на простоту числа k достаточно было проверить на делимость все нечетные числа до квадратного корня из k . Например, если $k = 15$, то вовсе не обязательно проверять в качестве делителя число 5. Поскольку $k = 5 \cdot 3$, то меньший делитель (3) будет

найден на более ранних итерациях. Поэтому проверку следует осуществлять до квадратного корня из k , например $25=5 \cdot 5$.

Блок-схема показана на рис. 1.3.5, а программный код приведен в листинге 1.3.4.

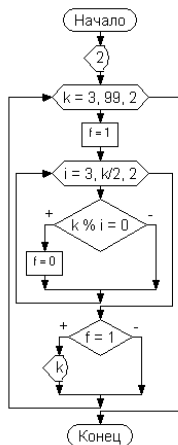


Рис. 1.3.5. Блок-схема решения задачи о простых числах

```
using System;
namespace Example
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("2 ");
            for (int k = 3; k < 100; k += 2)
            {
                int f = 1;
                for (int i = 3; i <= Math.Sqrt(k); i += 2)
                {
                    //если найдется делитель числа, т. е. оно не
                    //переменная-флажок f меняет значение
                    if (k % i == 0) f = 0;
                }
                if (f == 1)
                    Console.WriteLine(k + " ");
            }
        }
    }
}
```

Листинг 1.3.4. Программный код решения задачи 4 на языке C#

Задача 5. Распечатать все четырехзначные числа, в записи которых нет повторяющихся цифр.

Первая цифра числа может принимать значение от 1 до 9. Остальные – от 0 до 9. Поэтому в алгоритме создаются четыре вложенных друг в друга цикла. Переменные-счетчики каждого цикла соответствуют цифре разряда числа. При получении цифры очередного разряда проверяется, не встречалась ли она ранее. Когда все цифры искомого числа получены, формируем само число и печатаем его.

Блок-схема показана на рис. 1.3.6, а программный код – в листинге 1.3.5.

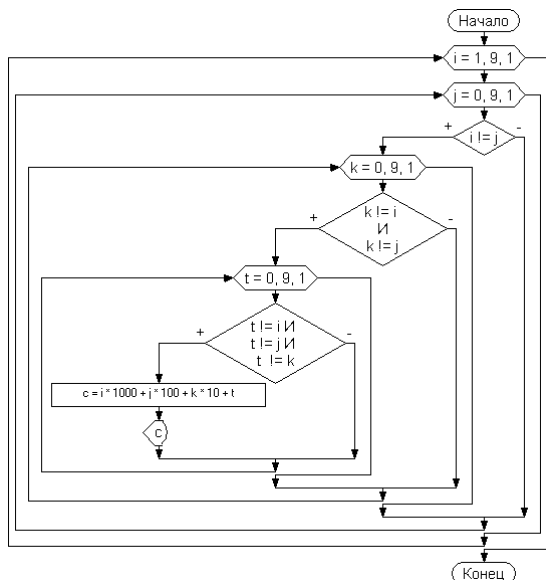


Рис. 1.3.6. Блок-схема решения задачи о числах с неповторяющимися цифрами

Задача 6. Разложение числа на простые множители. Известно, что любое целое положительное число можно представить в виде произведения простых чисел. Например, $84=2 \cdot 2 \cdot 3 \cdot 7$.

Для решения этой задачи требуется проверить все возможные делители числа, начиная с 2. Пусть i – очередной делитель числа k . Далее ищем делители числа k/i до тех пор, пока делимое не станет равным 1. Каждый делитель может встречаться в произведении несколько раз. Поэтому, не изменяя делитель до тех пор, пока не получим число, которое на него не делится. Заметим, что не требуется проверять делители на «простоту», поскольку составные делители уже вошли в произведение в виде их разложения на простые множители.

Блок-схема показана на рис. 1.3.7, а программный код приведен в листинге 1.3.6.

```
#include <iostream>
#include <locale>
using namespace std;
int main(void)
{
    setlocale(LC_CTYPE, "rus");
    // i, j, k, t - разные цифры в записи числа
    for (int i = 1; i <= 9; i++)
        for (int j = 0; j <= 9; j++)
            if (i != j)
                for (int k = 0; k <= 9; k++)
                    if (k != i && k != j)
                        for (int t = 0; t <= 9; t++)
                            if (t != i && t != j && t != k)
                                {
                                    // формируем четырехзначное число из
                                    // зафиксированных цифр и его печать
                                    int c = i * 1000 + j * 100 + k * 10 + t;
                                    cout << c << endl;
                                }
    return 0;
}
```

Листинг 1.3.5. Программный код решения задачи 5 на языке C++

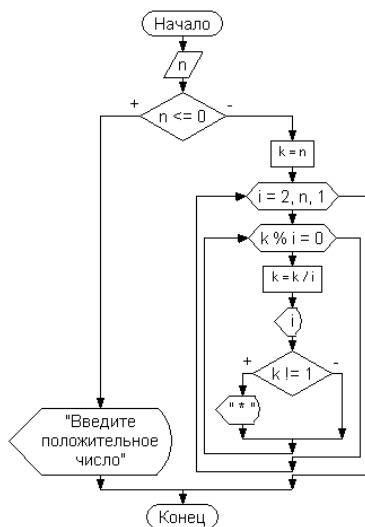


Рис. 1.3.7. Блок-схема решения задачи о разложении числа на простые множители


```

using System;
namespace Example
{
    class Program
    {
        static void Main(string[] args)
        {
            int n;
            Console.Write("Введите n:");
            n = int.Parse(Console.ReadLine());
            if (n <= 0)
            {
                Console.Write("Введите положительное число");
                return;
            }
            int k = n;
            for (int i = 2; i <= n; i++)
                while (k % i == 0)
                {
                    k = k / i; Console.Write(" " + i);
                    if (k != 1) Console.Write("*");
                }
        }
    }
}

```

Листинг 1.3.6. Программный код решения задачи 7 на языке C#

Домашнее задание

1. Дано натуральное n . Вычислить

$$1 \cdot 2 + 2 \cdot 3 \cdot 4 + \dots + n \cdot (n + 1) \cdot \dots \cdot (2n).$$

2. Найти значение $n!!$. Если n – четное, то

$$n!! = 2 \cdot 4 \cdot 6 \cdot \dots \cdot n,$$

если n – нечетное, то

$$n!! = 1 \cdot 3 \cdot 5 \cdot \dots \cdot n.$$

3. Даны два положительных числа a, b . Найти число точек с целыми координатами, попадающих в эллипс:

$$\left\{ (x, y) : \frac{x^2}{a} + \frac{y^2}{b} \leq 1 \right\}.$$

-
4. Найти все числа Армстронга, принадлежащие заданному отрезку $[a, b]$. Числом Армстронга является число, сумма кубов цифр которого совпадает с самим числом.
Например, $153 = 1^3 + 5^3 + 3^3$.
5. Найти все пары дружественных чисел, принадлежащих отрезку $[a, b]$. Два числа называются дружественными, если они равны сумме делителей друг друга (исключая сами числа). Например, дружественными числами являются числа 220 и 284.
6. Вычислить x^n .
7. Вывести все номера счастливых билетов. Номер билета состоит из 6 цифр. Счастливым считается билет, сумма первых трех цифр записи которого совпадает с суммой трех последних цифр записи.
8. Найти n -е по порядку число Фибоначчи. Напомним, что числа Фибоначчи определяются по правилу:

$$F_1 = F_2 = 1, F_i = F_{i-1} + F_{i-2} \text{ для } i = 3, 4, 5 \dots$$

9. Найти количество различных цифр, участвующих в записи заданного целого числа.
10. Вычислить по схеме Горнера следующую функцию:

$$y = x^{10} + 2x^9 + 3x^8 + \dots + 10x + 11.$$

Согласно схеме Горнера:

$$\begin{aligned} y &= x^{10} + 2x^9 + 3x^8 + \dots + 10x + 11 = \\ &= x(x^9 + 2x^8 + \dots + 10) + 11 = \\ &= x(x(x^8 + 2x^7 + \dots + 9) + 10) + 11 = \dots \end{aligned}$$

2.1. Массивы

Под структурой данных понимается способ организации хранения набора данных и алгоритмов доступа к ним. В общем случае простой структурой данных является любой встроенный тип данных. Однако в алгоритмах особый интерес представляют составные структуры данных, т. е. структуры, предоставляющие возможности хранения нескольких значений. Одной из самых простых и часто используемых таких структур данных является массив.

Использование массивов позволяет удобным способом размещать в памяти большое количество необходимой однотипной информации. Массив – это набор переменных одинакового типа, имеющих одно базовое имя. Обратиться к конкретной переменной массива можно по ее базовому имени и номеру (индексу). Например, к десятому элементу массива a обращаемся как к $a[10]$. В памяти компьютера переменные массива расположены друг за другом в одном непрерывном блоке.

С элементами массива работают как с обычными переменными. Поэтому, когда требуется работать с массивом в целом, для перебора элементов используется цикл. Например, ввод и вывод массива происходит по следующей схеме:

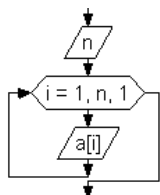


Рис. 2.1.1. Фрагмент блок-схемы ввода массива

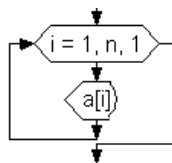


Рис. 2.1.2. Фрагмент блок-схемы вывода массива

В дальнейшем для краткости будем обозначать каждый из этих блоков так:

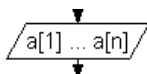


Рис. 2.1.3. Блок ввода массива

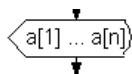


Рис. 2.1.4. Блок вывода массива

В языках программирования трактовка принципов хранения массивов может быть разной. Например, в языке C++ массив располагается в непрерывном фрагменте оперативной памяти без зазоров, сохраняя все

элементы массива подряд (рис. 2.1.5). При этом имя массива трактуется как адрес, по которому располагается первый элемент массива (хранение адреса осуществляется через специальный тип данных – указатель). Таким образом, индекс элемента массива можно интерпретировать как смещение относительно начала массива. Именно поэтому нумерация в массиве начинается с 0, так как никакого смещения для получения первого элемента массива делать не требуется.



Рис. 2.1.5. Схема расположения элементов массива в памяти

В языке C# логически подобная трактовка массива сохраняется, правда, непосредственная работа с элементами массива и его началом как с адресом памяти не поддерживается.

Задача 1. Найти сумму отрицательных элементов массива.

Кратко алгоритм решения задачи таков. Просматриваем все элементы массива, перебирая индексы элементов (от 0 до $n - 1$). Если очередной элемент отрицательный, добавляем его значение к сумме. После завершения цикла определяем, были ли отрицательные элементы в массиве, по значению переменной, хранящей сумму. Если таких элементов не было, сумма остается равной 0.

Блок-схема решения задачи представлена на рис. 2.1.6.

Программный код приведем для обоих языков программирования C++ и C# (листинги 2.1.1 и 2.1.2). Главное отличие будет заключаться не в способе обработки массива, а в способе выделения и освобождения памяти, выделенной для его элементов.

Для работы с массивами важно понимать способы динамического выделения памяти, т. е. по исходным данным задачи определяется количество элементов массива и происходит выделение памяти, достаточной для их хранения. В обоих языках программирования (C++ и C#) выделение памяти осуществляет операция *new*. Но так как количество элементов может быть таким, что выделение памяти будет невозможно (оно может быть отрицательными или слишком большим), операция *new* имеет возможность сигнализировать об этом, вернув специальную константу (*NULL* в C++ и *null* – в C#). Если выделение памяти пройдет удачно, то при написании программ на языке C++ необходимо будет эту память освободить с помощью операции *delete*. Для языка C# необходимости в этом нет – работу по освобождению памяти осуществляет специальная утилита, которая называется сборщиком мусора.

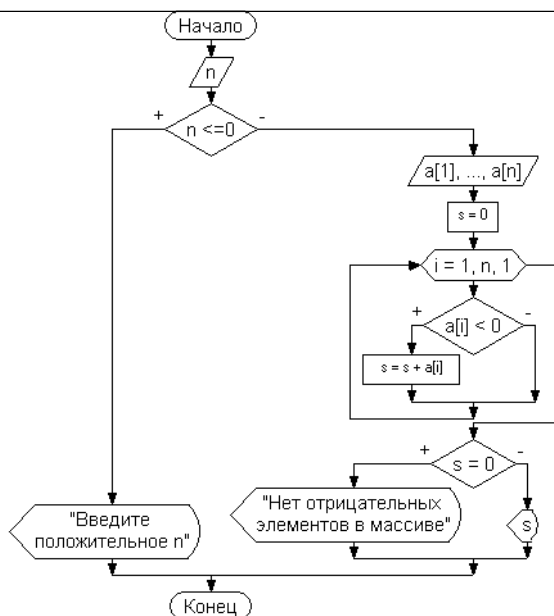


Рис. 2.1.6. Блок-схема решения задачи о сумме отрицательных элементов массива

```

#include <iostream>
#include <locale>

using namespace std;

int main(void)
{
    setlocale(LC_CTYPE, "rus");
    int n;
    int * a; // переменная-указатель для адреса начала массива
    cout << "Введите количество элементов массива n:";
    cin >> n;
    if (n <= 0)
    {
        cout << "Введите положительное n." << endl;
        return 0;
    }
    // выделение памяти для массива размера n
    a = new int[n];

```

Листинг 2.1.1. Программный код решения задачи 1 на языке C++
(Начало)

```

if (a == NULL)
{
    cout << "Запрошено большое количество памяти." << endl;
    cout << "Попробуйте ввести меньшее количество." << endl;
    return 0;
}
cout << "Введите элементы массива:" << endl;
for (int i = 0; i < n; i++)
    cin >> a[i];
// переменная для хранения суммы
// отрицательных элементов
int s = 0;
// цикл перебора элементов массива
for (int i = 0; i < n; i++)
    if (a[i] < 0)
        s = s + a[i];
if (s == 0)
    cout << "Нет отрицательных элементов в массиве" << endl;
else
    cout << "s =" << s << endl;
// освобождение памяти, занимаемой массивом
delete[] a;
return 0;
}

```

Листинг 2.1.1. Программный код решения задачи 1 на языке C++
(Окончание)

```

using System;

namespace Example
{
    class Program
    {
        static void Main(string[] args)
        {
            int n;
            int[] a; // объявление массива
            Console.Write("Введите количество элементов массива n:");
            n = int.Parse(Console.ReadLine());
            if (n <= 0)
            {
                Console.WriteLine("Введите положительное n");
                return;
            }
        }
    }
}

```

Листинг 2.1.2. Программный код решения задачи 1 на языке C# (Начало)

```

// выделение памяти для массива размера n
a = new int[n];
if (a == null)
{
    Console.WriteLine("Запрошено большое количество
                        памяти.");
    Console.WriteLine("Попробуйте ввести меньшее
                        количество.");
    return;
}
Console.WriteLine("Введите элементы массива:");
for (int i = 0; i < n; i++)
    a[i] = int.Parse(Console.ReadLine());
// переменная для хранения суммы
// отрицательных элементов
int s = 0;
// цикл перебора элементов массива
for (int i = 0; i < n; i++)
    if (a[i] < 0)
        s = s + a[i];
if (s == 0)
    Console.WriteLine("Нет отрицательных элементов в
                        массиве.");
else
    Console.WriteLine("s = {0}", s);
}
}
}

```

Листинг 2.1.2. Программный код решения задачи 1 на языке C#
(Окончание)

Задача 2. Найти значение максимального элемента массива.

Максимальным может быть любой из элементов массива. В том числе и первый элемент. Поэтому запомним значение первого элемента массива в переменную *max*, т. е. принимаем это значение за текущий максимум. Далее просматриваем все элементы массива, начиная со второго, и сравниваем каждый со значением переменной *max* (с текущим максимумом). Если очередной элемент больше текущего максимума, то заменяем значение текущего максимума, запоминая тем самым найденный больший элемент. Просмотрев весь массив, в переменной *max* получим максимальное значение.

Блок-схема решения задачи представлена на рис. 2.1.7. Программный код приведен на языке C++ и представлен в листинге 2.1.3.

Для краткости не будем далее приводить фрагмент программного кода, в котором проверяется, была ли выделена память под хранение массива.

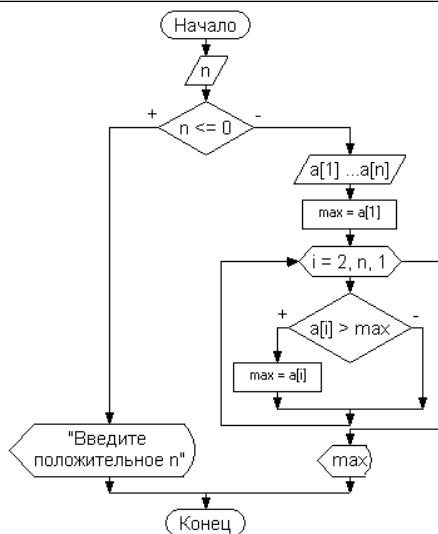


Рис. 2.1.7. Блок-схема решения задачи
о максимальном элементе массива

```

#include <iostream>
#include <locale>

using namespace std;

int main(void)
{
    setlocale(LC_CTYPE, "rus");
    int n;
    int * a;
    cout << "Введите количество элементов массива n:";
    cin >> n;
    if (n <= 0)
    {
        cout << "Введите положительное n" << endl;
        return 0;
    }
    // выделение памяти для массива размера n
    a = new int[n];
    cout << "Введите элементы массива:" << endl;
    for (int i = 0; i < n; i++)
        cin >> a[i];
}

```

Листинг 2.1.3. Программный код решения задачи 2 на языке C++
(Начало)


```

int max = a[0];    // переменная для текущего максимума
for (int i = 1; i < n; i++)
    if (a[i] > max)
        max = a[i];
cout << "max = " << max << endl;
delete[] a;
return 0;
}

```

Листинг 2.1.3. Программный код решения задачи 2 на языке C++
(Окончание)

Задача 3. Определить количество минимальных элементов массива.

Рассмотрим два алгоритма решения этой задачи. Первый алгоритм (более понятный) заключается в разбиении исходной задачи на две подзадачи: нахождение минимального элемента и подсчет количества равных ему элементов. Заметим, что при этом приходится дважды просмотреть весь массив.

Второй алгоритм (более эффективный) решает обе подзадачи за один просмотр массива. Так же как в предыдущей задаче, предполагаем, что первый элемент может являться минимальным. Соответственно, количество минимальных элементов в начальный момент равно 1. Встретив элемент, совпадающий с текущим минимумом, увеличиваем их количество на 1. Если же найден элемент, меньший текущего минимума, то меняем минимум на новый и устанавливаем количество в 1.

Блок-схемы, описывающие эти алгоритмы, представлены на рис. 2.1.8 и 2.1.9. Программный код реализации первого алгоритма приведем на языке C++ в листинге 2.1.4, а программный код второго алгоритма – на языке C# в листинге 2.1.5.

```

#include <iostream>
#include <locale>

using namespace std;

int main(void)
{
    setlocale(LC_CTYPE, "rus");
    int n;
    int * a;
    cout << "Введите количество элементов массива n:";
    cin >> n;
}

```

Листинг 2.1.4. Программный код решения задачи 3 на языке C++
(Начало)

```

if (n <= 0)
{
    cout << "Введите положительное n" << endl; return 0;
}
// выделение памяти для массива размера n
a = new int[n];
cout << "Введите элементы массива:" << endl;
for (int i = 0; i < n; i++)
    cin >> a[i];
// поиск значения минимального элемента
int min = a[0];
for (int i = 1; i < n; i++)
    if (min > a[i]) min = a[i];
// подсчет количества минимальных элементов
int kol = 0;
for (int i = 0; i < n; i++)
    if (a[i] == min) kol++;
cout << "Количество = " << kol << endl;
delete [] a;
return 0;
}

```

Листинг 2.1.4. Программный код решения задачи 3 на языке C++
(Окончание)

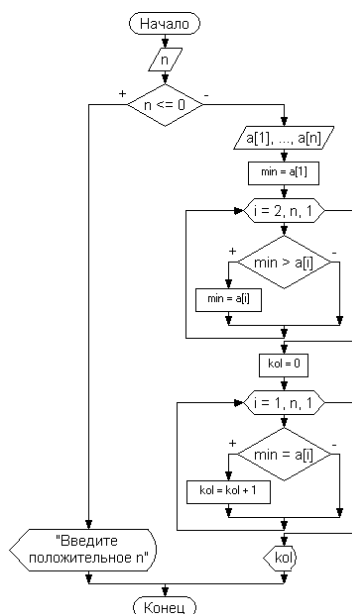


Рис. 2.1.8. Блок-схема решения задачи о количестве минимальных элементов массива (алгоритм 1)

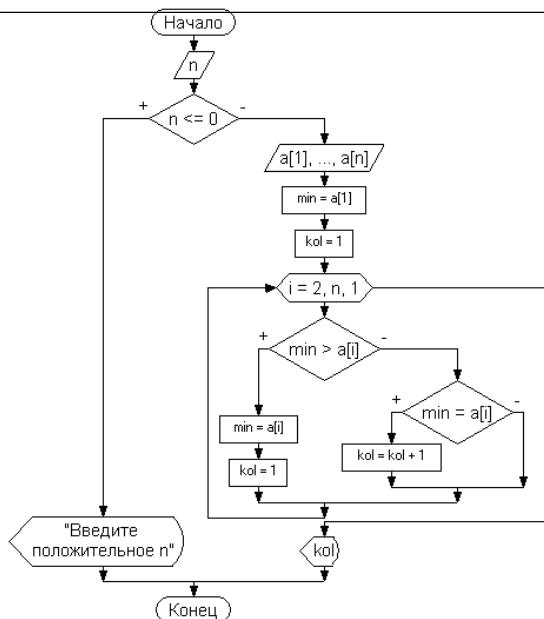


Рис. 2.1.9. Блок-схема решения задачи о количестве минимальных элементов массива (алгоритм 2)

```

using System;
namespace Example
{
    class Program
    {
        static void Main(string[] args)
        {
            int n;
            int[] a;
            Console.Write("Введите количество элементов массива n:");
            n = int.Parse(Console.ReadLine());
            if (n <= 0)
            {
                Console.WriteLine("Введите положительное n");
                return;
            }
            a = new int[n];
            Console.WriteLine("Введите элементы массива:");
            for (int i = 0; i < n; i++)
                a[i] = int.Parse(Console.ReadLine());
        }
    }
}

```

Листинг 2.1.5. Программный код решения задачи 3 на языке C# (Начало)

```

int min = a[0];
int kol = 1;
for (int i = 1; i < n; i++)
    if (min > a[i])
    {
        // нахождение нового минимального элемента
        min = a[i];
        kol = 1;
    }
    else
        // получение еще одного минимального элемента
        if (min == a[i]) kol++;
        Console.WriteLine("Количество = {0}", kol);
}
}
}

```

Листинг 2.1.5. Программный код решения задачи 3 на языке C#
(Окончание)

Задача 4. Поменять в массиве местами минимальный и максимальный элементы.

Отличие этой задачи от двух предыдущих заключается в том, что для перестановки двух элементов массива надо запомнить их порядковые номера в массиве. Для этого вводятся две переменные *kmin* и *kmax*. Их значения меняются одновременно с изменением текущих минимума и максимума (значений переменных *min* и *max*).

Блок-схема решения задачи 4 показана на рис. 2.1.10, а программный код представлен в листинге 2.1.6.

```

#include <iostream>
#include <locale>
using namespace std;
int main(void)
{
    setlocale(LC_CTYPE, "rus");
    int n;
    int * a;
    cout << "Введите количество элементов массива n:";
    cin >> n;
    if (n <= 0)
    {
        cout << "Введите положительное n" << endl;
        return 0;
    }
    // выделение памяти для массива размера n
    a = new int[n];
}

```

Листинг 2.1.6. Программный код решения задачи 4 на языке C++
(Начало)

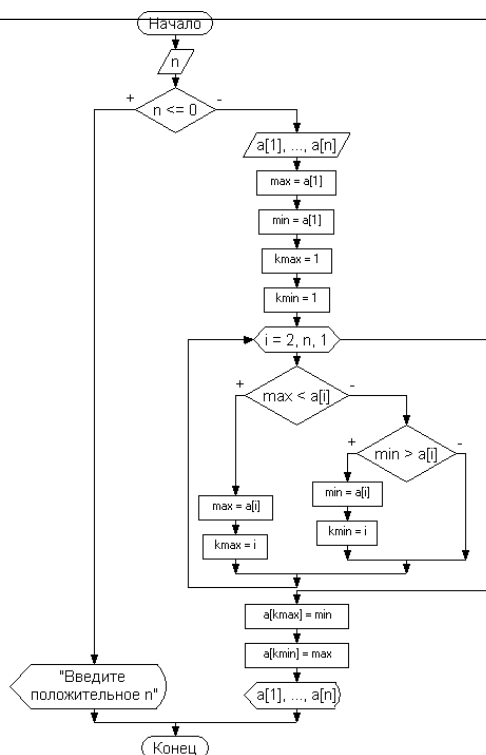


Рис. 2.1.10. Блок-схема решения задачи о перестановке максимального и минимального элементов массива

```

cout << "Введите элементы массива:" << endl;
for (int i = 0; i < n; i++)
    cin >> a[i];
// поиск максимального и минимального элементов и их
// местоположений
int max = a[0], min = a[0];
int kmax = 0, kmin = 0;
for (int i = 1; i < n; i++)
    if (max < a[i])
    {
        max = a[i]; kmax = i;
    }
    else if (min > a[i])
    {
        min = a[i]; kmin = i;
    }
  
```

Листинг 2.1.6. Программный код решения задачи 4 на языке C++
(Продолжение)

```

// изменение местоположения элементов в массиве
a[kmax] = min;
a[kmin] = max;
cout << "Массив после перестановки элементов:" << endl;
for (int i = 0; i<n; i++)
    cout << a[i] <<" ";
cout << endl;
delete[] a;
return 0;
}

```

Листинг 2.1.6. Программный код решения задачи 4 на языке C++
(Окончание)

Задача 5. Дан массив. Отсортировать элементы массива по возрастанию. Существует множество алгоритмов сортировки массивов, различных по эффективности, размерам занимаемой памяти, рекомендациям по их применению. Рассмотрим два наиболее простых и понятных алгоритма сортировки.

5.1. Метод линейного выбора

Сначала требуется найти минимальный элемент массива. Его нужно поставить на первое место. Поэтому меняем местами первый и минимальный элементы массива. После выполнения этой процедуры первый элемент массива находится на своем месте. Далее следует повторить эти же действия, исключив из рассмотрения первый элемент. В оставшемся массиве ищется минимальный элемент и устанавливается на второе место и т. д. В итоге получим отсортированный массив.

Блок-схема решения задачи 5 методом линейного выбора представлена на рис. 2.1.11, а программный код – в листинге 2.1.7.

```

#include <iostream>
#include <locale>
using namespace std;
int main(void)
{
    setlocale(LC_CTYPE, "rus");
    int n;
    int * a;
    cout << "Введите количество элементов массива n:";
    cin >> n;
    if (n <= 0)
    {
        cout << "Введите положительное n" << endl; return 0;
    }
    // выделение памяти для массива размера n
    a = new int[n];

```

Листинг 2.1.7. Программный код решения задачи 5 на языке C++
(Начало)

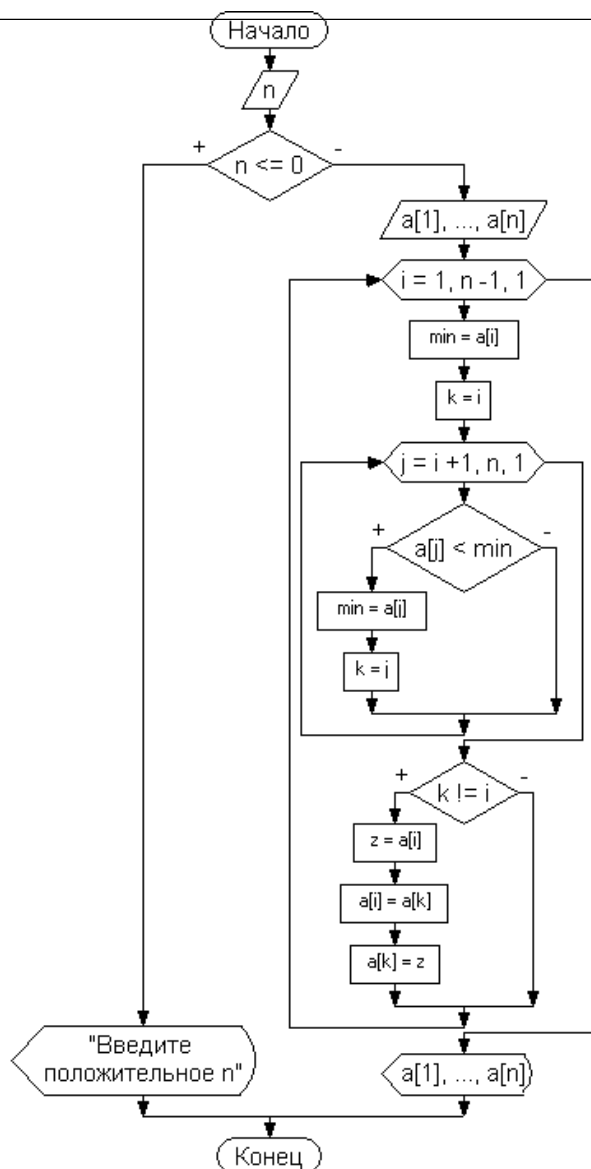


Рис. 2.1.11. Блок-схема решения задачи сортировки массива методом линейного выбора

```

cout << "Введите элементы массива:" << endl;
for (int i = 0; i < n; i++)
    cin >> a[i];
for (int i = 0; i < n - 1; i++)
{
    // поиск минимального элемента, начиная с i-го
    int min = a[i];
    int k = i;
    for (int j = i + 1; j < n; j++)
        if (a[j] < min)
        {
            min = a[j]; k = j;
        }
    // устанавливаем найденный минимум на последнее место
    if (k != i)
    {
        int z = a[i]; a[i] = a[k]; a[k] = z;
    }
}
cout << "Отсортированный массив:" << endl;
for (int i = 0; i < n; i++)
    cout << a[i] << " ";
cout << endl;
delete [] a;
return 0;
}

```

Листинг 2.1.7. Программный код решения задачи 5 на языке C++
(Окончание)

5.2. Метод «пузырька»

При сортировке массива методом «пузырька» сравниваются последовательно пары соседних элементов массива. Если упорядоченность сравниваемых элементов нарушена, меняем их местами. В результате за один проход по массиву максимальный элемент встает на последнее место (всплывает как воздушный пузырек в воде). Поэтому дальше можно рассматривать массив, количество элементов в котором на 1 меньше (исключаем из рассмотрения последний элемент). В худшем случае будет $n - 1$ таких проходов по массиву. Если порядок элементов в сравниваемых парах не был нарушен ни разу, массив является упорядоченным. Для проверки этого создается переменная-флажок с именем f , принимающая значение 1 в случае, если массив не упорядочен по возрастанию, и 0 – в противном случае.

Блок-схема решения задачи 5 методом пузырька показана на рис. 2.1.12, а программный код приведен в листинге 2.1.8.

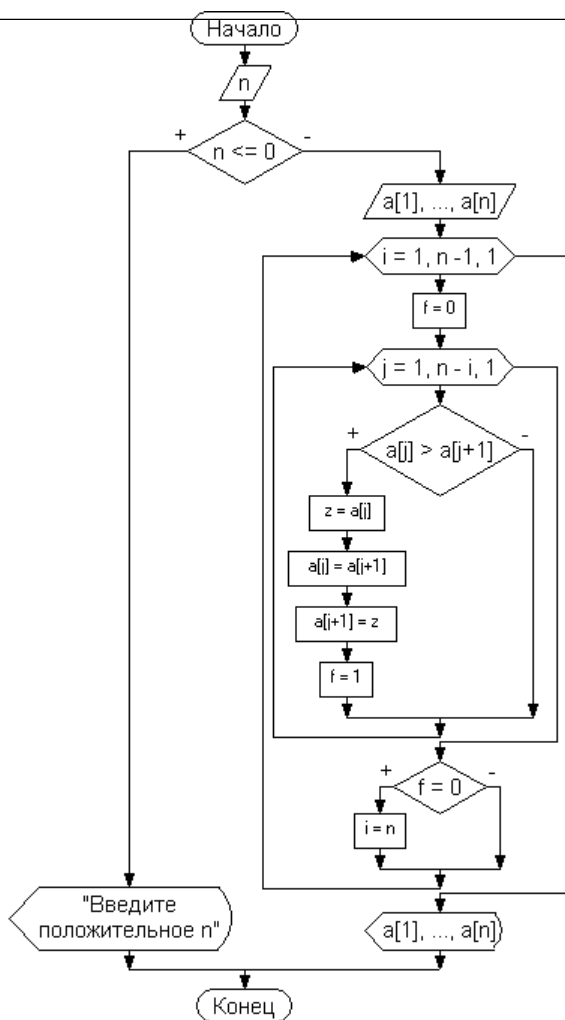


Рис. 2.1.12. Блок-схема решения задачи сортировки массива методом «пузырька»

```

using System;
namespace Example
{
    class Program
    {
        static void Main(string[] args)
        {
            int n;
            int[] a;
            Console.Write("Введите количество элементов
                               массива n:");

            n = int.Parse(Console.ReadLine());
            if (n <= 0)
            {
                Console.WriteLine("Введите положительное n.");
                return;
            }
            a = new int[n];
            Console.WriteLine("Введите элементы массива:");
            for (int i = 0; i < n; i++)
                a[i] = int.Parse(Console.ReadLine());
            // метод «пузырька»
            for (int i = 0; i < n - 1; i++)
            {
                // проход по массиву для обмена неправильно
                // расположенных соседних элементов
                int f = 0;
                for (int j = 0; j < n - i - 1; j++)
                    if (a[j] > a[j + 1])
                    {
                        int z = a[j]; a[j] = a[j + 1];
                        a[j + 1] = z; f = 1;
                    }
                // проверка того, что массив уже отсортирован
                if (f == 0)
                    break;
            }
            Console.WriteLine("Отсортированный массив:");
            for (int i = 0; i < n; i++)
                Console.Write("{0} ", a[i]);
            Console.WriteLine();
        }
    }
}

```

Листинг 2.1.8. Программный код решения задачи 5 на языке C#

Задача 6. Осуществить слияние двух отсортированных по возрастанию массивов.

В алгоритме решения этой задачи требуется учесть то, что массивы уже отсортированы. Если бы этого не было, то задача решалась

бы просто – создается новый массив, куда переписываются сначала первый из исходных массивов, потом второй массив. После этого можно провести сортировку вновь сформированного массива. Однако за счет уже имеющейся отсортированности массивов можно добиться лучшей эффективности.

Поскольку оба исходных массива упорядочены, минимальные элементы в них стоят на первом месте. Выбираем из них наименьший и заносим его во вновь создаваемый массив. Далее следует исключить этот элемент из рассмотрения посредством перехода к следующему элементу того массива, откуда был взят минимальный элемент. Повторяем эти действия со следующей парой элементов (с наименьшими из нерассмотренных элементов исходных массивов) и так до тех пор, пока не закончится один из массивов. Остатки второго массива просто переносим в конец нового массива.

Блок-схема решения задачи 6 показана на рис. 2.1.13, а программный код – в листинге 2.1.9.

```
#include <iostream>
#include <locale>
using namespace std;
int main(void)
{
    setlocale(LC_CTYPE, "rus");
    int n, m;
    int *a, *b, *c;
    cout << "Введите количество элементов массива 1: ";
    cin >> n;
    cout << "Введите количество элементов массива 2:";
    cin >> m;
    if (n <= 0 || m <= 0)
    {
        cout << "Введите положительные размеры." << endl;
        return 0;
    }
    a = new int[n];
    b = new int[m];
    c = new int[n + m];
    cout << "Введите элементы массива 1:" << endl;
    for (int i = 0; i < n; i++)
        cin >> a[i];
```

Листинг 2.1.9. Программный код решения задачи 6 на языке C++
(Начало)

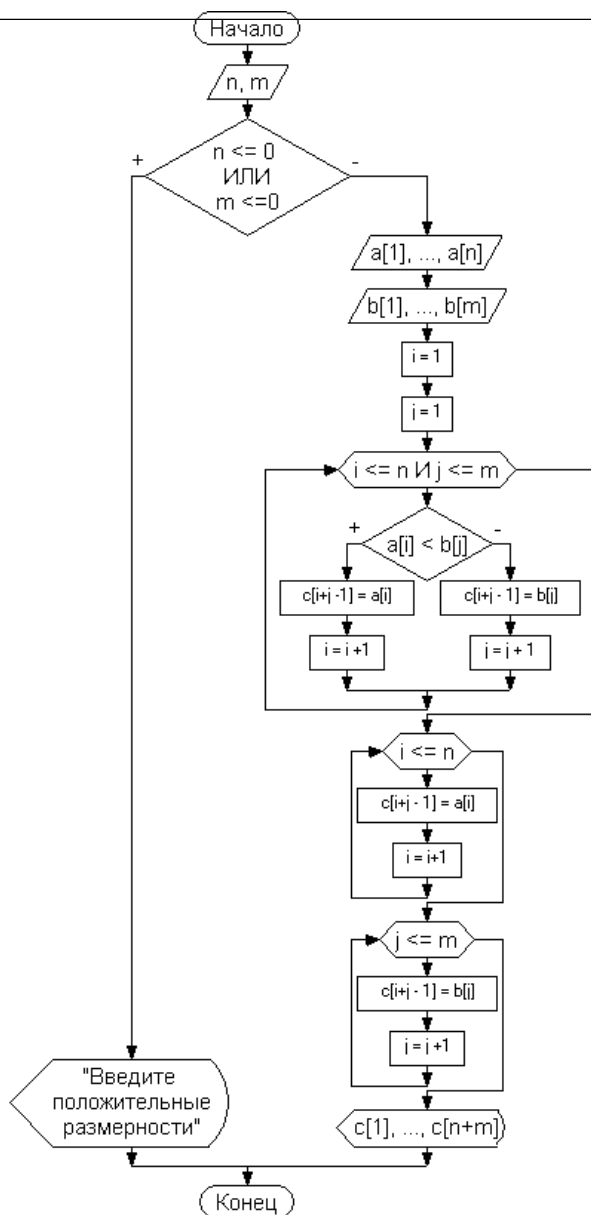


Рис. 2.1.13. Блок-схема решения задачи слияния двух отсортированных массивов

```

cout << "Введите элементы массива 2:" << endl;
for (int i = 0; i < m; i++)
    cin >> b[i];
int k;
// цикл попарного сравнения минимальных элементов
// исходных двух массивов из еще не включенных
// в массив-результат
int i = 0; k = 0;
while (i < n && k < m)
    // включаем в массив-результат элемент, который меньше
    // и сдвигаем индекс по массиву, откуда этот элемент
    if (a[i] < b[k])
    {
        c[i + k] = a[i];
        i++;
    }
    else
    {
        c[i + k] = b[k];
        k = k + 1;
    }
// один из массивов кончился, нерассмотренные элементы
// второго массива нужно поместить
// в конец массива-результата
for (; i < n; i++)
    c[i + k] = a[i];
for (; k < m; k++)
    c[i + k] = b[k];
cout << "Объединенный массив:" << endl;
for (i = 0; i < n + m; i++)
    cout << c[i] << " ";
cout << endl;
delete [] a;
delete [] b;
delete [] c;
return 0;
}

```

Листинг 2.1.9. Программный код решения задачи 6 на языке C++
(Окончание)

Задача 7. Поиск местоположения заданного элемента в отсортированном массиве методом деления пополам (алгоритм бинарного поиска).

В данном алгоритме существенную роль играет факт упорядоченности массива. Итерация алгоритма поиска заключается в следующем.

Выбираем элемент, расположенный в середине массива. Если он совпадает с искомым, то поиск прекращается. Если же этот элемент больше искомого, то в силу упорядоченности массива продолжать поиск следует в левой его половине, иначе – в правой половине массива.

Алгоритм бинарного поиска является эффективным, поскольку после каждой итерации количество рассматриваемых элементов уменьшается вдвое. Для определения границ сегмента поиска элемента создаются две переменные: l – левая граница сегмента поиска, r – правая граница сегмента поиска. На каждой из итераций цикла поиска одна из переменных l или r меняет свое значение. Поиск заканчивается тогда, когда левая и правая границы совпадут, т. е. в сегменте останется только один элемент. Если он равен искомому, выводим его индекс, иначе делаем вывод о том, что такого элемента в массиве нет.

Блок-схема решения задачи 7 показана на рис. 2.1.14, а программный код – в листинге 2.1.10.

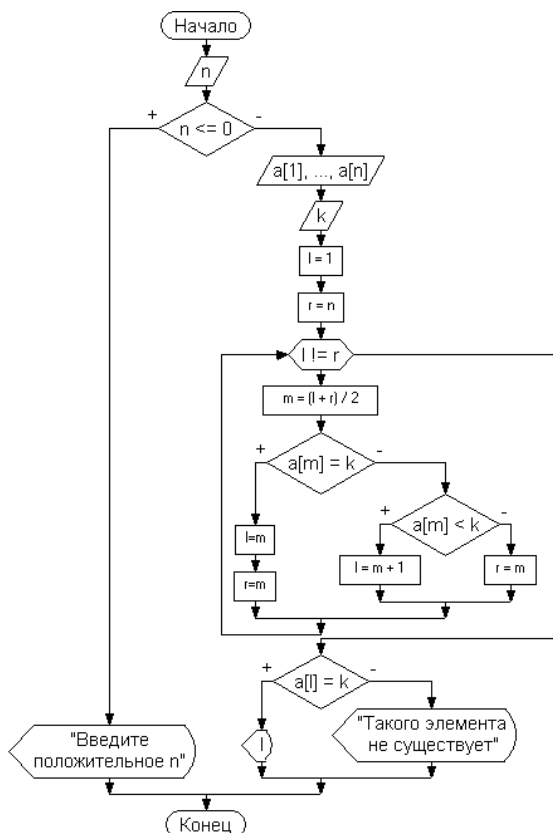


Рис. 2.1.14. Блок-схема решения задачи поиска заданного элемента в отсортированном массиве

```

using System;
namespace Example
{
    class Program
    {
        static void Main(string[] args)
        {
            int n;
            int[] a;
            Console.WriteLine("Введите количество элементов массива:");
            n = int.Parse(Console.ReadLine());
            if (n <= 0)
            {
                Console.WriteLine("Введите положительные размеры");
                return;
            }
            a = new int[n];
            Console.WriteLine("Введите элементы массива:");
            for (int i = 0; i < n; i++)
                a[i] = int.Parse(Console.ReadLine());
            int k;
            Console.WriteLine("Введите искомый элемент:");
            k = int.Parse(Console.ReadLine());
            int l = 0, r = n - 1;
            // цикл поиска элемента в интервале [l,r]
            // цикл закончится, когда останется 1 элемент
            while (l != r)
            {
                // m - середина отрезка [l,r]
                int m = (l + r) / 2;
                // если m-й элемент совпадает с искомым, выход
                if (a[m] == k)
                {
                    l = m; r = m;
                }
                else
                {
                    // определение, в какой половине интервала искать
                    if (a[m] < k)
                        l = m + 1;
                    else
                        r = m;
                }
            }
            if (a[l] == k)
                Console.WriteLine("a[{0}]={1}", l + 1, k);
            else
                Console.WriteLine("Такого элемента нет");
        }
    }
}

```

Листинг 2.1.10. Программный код решения задачи 7 на языке C#


```

#include <iostream>
#include <locale>
using namespace std;
int main(void)
{
    setlocale(LC_CTYPE, "rus");
    int k, n;
    cout << "Введите число k:";
    cin >> k;
    // знак числа - символ пробела или символ '-'
    char z = ' ';
    n = k;
    if (k < 0)
    {
        k = - k; z = '-';
    }
    // подсчет количества двоичных разрядов (переменная i)
    int i = 0, x = k;
    while (x > 0)
    {
        x = x / 2;
        i++;
    }
    // выделение памяти под массив двоичных разрядов числа
    int* a = new int[i];
    i = 0; x = k;
    // последовательное вычисление двоичных разрядов
    // путем деления на 2
    while (k > 0)
    {
        a[i] = k % 2;
        k = k / 2;
        i++;
    }
    // переворот массива двоичных разрядов
    for (int il = 0; il < i / 2; il++)
    {
        x = a[i1];
        a[i1] = a[i - i1 - 1];
        a[i - i1 - 1] = x;
    }
    // печать результата
    cout << "Двоичный код числа " << n << " = " << z ;
    for (int il = 0; il < i; il++)
        cout << a[i1];
    cout << endl;
    delete [] a;
    return 0;
}

```

Листинг 2.1.11. Программный код решения задачи 8 на языке C++

1. Дан массив целых чисел. Определить, является ли он возрастающей последовательностью.
2. Найти в массиве, состоящем из целых чисел, возрастающую подпоследовательность максимальной длины. Под термином «подпоследовательность» будем понимать некоторый набор подряд идущих элементов массива.
3. Найти в массиве целых чисел симметричную подпоследовательность максимальной длины. Под термином «подпоследовательность» будем понимать некоторый набор подряд идущих элементов массива.
4. Найти в массиве, состоящем из целых чисел, возрастающую подпоследовательность максимальной длины. Под термином «подпоследовательность» будем понимать некоторый набор элементов массива, который можно получить путем вычеркивания некоторых элементов массива.
5. Дан массив, состоящий из целых чисел. Определить количество элементов, встречающихся в нем по одному разу.
6. Дан массив, состоящий из целых чисел. Найти элемент, который встречается в массиве максимальное число раз.
7. Дан массив, состоящий из целых чисел. Определить количество различных элементов этого массива.
8. Дан массив, элементами которого являются результаты забега спортсменов на 100 м. Составить команду из 4 лучших спортсменов.
9. Даны два массива, состоящие из целых чисел. Определить количество элементов первого массива, которые содержатся также и во втором массиве.
10. Дан массив, состоящий из целых чисел. Определить, образуют ли элементы массива множество (все элементы множества должны быть уникальны).

11. Даны два массива, образующие элементы двух множеств целых чисел. Определить, равны ли эти множества (для множеств имеет значение только наличие элемента в множестве, а не порядок следования элементов в нем).

12. Даны два массива, содержащие элементы двух множеств. Получить массивы, являющиеся:

а) объединением исходных множеств (объединение содержит элементы обоих множеств, причем элементы, присутствующие в обоих множествах, включаются в их объединение в единственном экземпляре);

б) пересечением исходных множеств (пересечение двух множеств содержит только элементы, присутствующие сразу в обоих множествах);

в) разностью исходных множеств (разность двух множеств содержит те элементы первого множества, которых нет во втором множестве).

Следует реализовать две версии алгоритмов – для случая, когда массивы-множества хранят элементы в произвольном порядке, и для случая хранения элементов в отсортированном порядке. Провести сравнение этих алгоритмов.

Символьные строки – это специальный вид массивов, элементами которого являются символы (тип данных *char*). Особенность символьных строк заключается в том, что не все элементы массива могут быть заполнены. Например, для хранения фамилии студента выделяется символьный массив длины, достаточной для хранения любой фамилии (к примеру, 30 символов). Однако естественно, что разные фамилии имеют различную длину в символах, таким образом, размер содержательной части символьной строки будет разным. Поэтому нередко в языках программирования используется специальный символ, чтобы задать признак конца символьной строки (в алгоритмах он обозначается как *EOL* – end-of-line). Например, в языке C++ *EOL* – это символ `'\0'`, который принято называть «ноль-символом».

Так как символьная строка – это массив, то многие алгоритмы работы с массивами можно применять и для символьных строк. Тем не менее гораздо больше задач обработки символьных строк требуют анализировать не отдельные символы строки, а целые слова. В этом случае требуются отдельные приемы построения алгоритмов обработки строк. Для простоты будем полагать, что слова состоят из любых символов, кроме пробела и символа конца строки – `'\0'`. Слова в строке могут разделяться любым числом пробелов. Пробелы также могут находиться в начале и в конце символьной строки.

Отметим также небольшие особенности реализации символьных строк в языке программирования C#. Использование в качестве основного представления строки массива символов нехарактерно для этого языка. В нем существует специальный тип данных (класс) *string* для представления строки. Данному представлению не требуется специальный символ для обозначения конца содержательной части строки – размер строки автоматически подстраивается под ее содержимое. Узнать размер строки можно будет с помощью специальных свойств типа *string*.

Задача 1. Найти количество символов в символьной строке.

Это самая простая задача, которая демонстрирует основные инструменты языков программирования для определения размера содержательной части строки. Алгоритм для этой задачи прост: в языке C++ подсчитываем все символы до тех пор, пока не встретится символ конца строки (`'\0'`). Блок-схема решения этой задачи приведена на рис. 2.2.1, а программный код – в листинге 2.2.1. Для реализации на языке C# приведем только программный код (листинг 2.2.2), так как задача здесь решается исключительно техническими средствами этого языка.

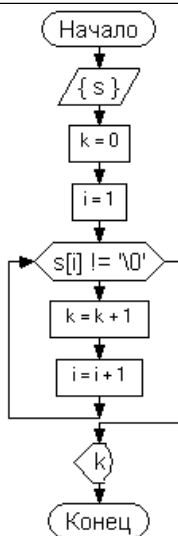


Рис. 2.2.1. Блок-схема решения задачи
о количестве символов в строке

```

include <iostream>
include <locale>

using namespace std;

int main(void)
{
    setlocale(LC_CTYPE, "rus");
    char s[100];
    // ввод символьной строки осуществляется с помощью
    // специальной функции, чтобы можно было вводить пробелы
    cout << "Введите символьную строку:" << endl;
    cin.getline(s,100);
    int k = 0;
    int i = 0;
    while (s[i] != '\0')
    {
        k++; i++;
    }
    cout << "Длина строки = " << k << endl;
}
  
```

Листинг 2.2.1. Программный код решения задачи 1 на языке C++

```

using System;

namespace Example
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Введите символьную строку");
            string s = Console.ReadLine();
            // длину строки можно получить с помощью специального
            // свойства строки Length
            Console.WriteLine("Длина строки = {0}", s.Length);
        }
    }
}

```

Листинг 2.2.2. Программный код решения задачи 1 на языке C#

Задача 2. Подсчитать количество слов в символьной строке.

Для решения этой задачи достаточно определить количество первых символов слов. Слово начинается, если текущий символ – пробел, а следующий – не пробел и не символ конца строки. Это условие не будет выполняться, если первый символ строки – начало слова. Поэтому требуется рассмотреть этот случай отдельно.

Блок-схема представлена на рис. 2.2.2. Программный код приведем для языка C++ в листинге 2.2.3, для языка C# – в листинге 2.2.4. Обратите внимание на отличие условий проверки достижения конца строки.

```

# include <iostream>
# include <clocale>

using namespace std;

int main(void)
{
    setlocale(LC_CTYPE, "rus");
    char s[100];
    cout << "Введите символьную строку:" << endl;
    cin.getline(s,100);
    // отдельная проверка на первую букву строки
    // если это буква - то посчитаем первое слово
    int k;
    if (s[0] == ' ' || s[0] == '\0') k = 0;
    else k = 1;
}

```

Листинг 2.2.3. Программный код решения задачи 2 на языке C++
(Начало)

```

int i = 0;
// цикл просмотра символов строки
while (s[i] != '\0')
{
    // выявляем очередное слово, если текущий символ - пробел,
    // а следующий - не пробел и не конец строки
    if (s[i] == ' ' && s[i + 1] != ' ' && s[i + 1] != '\0')
        k++;
    i++;
}
cout << "Количество слов в строке = " << k << endl;
}

```

Листинг 2.2.3. Программный код решения задачи 2 на языке C++
(Окончание)

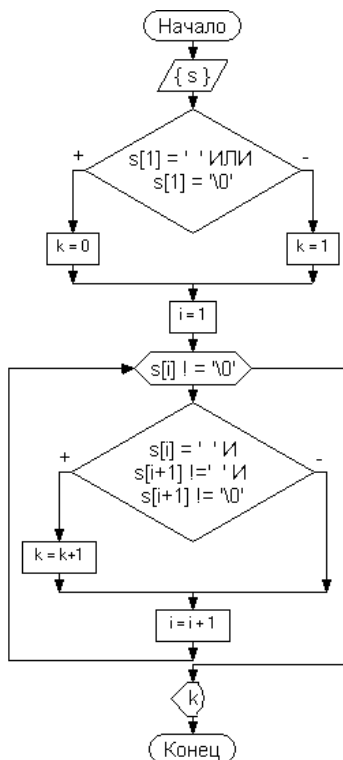


Рис. 2.2.2. Блок-схема решения задачи о количестве слов в строке

```

using System;

namespace Example
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Введите символьную строку");
            string s = Console.ReadLine();
            int k;
            int n = s.Length;          // получение длины строки
            // отдельная проверка на первую букву строки
            // если это буква - то посчитаем первое слово
            if (s[0] == ' ' || n == 0)
                k = 0;
            else k = 1;
            int i = 0;
            // цикл просмотра символов строки
            while (i < n)
            {
                // выявляем очередное слово, если текущий символ -
                // пробел, а следующий - не пробел и не конец строки
                if (s[i] == ' ' && i + 1 != n && s[i + 1] != '\0')
                    k++;
                i++;
            }
            Console.WriteLine("Количество слов в строке = {0}", k);
        }
    }
}

```

Листинг 2.2.4. Программный код решения задачи 2 на языке C#

Задача 3. Из заданной символьной строки распечатать слово максимальной длины.

Решаем эту задачу следующим образом. Последовательно в символьной строке выделяем группы «пробелы – слово». Сначала пропускаем все пробелы, предшествующие текущему слову. Затем подсчитываем количество символов в слове (двигаемся до пробела или до символа '\0'). При этом требуется запомнить позицию, с которой начинается текущее слово. Если количество символов текущего слова больше, чем в достигнутом на данный момент максимуме, запоминаем позицию начала слова и его длину. Далее переходим к рассмотрению следующей группы «пробелы – слово». Дойдя до конца строки, печатаем самое длинное слово или же сообщаем о том, что строка не содержит слов. Блок-схема решения задачи 3 представлена на рис. 2.2.3, а программный код приведен на языке C++ в листинге 2.2.5.

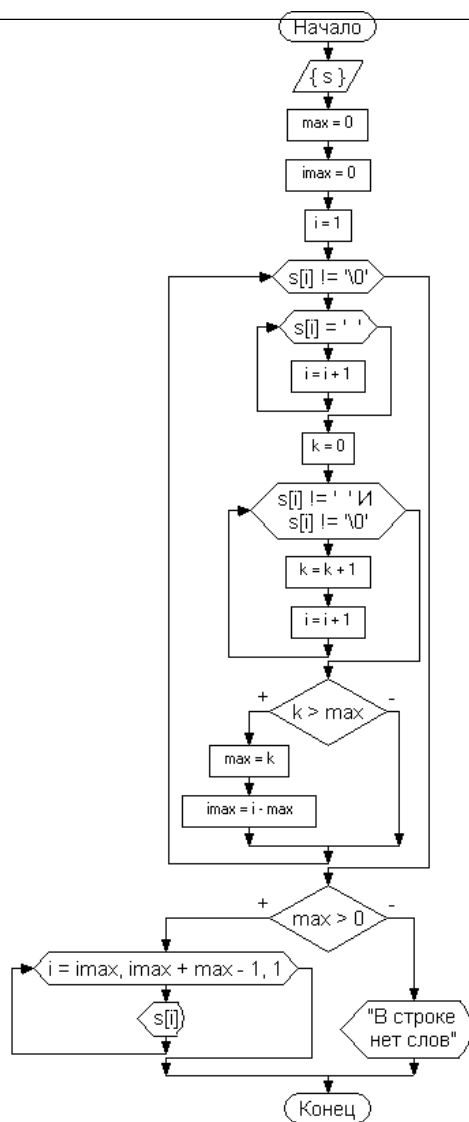


Рис. 2.2.3. Блок-схема решения задачи о слове максимальной длины

```

# include <iostream>
# include <locale>

using namespace std;

int main(void)
{
    setlocale(LC_CTYPE, "rus");
    char s[100];
    cout << "Введите строку:" << endl;
    cin.getline(s,100);
    int max = 0,      // длина максимального слова
        imax = 0;    // позиция первого символа максимал. слова
    // цикл перебора групп «пробелы - слово»
    int i = 0;
    while (s[i] != '\0')
    {
        // цикл пропуска пробелов
        while (s[i] == ' ')
            i++;
        // цикл прохода по слову - пока не найдем пробел или
        // символ конца строки, подсчитываем длину слова
        int k = 0;      // длина слова
        while (s[i] != ' ' && s[i] != '\0')
        {
            k++; i++;
        }
        // проверяем, является ли длина текущего слова
        // максимальной из уже просмотренных слов
        if (k > max)
        {
            max = k;
            // запоминаем индекс первой буквы слова
            imax = i - max;
        }
    }
    // если max осталось 0, слов в строке не было найдено
    if (max > 0)
    {
        for (i = imax; i < imax + max; i++)
            cout << s[i];
        cout << endl;
    }
    else
        cout << "В строке нет слов." << endl;
    return 0;
}

```

Листинг 2.2.5. Программный код решения задачи 3 на языке C++

Задача 4. Проверить, входит ли заданное пользователем слово в символьную строку.

При решении этой задачи сначала требуется получить количество символов в заданном слове. Далее осуществляется просмотр слов символьной строки (алгоритм выделения слов из строки рассмотрен при решении задачи 3). Каждое выделенное слово побуквенно сравнивается с заданным словом. Требуется, чтобы не только совпали все буквы слова, но и следом за ним в строке стоял символ-разделитель, т. е. пробел или символ конца строки. Например, «сад» и «садовник» – это два разных слова. Строка, содержащая слово «садовник», не содержит слово «сад».

Блок-схема решения задачи 4 представлена на рис. 2.2.4. Программную реализацию алгоритма приведем на языке программирования C# в листинге 2.2.6.

```
using System;
namespace Example
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Введите символьную строку");
            string s = Console.ReadLine();
            int n = s.Length;           // длина строки
            Console.Write("Введите слово:");
            string c = Console.ReadLine();
            int k = c.Length;           // длина искомого слова
            int i = 0,
                f = 0;                 // переменная-флажок
                                     // f = 1, когда будет найдено искомое слово
            // цикл просмотра строки, пока не будет найдено слово
            // или не кончится символьная строка
            while (i < n && f == 0)
            {
                // пропускаем пробелы перед словом
                while (i < n && s[i] == ' ')
                    i++;
                // цикл прохода по слову и подсчет совпадающих
                // символов с символами заданного слова
                // выход из цикла, когда либо кончится строка,
                // либо найдем несовпадающий символ, либо
                // будут просмотрены k символов - длина искомого слова
                int t = 0;
                while (i < n && t < k && s[i] == c[t])
                {
                    i++; t++;
                }
            }
        }
    }
}
```

Листинг 2.2.6. Программный код решения задачи 4 на языке C# (Начало)

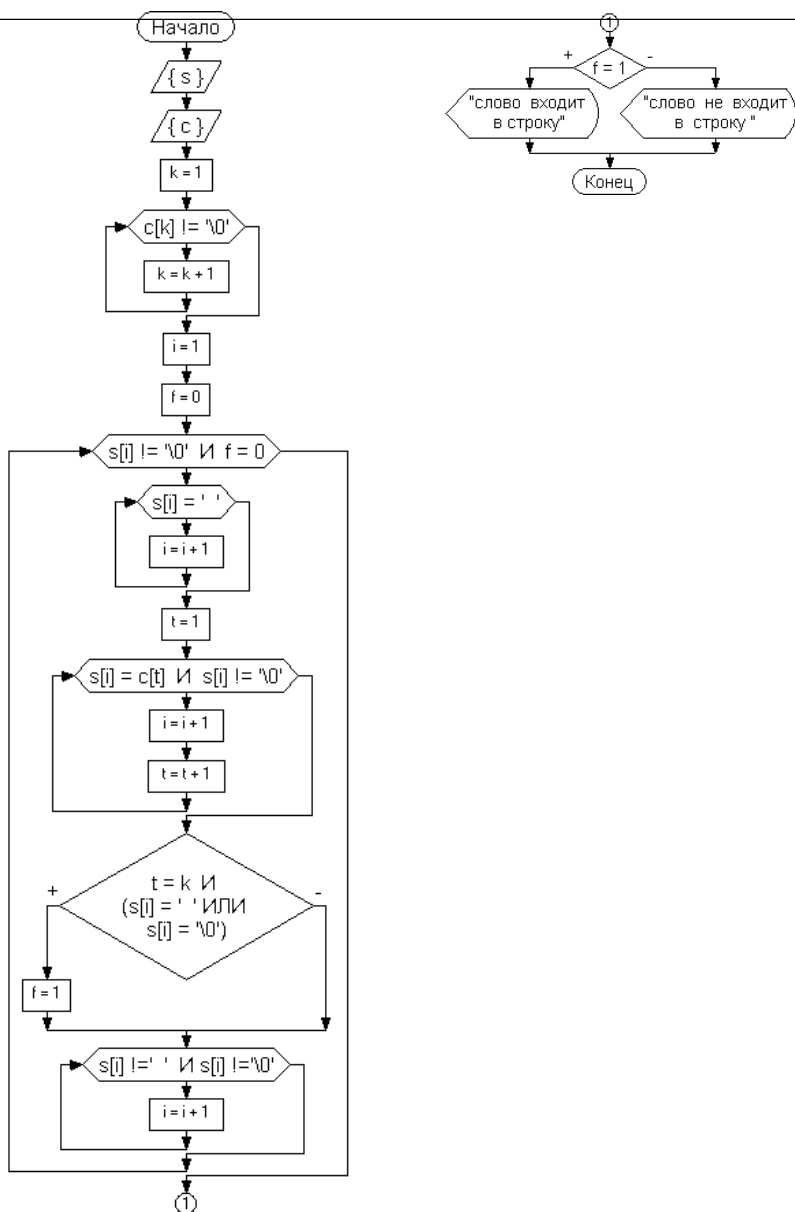


Рис. 2.2.4. Блок-схема решения задачи о вхождении заданного слова в символьную строку

```

        // слово найдено, если длина слова совпала с длиной
        // искомого слова и слово закончилось – далее конец
        // строки или пробел
        if (t == k && (i == n || s[i] == ' '))
            f = 1;
        // пропускаем символы в окончании слова, если слово
        // оказалось длиннее искомого слова
        while (i != n && s[i] != ' ') i++;
    }
    // печать результата
    if (f == 1)
        Console.WriteLine("Слово {0} входит в строку", c);
    else
        Console.WriteLine("Слово {0} не входит в строку", c);
}
}
}

```

Листинг 2.2.6. Программный код решения задачи 4 на языке C#
(Окончание)

Задача 5. В заданной символьной строке оставить между словами по одному пробелу и удалить лишние пробелы в начале и в конце строки.

Алгоритм решения задачи таков. Двигаясь по символьной строке, подсчитываем суммарное количество лишних пробелов между словами (переменная k). Дойдя до очередного слова, сдвигаем его на k позиций влево, оставив между словами только один пробел. Затем повторяем процедуру до тех пор, пока не закончится строка. Отдельно удаляем пробел после последнего слова, если будут «лишние» пробелы в конце строки.

Блок-схема решения задачи представлена на рис. 2.2.5. Программную реализацию приведем для языка C++ (в листинге 2.2.7) и для языка C# (в листинге 2.2.8).

В этой задаче для реализации на языке программирования C# уже понадобится перевести представление строки из типа *string* в массив из символов (*char[]*), так как частично содержимое переменной типа *string* изменить нельзя. В конце алгоритма удаления лишних пробелов будет создана новая строка на основании обновленного массива символов, которая и будет строкой-результатом.

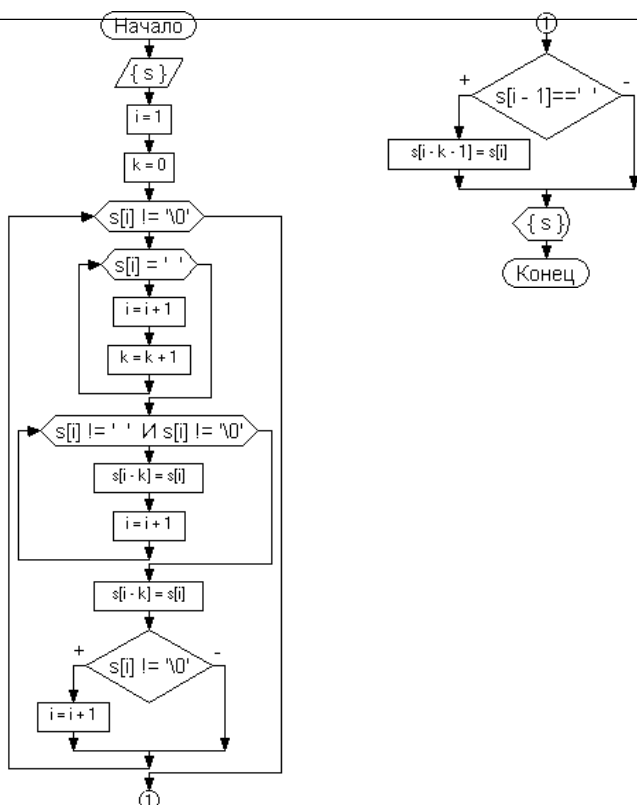


Рис. 2.2.5. Блок-схема решения задачи об удалении «лишних» пробелов

```

# include <iostream>
# include <locale>

using namespace std;

int main(void)
{
    setlocale(LC_CTYPE, "rus");
    char s[100];
    cout << "Введите строку:" << endl;
    cin.getline(s,100);
    int i = 0, // индекс для просмотра символов строки
        k = 0; // количество "лишних" пробелов

```

Листинг 2.2.7. Программный код решения задачи 5 на языке C++
(Начало)

```

// цикл прохода по символам строки
while (s[i] != '\0')
{
    // пропуск "лишних" пробелов перед словом и учет их в k
    while (s[i] == ' ')
    {
        k++; i++;
    }
    // проход по символам слова и их смещение на количество
    // "лишних" пробелов, найденных с начала работы алгоритма
    while (s[i] != ' ' && s[i] != '\0')
    {
        // смещение символа влево, удаляя пробелы
        s[i - k] = s[i];
        i++;
    }
    // перенос влево символа после слова ("нелишний" пробел
    // или символ конца строки
    s[i - k] = s[i];
    // если строка не закончилась, смещение на следующий символ
    // i-й символ - "нелишний" пробел, он уже учтен
    if (s[i] != '\0')
        i++;
}
cout << "Полученная строка:" << s << endl;
return 0;
}

```

Листинг 2.2.7. Программный код решения задачи 5 на языке C++
(Окончание)

```

using System;

namespace Example
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Введите символьную строку");
            string s = Console.ReadLine();
            int n = s.Length;           // длина строки
            int i = 0, // индекс для просмотра символов строки
                k = 0; // количество "лишних" пробелов
            // получение массива символов строки
            char[] s1 = s.ToCharArray();
            // далее будем работать с массивом s1 - реализация
            // аналогична реализации на C++ с разницей в
            // формулировке условий окончания строки

```

Листинг 2.2.8. Программный код решения задачи 5 на языке C# (Начало)

```

// цикл прохода по массиву символов s1
while (i < n)
{
    // подсчет "лишних" пробелов
    while (i < n && s[i] == ' ')
    {
        k++; i++;
    }
    // сдвиг слова влево на найденное количество
    // "лишних" пробелов
    while (i < n && s1[i] != ' ')
    {
        s1[i - k] = s1[i];
        i++;
    }
    // если слово не последнее, сдвиг следующего
    // за ним пробела
    if (i < n)
    {
        s1[i - k] = s1[i];
        i++;
    }
}
// создание символьной строки из массива символов,
// начиная с 0-го символа
// в количестве длина исходной строки -
// число удаленных символов
s = new string(s1, 0, n - k);
Console.WriteLine("Полученная строка:");
Console.WriteLine(s);
}
}
}

```

Листинг 2.2.8. Программный код решения задачи 5 на языке C#
(Окончание)

Задача 6. В заданной символьной строке поменять местами слова минимальной и максимальной длины.

Решение задачи разбивается на две части. Первая часть заключается в поиске слов максимальной и минимальной длины (алгоритм рассматривался при решении задачи 3).

Вторая часть заключается собственно в перестановке найденных слов. Эту подзадачу также условно можно разделить на два этапа. Пусть длина минимального слова хранится в переменной *min*, а максимально – в переменной *max*. Сначала меняем местами первые буквы слов в количестве *min* штук. Затем нужно вставить на требуемое место «хвостик» от самого длинного слова. Эту вставку осуществляем по одному символу, смещая ту часть строки, которая находится между

словами, на один символ вправо или влево соответственно в зависимости от того, какое из слов – минимальное или максимальное – встретилось раньше. На освободившееся место устанавливаем очередной символ из «хвостика». Повторяем такую последовательность действий столько раз, какова разница между длинами слов.

Блок-схема решения этой задачи представлена на рис.2.2.6 и 2.2.7, а программный код приведен в листинге 2.2.9.

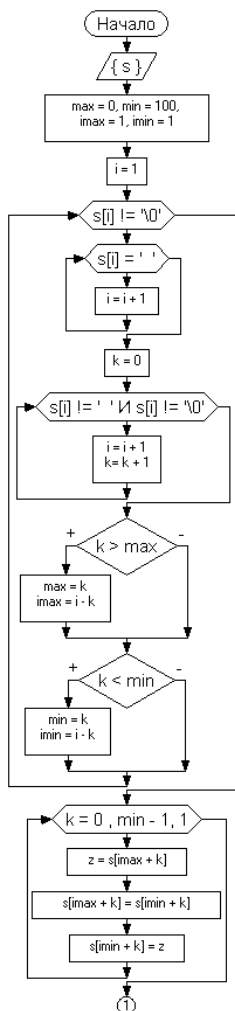


Рис. 2.2.6. Начало блок-схемы решения задачи о перестановке слов максимальной и минимальной длины

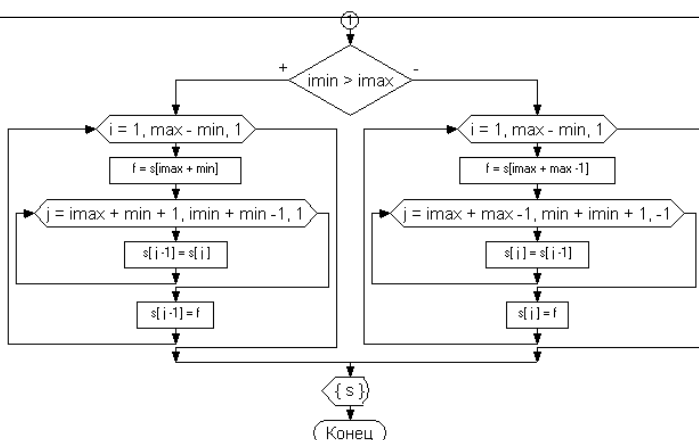


Рис. 2.2.7. Окончание блок-схемы решения задачи о перестановке слов максимальной и минимальной длины

```

# include <iostream>
# include <locale>
using namespace std;
int main(void)
{
    setlocale(LC_CTYPE, "rus");
    char s[100];
    cout << "Введите строку:" << endl;
    cin.getline(s,100);
    int imax = 0, // позиция слова максимальной длины
        imin = 0, // позиция слова минимальной длины
        max = 0, // длина максимального слова
        min = 100, // длина минимального слова
                // (начальное значение - длина строки)
    i = 0, k;
    // цикл прохода по группам «пробелы - слово»
    while (s[i] != '\0')
    {
        // выделяем пробелы
        while (s[i] == ' ') i++;
        // выделяем слово и подсчитываем его длину
        k = 0;
        while (s[i] != ' ' && s[i] != '\0')
        { i++; k++; }
        // проверка, имеет ли слово максимальную длину
        if (k > max)
        {
            max = k; imax = i - k;
        }
    }
}
  
```

Листинг 2.2.9. Программный код решения задачи 6 на языке C++
(Начало)

```

// проверка, имеет ли слово минимальную длину
if (k < min)
{
    min = k; imin = i - k;
}
}
// меняем местами первые символы слов в количестве min
for (k = 0; k < min; k++)
{
    char z = s[imax + k];
    s[imax + k] = s[imin + k];
    s[imin + k] = z;
}
// ставим на место "хвостик" максимального слова
if (imin > imax)
{
    // слово максимальной длины стоит в строке раньше,
    // поэтому нужен сдвиг символов влево
    for (i = 0; i < max - min; i++)
    {
        // запоминаем первый из неперемещенных символов
        // слова максимальной длины
        int f = s[imax + min];
        // сдвиг всех символов между словами
        int j = imax + min + 1;
        for (; j < imin + min; j++)
            s[j - 1] = s[j];
        // установка символа из f на свое место
        s[j - 1] = f;
    }
}
else
{
    // слово минимальной длины стоит в строке раньше,
    // поэтому нужен сдвиг символов вправо
    for (i = 0; i < max - min; i++)
    {
        // запоминаем последний из неперемещенных символов
        // слова максимальной длины
        int f = s[imax + max - 1];
        // сдвиг всех символов между словами
        int j = imax + max - 1;
        for (; j >= min + imin + 1; j--)
            s[j] = s[j - 1];
        // установка символа из f на свое место
        s[j] = f;
    }
}
cout << "Полученная строка: " << s << endl;
return 0;
}

```

Листинг 2.2.9. Программный код решения задачи 6 на языке C++
(Окончание)

1. Дана символьная строка. Распечатать все слова, начинающиеся на заданную букву.
2. Дана символьная строка. Найти в строке симметричное слово (палиндром) максимальной длины.
3. Дана символьная строка. Провести в ней реверс слова максимальной длины (реверс слова означает, что символы слова должны идти в обратном порядке).
4. Дана символьная строка. Удалить из каждого слова все последующие вхождения его первой буквы.
5. Дана символьная строка. Удалить из строки слова длиной менее трех букв.
6. Дана символьная строка. Удалить из строки все слова максимальной длины.
7. Дана символьная строка. Провести в ней реверс слов (в строке-результате слова должны следовать в обратном порядке по сравнению с исходной строкой).
8. Даны две символьные строки. Первая строка содержит пары «слово – синоним». Во второй строке заменить все слова на их синонимы, если таковы найдены.
9. Дана символьная строка. Определить слово с максимальной долей гласных букв (процент гласных букв в слове).
10. Дана символьная строка. Отсортировать в строке слова по возрастанию длин слов.

Двумерные массивы (матрицы) являются примером использования табличной структуры данных. Данные здесь упорядочиваются по двум измерениям – строкам и столбцам. Таким образом, для обозначения элемента матрицы используются два индекса – индексы строки и столбца, на пересечении которых находится этот элемент. Например, чтобы обратиться к элементу матрицы a , находящемуся в 3-й строке и 1-м столбце, надо использовать следующую запись: $a[3][1]$. Для того чтобы просмотреть все элементы двумерного массива, требуется «пробежаться» по всем строкам, а внутри каждой строки – по столбцам. Поэтому для ввода элементов матрицы используют два цикла.

При использовании двумерного массива в программах на языках C++ и C# следует помнить, что, как и в массивах, нумерация строк и столбцов двумерного массива осуществляется, начиная с 0, поскольку индексы строк и столбцов также трактуются как смещения в памяти.

Задача 1. Транспонировать квадратную матрицу (поменять местами элементы так, чтобы строки матрицы стали столбцами, а столбцы – строками).

Для транспонирования матрицы достаточно поменять местами элементы, стоящие симметрично относительно главной диагонали. Элементы главной диагонали расположены так, что индексы строки и столбца у них совпадают.

Будем перебирать те элементы из симметричных пар, которые находятся ниже главной диагонали (у них номер столбца меньше номера строки). Поэтому осуществляем перебор всех строк (переменная x) и для каждой строки перебор столбцов (переменная y) от 1 до $x - 1$. Для каждой пары индексов (x, y) меняем местами элементы $a[x][y]$ и $a[y][x]$.

При построении блок-схемы решения задачи 1 для ввода и вывода матрицы будем использовать следующие группы блоков (рис. 2.3.1 и 2.3.2):

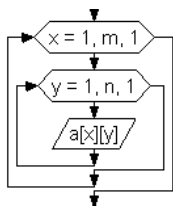


Рис. 2.3.1. Блок ввода матрицы

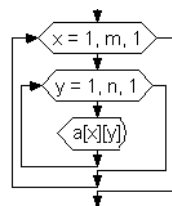


Рис. 2.3.2. Блок печати матрицы

При решении следующих задач заменим их более короткой записью (рис. 2.3.3 и 2.3.4):



Рис. 2.3.3. Блок ввода матрицы

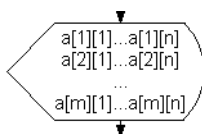


Рис. 2.3.4. Блок печати матрицы

Блок-схема решения задачи 1 представлена на рис. 2.3.5, а программный код приведен на языке C++ в листинге 2.3.1, а также на языке C# в листинге 2.3.2, чтобы продемонстрировать основные технологические особенности работы с матрицами в этих языках программирования.

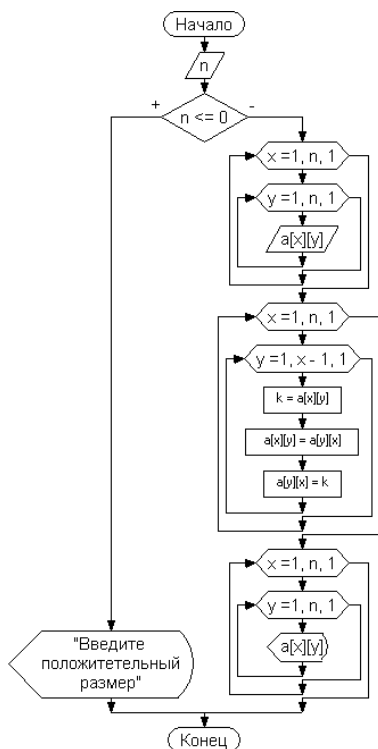


Рис. 2.3.5. Блок-схема решения задачи о транспонировании квадратной матрицы

```

#include <iostream>
#include <locale>

using namespace std;

int main(void)
{
    setlocale(LC_CTYPE, "rus");
    int n;
    cout << "Введите размер матрицы n:";
    cin >> n;
    if (n <= 0)
    {
        cout << "Введите положительный размер." << endl;
        return 0;
    }
    // выделение памяти под матрицу
    // a - массив адресов строк матрицы
    int **a = new int*[n];
    for (int x = 0; x < n; x++)
    {
        // выделение памяти под x-ю строку матрицы
        a[x] = new int[n];
    }
    cout << "Введите элементы матрицы:" << endl;
    for (int x = 0; x < n; x++)
        for (int y = 0; y < n; y++)
            cin >> a[x][y];
    // цикл транспонирования матрицы
    for (int x = 0; x < n; x++)
        for (int y = 0; y < x; y++)
        {
            int k = a[x][y];
            a[x][y] = a[y][x];
            a[y][x] = k;
        }
    cout << "Транспонированная матрица:" << endl;
    for (int x = 0; x < n; x++)
    {
        for (int y = 0; y < n; y++)
            cout << a[x][y] << "\t";
        cout << endl;
    }
    // освобождение памяти
    for (int x = 0; x < n; x++)
        delete [] a[x];
    delete [] a;
    return 0;
}

```

Листинг 2.3.1. Программный код решения задачи 1 на языке C++

В программном коде на языке программирования C# обратим внимание на упрощение записи операторов выделения памяти и отсутствие необходимости освобождать выделенную память. Как и в случае с одномерными массивами, освобождение памяти осуществляется с помощью сборщика мусора.

```
using System;

namespace Example
{
    class Program
    {
        static void Main(string[] args)
        {
            int n;
            Console.Write("Введите размер матрицы n:");
            n = int.Parse(Console.ReadLine());
            if (n <= 0)
            {
                Console.WriteLine("Введите положительный размер");
                return;
            }
            // выделение памяти под матрицу
            int[,] a = new int[n, n];
            Console.WriteLine("Введите элементы матрицы:");
            for (int x = 0; x < n; x++)
                for (int y = 0; y < n; y++)
                    a[x,y] = int.Parse(Console.ReadLine());
            // цикл перебора элементов, расположенных
            // ниже главной диагонали
            for (int x = 0; x < n; x++)
                for (int y = 0; y < x; y++)
                {
                    int k = a[x, y];
                    a[x, y] = a[y, x];
                    a[y, x] = k;
                }
            Console.WriteLine("Транспонированная матрица:");
            for (int x = 0; x < n; x++)
            {
                for (int y = 0; y < n; y++)
                    Console.Write("{0}\t", a[x, y]);
                Console.WriteLine();
            }
        }
    }
}
```

Листинг 2.3.2. Программный код решения задачи 1 на языке C#

Задача 2. Дана прямоугольная матрица. Все максимальные элементы матрицы заменить на 0.

Перебирая все элементы матрицы, находим максимальный элемент (аналогично поиску максимума в одномерном массиве). Далее еще раз обходим всю матрицу и заменяем элемент нулем при его совпадении с максимумом.

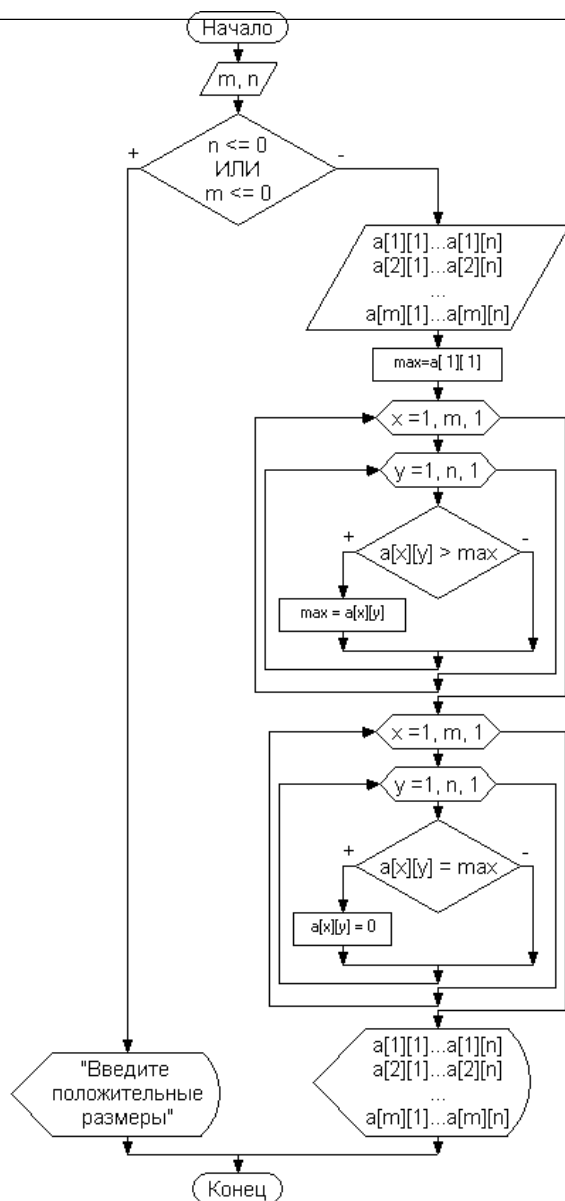
Блок-схема представлена на рис. 2.3.6, а программный код на языке C++ приведен в листинге 2.3.3.

```
# include <iostream>
# include <locale>

using namespace std;

int main(void)
{
    setlocale(LC_CTYPE, "rus");
    int m, n;
    cout << "Введите количество строк матрицы:";
    cin >> m;
    cout << "Введите количество столбцов матрицы:";
    cin >> n;
    if (n <= 0 || m <= 0)
    {
        cout << "Введите положительные размеры" << endl;
        return 0;
    }
    // выделение памяти под матрицу
    int **a = new int*[m];
    for (int x = 0; x < m; x++)
        a[x] = new int[n];
    cout << "Введите элементы матрицы:" << endl;
    for (int x = 0; x < m; x++)
        for (int y = 0; y < n; y++)
            cin >> a[x][y];
    // поиск максимального элемента в матрице
    int max = a[0][0];
    for (int x = 0; x < m; x++)
        for (int y = 0; y < n; y++)
            if (a[x][y] > max)
                max = a[x][y];
    // цикл прохода по матрице и замены максимальных элементов 0
    for (int x = 0; x < m; x++)
        for (int y = 0; y < n; y++)
            if (a[x][y] == max)
                a[x][y] = 0;
```

Листинг 2.3.3. Программный код решения задачи 2 на языке C++
(Начало)



```

cout << "Полученная матрица:" << endl;
for (int x = 0; x < m; x++)
{
    for (int y = 0; y < n; y++)
        cout << a[x][y] << "\t";
    cout << endl;
}
for (int x = 0; x < m; x++)
    delete [] a[x];
delete [] a;
return 0;
}

```

Листинг 2.3.3. Программный код решения задачи 2 на языке C++
(Окончание)

Задача 3. Найти произведение двух матриц с размерами $m \times n$ и $k \times t$.

По правилам перемножения матриц количество столбцов левой матрицы-операнда должно совпадать с количеством строк правой матрицы-операнда. Поэтому сначала требуется проверить это условие.

Если размеры были введены корректно, осуществляется перемножение. Каждый элемент матрицы-результата вычисляется как скалярное произведение соответствующей строки первой матрицы на столбец второй матрицы:

$$c_{xy} = \sum_{p=1}^n a_{xp} b_{py}.$$

Поэтому при решении задачи создаются три цикла: первые два перебирают элементы результирующей матрицы (по переменной x – цикл перебора строк матрицы, по переменной y – цикл перебора столбцов матрицы), а третий используется для вычисления самого элемента (по переменной p).

Блок-схема решения задачи 3 представлена на рис. 2.3.7, а программный код на языке программирования C# – в листинге 2.3.4.

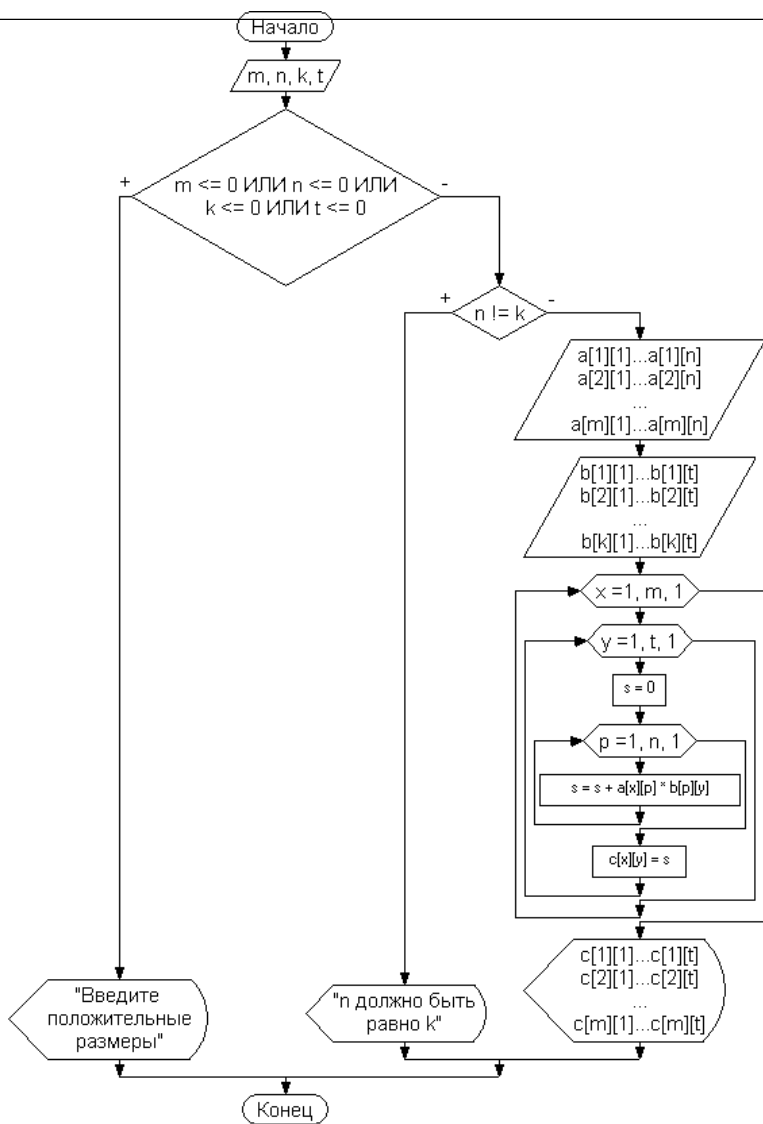


Рис. 2.3.7. Блок-схема решения задачи о перемножении матриц

```

using System;

namespace Example
{
    class Program
    {
        static void Main(string[] args)
        {
            int m,n,k,t;
            Console.WriteLine("Введите количество строк матрицы A:");
            m = int.Parse(Console.ReadLine());
            Console.WriteLine("Введите количество столбцов матрицы A:");
            n = int.Parse(Console.ReadLine());
            Console.WriteLine("Введите количество строк матрицы B:");
            k = int.Parse(Console.ReadLine());
            Console.WriteLine("Введите количество столбцов матрицы B:");
            t = int.Parse(Console.ReadLine());
            if( m<= 0 || n <= 0 || k <= 0 || t <= 0 )
            {
                Console.WriteLine("Введите положительные размеры");
                return;
            }
            // проверка возможности операции перемножения матриц
            if(n != k)
            {
                Console.WriteLine("n должно быть равно k");
                return;
            }
            // выделение памяти под матрицы-операнды
            // и матрицу-результат
            int [,] a = new int [m, n];
            int [,] b = new int [k, t];
            int [,] c = new int [m, t];
            Console.WriteLine("Введите матрицу A:");
            for(int x = 0; x < m; x++)
                for(int y = 0; y < n; y++)
                    a[x, y] = int.Parse(Console.ReadLine());
            Console.WriteLine("Введите матрицу B:");
            for(int x = 0; x < k; x++)
                for(int y = 0; y < t; y++)
                    b[x, y] = int.Parse(Console.ReadLine());
            // вычисление произведения матриц
            for(int x = 0; x < m; x++)
                for(int y = 0; y < t; y++)
                {
                    // вычисление очередного элемента матрицы-результата
                    int s=0;
                    for(int p = 0; p < n; p++)
                        s = s + a[x, p]*b[p, y];
                    c[x, y]=s;
                }
        }
    }
}

```

Листинг 2.3.4. Программный код решения задачи 3 на языке C# (Начало)

```

        Console.WriteLine("Полученная матрица:");
        for (int x = 0; x < m; x++)
        {
            for (int y = 0; y < t; y++)
                Console.Write("{0}\t", c[x, y]);
            Console.WriteLine();
        }
    }
}

```

Листинг 2.3.4. Программный код решения задачи 3 на языке C#
(Окончание)

Задача 4. Дана квадратная матрица. Поменять в ней местами элементы главной и побочной диагоналей.

Здесь сделаем только одно пояснение: элемент, находящийся в строке с номером x на главной диагонали, имеет индексы (x, x) , а соответствующий элемент побочной диагонали – $(x, n - x + 1)$ (при нумерации с 0 – $(x, n - x - 1)$). Поэтому требуется просто перебрать строки матрицы и поменять в них местами элементы, имеющие такие индексы. Блок-схема решения задачи представлена на рис. 2.3.8, а программный код на языке C++ приведен в листинге 2.3.5.

```

#include <iostream>
#include <locale>
using namespace std;

int main(void)
{
    setlocale(LC_CTYPE, "rus");
    int n;
    cout << "Введите количество строк и столбцов матрицы:";
    cin >> n;
    if (n <= 0)
    {
        cout << "Введите положительный размер" << endl; return 0;
    }
    // выделение памяти под матрицу
    int **a = new int*[n];
    for (int x = 0; x < n; x++)
        a[x] = new int[n];
    cout << "Введите элементы матрицы:" << endl;
    for (int x = 0; x < n; x++)
        for (int y = 0; y < n; y++)
            cin >> a[x][y];
}

```

Листинг 2.3.5. Программный код решения задачи 4 на языке C++
(Начало)

```

// проход по строкам и перестановка элементов диагоналей
for(int x = 0; x < n; x++)
{
    int k = a[x][x];
    a[x][x] = a[x][n - x - 1];
    a[x][n - x - 1] = k;
}
cout << "Полученная матрица:" << endl;
for (int x = 0; x < m; x++)
{
    for (int y = 0; y < n; y++)
        cout << a[x][y] << "\t";
    cout << endl;
}
for (int x = 0; x < m; x++)
    delete [] a[x];
delete [] a;
return 0;
}

```

Листинг 2.3.5. Программный код решения задачи 4 на языке C++
(Окончание)

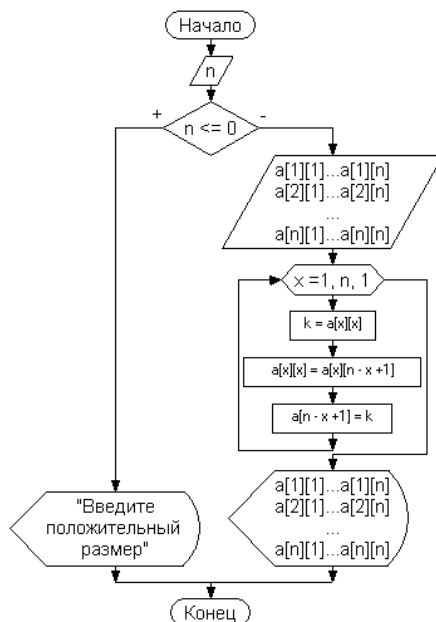


Рис. 2.3.8. Блок-схема решения задачи о перестановке диагоналей квадратной матрицы

Задача 5. Заполнить двумерный массив размера $n \times n$ числами $1, 2, \dots, n^2$ по спирали, например при $n = 4$ матрица должна выглядеть следующим образом:

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 12 & 13 & 14 & 5 \\ 11 & 16 & 15 & 6 \\ 10 & 9 & 8 & 7 \end{pmatrix}$$

Заполнение одного витка спирали состоит из четырех циклов, определяющих последовательное заполнение элементов в направлениях слева направо, сверху вниз, справа налево и снизу вверх. Границы заполнения в соответствующей строке или столбце определяются элементами, стоящими на главной и побочной диагоналях. В случае нечетного значения n требуется заполнить последний элемент отдельно, поскольку он стоит на пересечении главной и побочной диагоналей.

Блок-схема решения этой задачи представлена на рис. 2.3.9, а программный код на языке C# – в листинге 2.3.6.

```
using System;

namespace Example
{
    class Program
    {
        static void Main(string[] args)
        {
            int n;
            Console.Write("Введите размер матрицы n:");
            n = int.Parse(Console.ReadLine());
            if (n <= 0)
            {
                Console.WriteLine("Введите положительный размер");
                return;
            }
            int[,] a = new int[n, n];
            int x = 1, //номер строки очередного элемента (с 1)
                y = 1, // номер столбца очередного элемента (с 1)
                k = 1; // значение очередного элемента
```

Листинг 2.3.6. Программный код решения задачи 5 на языке C#
(Начало)

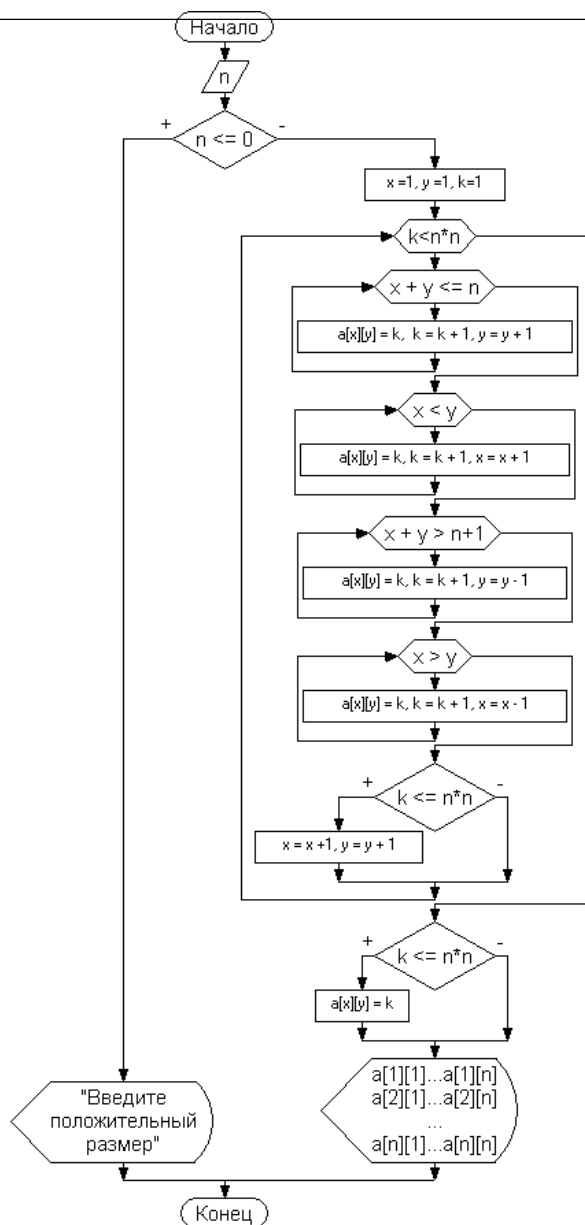


Рис. 2.3.9. Блок-схема решения задачи о заполнении квадратной матрицы элементами по спирали

```

// должно быть заполнено n*n элементов
while(k < n * n)
{
    // направление слева направо
    // заполняем от главной до побочной диагонали
    while(x + y <= n)
    {
        a[x - 1, y - 1] = k;
        k++; y++;
    }
    // направление сверху вниз
    // заполняем от побочной до главной диагонали
    while(x < y)
    {
        a[x - 1, y - 1] = k;
        k++; x++;
    }
    // направление справа налево
    // заполняем от главной до побочной диагонали
    while(x + y > n + 1)
    {
        a[x - 1, y - 1] = k;
        k++; y--;
    }
    // направление снизу вверх
    // заполняем от главной до побочной диагонали
    while(x > y)
    {
        a[x - 1, y - 1] = k;
        k++; x--;
    }
    // если это не последний виток спирали,
    // увеличиваем индексы
    if (k <= n * n)
    {
        x++; y++;
    }
}
// ставим последний элемент,
// если он еще не установлен (n - нечетно)
if(k <= n * n) a[x - 1, y - 1] = k;
Console.WriteLine("Полученная матрица:");
for (x = 0; x < n; x++)
{
    for (y = 0; y < n; y++)
        Console.Write("{0}\t", a[x, y]);
    Console.WriteLine();
}
}
}
}

```

Листинг 2.3.6. Программный код решения задачи 5 на языке C#
(Окончание)

Задача 6. Вычислить определитель квадратной матрицы, приведя ее к треугольному виду.

Треугольной матрицей является такая квадратная матрица, в которой выше или ниже главной диагонали все элементы равны нулю. Определитель треугольной матрицы равен произведению элементов, стоящих на главной диагонали. Поэтому основная задача состоит в приведении матрицы к треугольному виду с помощью эквивалентных преобразований (не меняющих определитель матрицы).

Будем приводить матрицу к виду верхней треугольной матрицы (элементы ниже главной диагонали равны 0). Последовательно для каждого столбца проводим следующую процедуру. Если элемент главной диагонали в этом столбце равен нулю, ищем ненулевые элементы в этом столбце в строках ниже. Если поиск окончится неудачей, то определитель равен 0. При нахождении ненулевого элемента в рассматриваемом столбце меняем местами две строки так, чтобы ненулевой элемент оказался на главной диагонали. При такой смене строк меняется знак определителя на противоположный.

Найдя ненулевой элемент на главной диагонали, можно исключить элементы, стоящие ниже его в том же столбце (сделать их 0). Для этого последовательно из нижних строк вычитается строка с рассматриваемым элементом главной диагонали, умноженная на коэффициент, гарантирующий, что все элементы ниже рассматриваемого будут равны 0. При таких преобразованиях значение определителя сохранится.

Блок-схема решения этой задачи представлена на рис. 2.3.10, а программный код – в листинге 2.3.7.

```
# include <iostream>
# include <locale>
using namespace std;
int main(void)
{
    setlocale(LC_CTYPE, "rus");
    int n;
    cout << "Введите количество строк и столбцов матрицы:";
    cin >> n;
    if (n <= 0)
    {
        cout << "Введите положительный размер" << endl;
        return 0;
    }
    // выделение памяти под матрицу
    double **a = new double*[n];
    for (int x = 0; x < n; x++)
        a[x] = new double [n];
```

Листинг 2.3.7. Программный код решения задачи 6 на языке C++
(Начало)

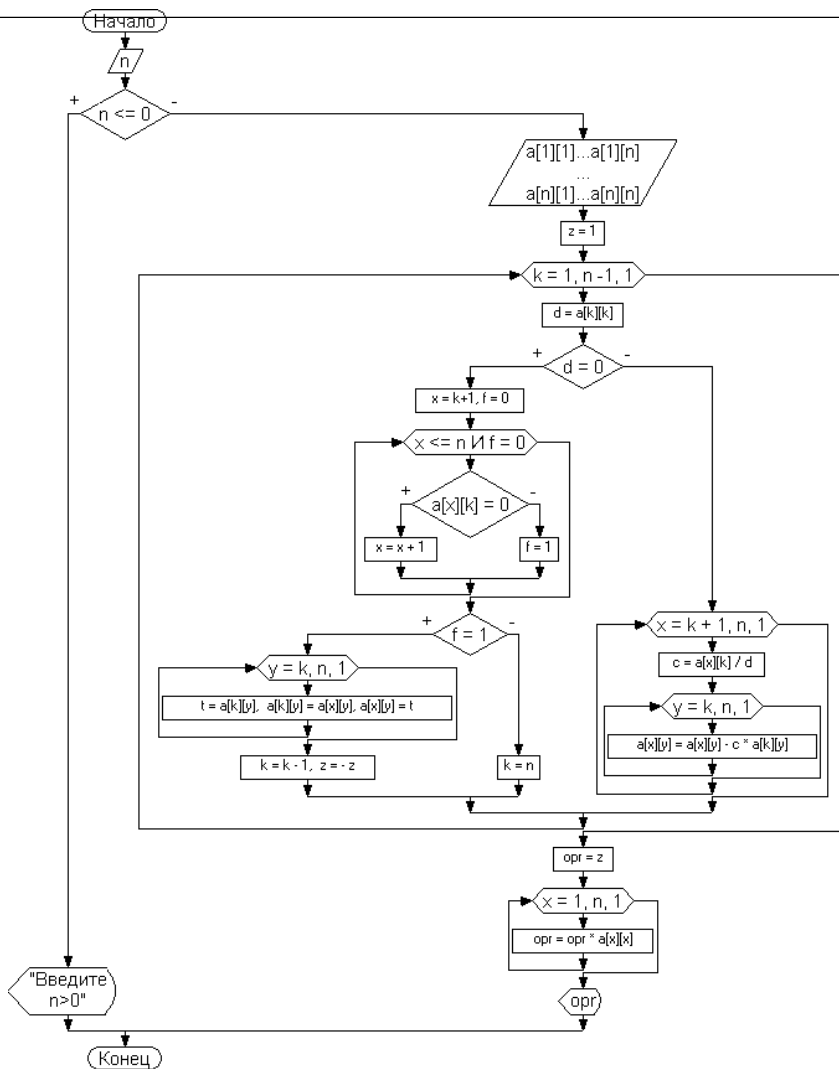


Рис. 2.3.10. Блок-схема решения задачи о вычислении определителя матрицы

```

cout << "Введите элементы матрицы:" << endl;
for (int x = 0; x < n; x++)
    for (int y = 0; y < n; y++)
        cin >> a[x][y];
double z = 1; // знак определителя
for(int k = 0; k < n - 1; k++)
{
    double d=a[k][k]; // элемент главной диагонали
    // если на главной диагонали стоит 0, то находим
    // ненулевой элемент ниже в том же столбце
    if(d == 0)
    {
        // ищем ненулевой элемент, начиная с k+1 строки
        int x = k + 1;
        int f = 0; // флажок - нашли или нет ненулевой элемент
        while (x < n && f == 0)
            if (a[x][k] != 0)
                x++;
        else
            f = 1;
        // если ненулевой элемент найден, меняем строки местами
        if (f == 1)
        {
            z = - z;
            for (int y = k; y < n; y++)
            {
                double t = a[k][y];
                a[k][y] = a[x][y];
                a[x][y] = t;
            }
            // на следующей итерации индекс k
            // должен остаться на той же строке
            k--;
        }
        else break;
    }
    else
    {
        // обнуляем все элементы, стоящие ниже в столбце
        for(int x = k + 1; x < n; x++)
        {
            double c = a[x][k] / d;
            for(int y = k; y < n; y++)
                a[x][y] = a[x][y] - c * a[k][y];
        }
    }
}
}

```

Листинг 2.3.7. Программный код решения задачи 6 на языке C++
(Продолжение)

```

cout << " Матрица после приведения к треугольному виду:"
                                     << endl;
for (int x = 0; x < n; x++)
{
    for (int y = 0; y < n; y++)
        cout << a[x] [y] << "\t";
    cout << endl;
}
// вычисление определителя как подсчет произведения
// элементов главной диагонали с учетом знака
double opr = z;
for(int x = 0; x < n; x++)
    opr = opr * a[x] [x];
cout << "Определитель = " << opr << endl;
for(int x = 0; x < n; x++)
    delete [] a[i];
delete [] a;
return 0;
}

```

Листинг 2.3.7. Программный код решения задачи 6 на языке C++
(Окончание)

Задача 7. Распечатать все седловые точки прямоугольной матрицы.

Седловая точка – это пара индексов строки и столбца, на пересечении которых находится элемент, максимальный в строке и одновременно минимальный в столбце, или наоборот – минимальный в строке и одновременно максимальный в столбце.

Для решения этой задачи в каждой строке матрицы находим максимальный и минимальный элементы. Затем все максимумы и минимумы строки проверяем на выполнение условия для седловых точек по столбцу. Если оно выполняется, то печатаем соответствующие пары индексов.

Блок-схема решения задачи показана на рис. 2.3.11 и 2.3.12. Программный код на языке C# приведен в листинге 2.3.8.

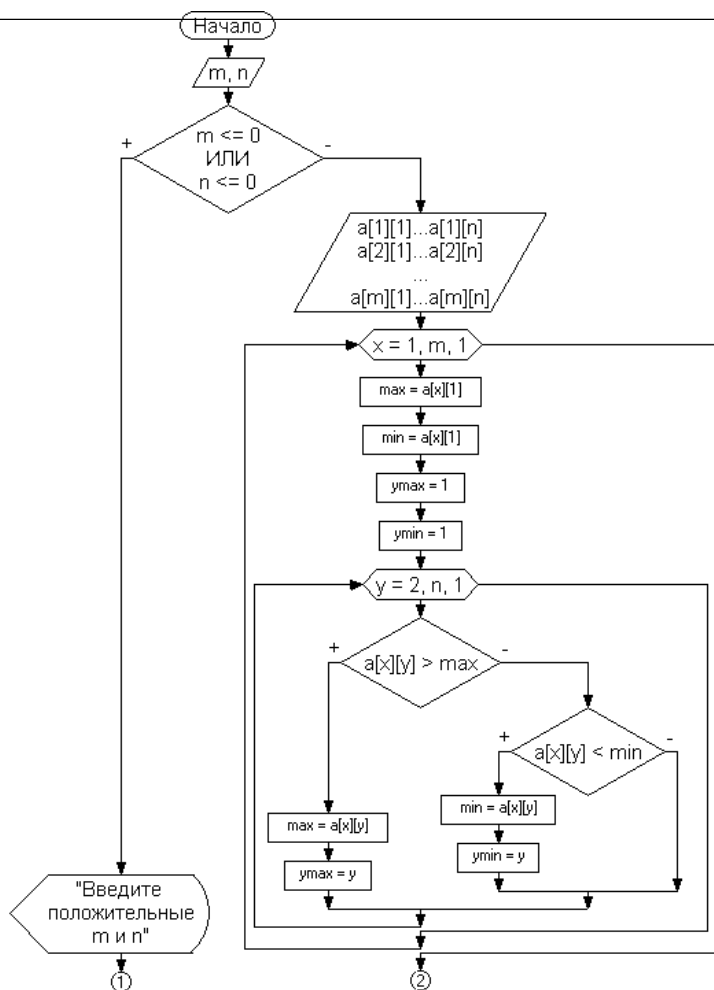


Рис. 2.3.11. Начало блок-схемы решения задачи об определении седловых точек матрицы

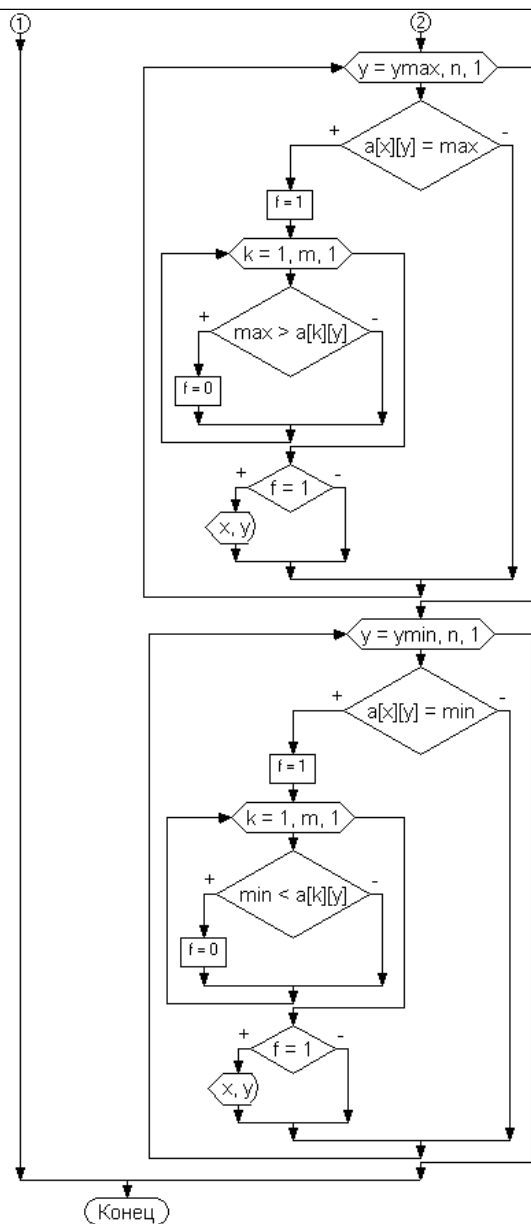


Рис. 2.3.12. Окончание блок-схемы решения задачи об определении седловых точек матрицы


```

using System;

namespace Example
{
    class Program
    {
        static void Main(string[] args)
        {
            int m, n;
            Console.Write("Введите количество строк матрицы:");
            m = int.Parse(Console.ReadLine());
            Console.Write("Введите количество столбцов матрицы:");
            n = int.Parse(Console.ReadLine());
            if (m <= 0 || n <= 0)
            {
                Console.WriteLine("Введите положительные размеры");
                return;
            }
            int[,] a = new int[m, n];
            Console.WriteLine("Введите элементы матрицы:");
            for (int x = 0; x < m; x++)
                for (int y = 0; y < n; y++)
                    a[x, y] = int.Parse(Console.ReadLine());
            for (int x = 0; x < m; x++)
            {
                // поиск максимального и минимального элемента
                // в x-й строке
                int max = a[x, 0], min = a[x, 0];
                int ymax = 0, ymin = 0;
                for (int y = 1; y < n; y++)
                {
                    if (a[x, y] > max)
                    {
                        max = a[x, y]; ymax = y;
                    }
                    else
                    {
                        if (a[x, y] < min)
                        {
                            min = a[x, y]; ymin = y;
                        }
                    }
                }
                // если максимальных элементов несколько,
                // проверяем их все на выполнение условия
                // для седловой точки в столбце
                for (int y = ymax; y < n; y++)
                {
                    if (a[x, y] == max)
                    {
                        int f = 1;
                        for (int k = 0; k < m; k++)
                            if (max > a[k, y])
                                f = 0;
                    }
                }
            }
        }
    }
}

```

Листинг 2.3.8. Программный код решения задачи 7 на языке C++
(Начало)

```

        if (f == 1)
            Console.WriteLine("{0}, {1})\t", x + 1, y + 1);
    }
    // если минимальных элементов несколько,
    // проверяем их все на выполнение условия
    // для седловой точки в столбце
    for (int y = ymin; y < n; y++)
        if (a[x, y] == min)
        {
            int f = 1;
            for (int k = 0; k < m; k++)
                if (min < a[k, y])
                    f = 0;
            if (f == 1)
                Console.WriteLine("{0}, {1})\t", x + 1, y + 1);
        }
    }
}
}
}
}

```

Листинг 2.3.8. Программный код решения задачи 7 на языке C++
(Окончание)

Домашнее задание

1. Найти сумму всех элементов прямоугольной матрицы.
2. Поменять местами максимальный и минимальный элементы прямоугольной матрицы.
3. Найти сумму двух прямоугольных матриц. В случае когда суммировать матрицы нельзя, нужно сообщить пользователю об ошибке.
4. Отсортировать строки прямоугольной матрицы в порядке возрастания их первых элементов.
5. Отсортировать столбцы прямоугольной матрицы в порядке возрастания суммы элементов столбцов.
6. Найти скалярное произведение строки и столбца квадратной матрицы, на пересечении которых находится минимальный элемент.

-
7. В прямоугольной матрице поменять местами столбцы с максимальным и минимальным количеством элементов, равных нулю.
 8. Дана квадратная матрица. Распечатать элементы диагонали, которая параллельна главной диагонали и имеет максимальную сумму элементов.
 9. Дана квадратная матрица. Определить, является ли она симметричной относительно главной диагонали.
 10. Дана квадратная матрица. Определить, является ли она треугольной. Треугольной является матрица, у которой выше (ниже) главной диагонали все элементы равны нулю.
 11. Создайте квадратную матрицу заданного размера, заполнив ее «горизонтальной змейкой». Например, для $n = 4$ должна получиться следующая матрица:

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 8 & 7 & 6 & 5 \\ 9 & 10 & 11 & 12 \\ 16 & 15 & 14 & 13 \end{pmatrix}$$

12. Создайте квадратную матрицу заданного размера, заполнив ее «диагональной змейкой». Например, для $n = 4$ должна получиться следующая матрица:

$$\begin{pmatrix} 1 & 2 & 6 & 7 \\ 3 & 5 & 8 & 13 \\ 4 & 9 & 12 & 14 \\ 10 & 11 & 15 & 16 \end{pmatrix}$$

3.1. Функции

Решение сложных задач обычно заключается в составлении большого алгоритма и соответственно в написании длинного программного кода его реализации. Для более эффективной разработки осуществляется разбиение этого алгоритма на небольшие и относительно независимые составные части, каждая из которых решает менее сложную отдельную подзадачу. Каждая из этих частей может разрабатываться отдельно. В этом и состоит суть структурного программирования, которое предполагает оформление этих составных частей программного кода в виде функций.

В программировании функция – это самостоятельная единица программы, которая решает конкретную задачу. Использование функций при создании программ создает программисту ряд удобств. Во-первых, это избавляет от повторного программирования. Если какие-либо действия надо выполнить в программе несколько раз, создаем соответствующую функцию, а затем будем ее вызывать по необходимости. Во-вторых, мы можем использовать одну и ту же функцию в нескольких программах, создав собственную библиотеку функций и подключая ее к различным проектам. В-третьих, применение функций облегчает понимание и отладку программы.

Функция – это преобразователь некоторых данных, которые по отношению к функции являются входными, в результат – выходные данные функции.

Для определения функции в языках С++ и С# используется следующая конструкция, которая состоит из заголовка и тела:

```
// Заголовок функции
ТипВозврЗначения ИмяФункции (СписокФормПараметров)
// Тело функции
{
    . . .
    return Результат;
}
```

Здесь *ТипВозврЗначения* (тип возвращаемого значения) – тип данных результата работы функции, *СписокФормПараметров* (список формальных параметров) – список имен объектов с указанием их типов, которые представляют собой входные данные функции:

```
СписокФормПараметров := Тип1 имяОб1, Тип2 имяОб2, ...
                                     ТипN имяОбN
```

Формальный параметр определяет локальную переменную функции, которая будет хранить значение какого-либо входного объекта. Временем жизни локальной переменной является время выполнения функции, т. е. после выполнения функции все переменные-параметры уничтожаются.

Тело функции содержит программную реализацию решения конкретной задачи. Результат работы функции возвращается с помощью оператора *return*.

Обращение к функции происходит посредством ее вызова. При вызове на место формальных параметров подставляются фактические параметры (аргументы). Типы аргументов должны совпадать с типами данных соответствующих формальных параметров или в некоторых случаях приводиться к ним.

Например, в листинге 3.1.1 (язык программирования C++) и в листинге 3.1.2 (язык программирования C#) приведен фрагмент программного кода, который содержит определение функции нахождения минимума из двух чисел (*MinElement()*) и пример ее вызова. Главное технологическое отличие в используемых языках программирования заключается в том, что в C++ функция должна быть объявлена до своего первого использования, а в языке C# достаточно обеспечить определение функции в области видимости, доступной в месте вызова функции. Так, в простом случае такой областью видимости является тот же класс, в котором была определена функция *Main()*. Отметим также, что стиль структурного программирования на языке C# требует, чтобы функции были статическими (определены с модификатором *static*).

Если функция в программе, написанной на языке C++, вызывается до ее определения, то в этом случае ее необходимо предварительно объявить. Объявлением (прототипом) функции является конструкция вида:

```
ТипВозврЗначения ИмяФункции (СписокТиповФормПараметров);
```

где *СписокТиповФормПараметров* (список типов формальных параметров) – список типов объектов, представляющих собой входные данные функции:

```
СписокТиповФормПараметров := Тип1, Тип2, ... ТипN
```

В список типов формальных параметров можно также добавить и имена параметров. Тогда конструкция объявления совпадает с заголовком самой функции.

```

// функция определения минимального из двух чисел
double MinElement(double a, double b)
{
    if (a < b) return a;
    else return b;
}

void main(void)
{
    int a = 3, b = 6;
    double x = 4.6, y = 8.1;
    // функция вызывается со значениями фактических
    // параметров x и y
    // результат работы функции записывается в переменную c
    double c = MinElement(x, y);
    . . .
    // функция вызывается со значениями фактических
    // параметров a и b, приведенных к типу double
    // результат работы функции записывается в переменную c
    c = MinElement((double)a, (double)b);
    . . .
    // функция вызывается со значениями фактических
    // параметров a и y,
    // целый параметр a приводится к типу double
    // результат работы функции записывается в переменную c
    c = MinElement((double)a, y);
    . . .
}

```

**Листинг 3.1.1. Фрагмент кода на языке C++ для функции
поиска минимума из двух чисел**

```

class Program
{
    // функция определения минимального из двух чисел
    static double MinElement(double a, double b)
    {
        if (a < b) return a; else return b;
    }
    static public void Main(string [] args)
    {
        int a = 3, b = 6;
        double x = 4.6, y = 8.1;
        // аналогичные вызовы функций при разных аргументах
        double c = MinElement(x, y);
        c = MinElement((double)a, (double)b);
        c = MinElement((double)a, y);
    }
}

```

**Листинг 3.1.2. Фрагмент кода на языке C# для функции
поиска минимума из двух чисел**

Иногда возникают ситуации, когда необходимо изменить значение фактических параметров внутри функции. Поэтому в некоторых языках существует способ передачи параметров по ссылке на переменную, т. е. в функцию передается не само значение переменной-параметра, а адрес памяти, по которому находится эта переменная. Тогда работа внутри функции происходит непосредственно с этой областью памяти. В разных языках программирования параметр-ссылка обозначается по-разному. В языке C++ для обозначения ссылки используется знак '&' в дополнение к типу данных, в C# – указывается ключевое слово *ref* перед описанием формального параметра и его фактического значения. Кстати, в языке C# переменная может передаваться по ссылке с указанием ключевого слова *out*. Данный случай отличается тем, что ссылка на переменную передается в функцию не для того, чтобы передать входные значения, а для того, чтобы получить результат ее работы (выходной параметр).

Приведем простой пример, демонстрирующий разницу работы функции с передачей параметров по значению и по ссылке, – написание двух версий функций, которые меняют местами значения двух переменных (листинги 3.1.3 и 3.1.4). При передаче параметров по значению все изменения будут произведены с переменными-копиями, а исходные значения останутся прежними.

```
// функция с передачей параметров по значению
void Change1(int a, int b)
{
    int t = a; a = b; b = t;
}

// функция с передачей параметров по ссылке
void Change2(int& a, int& b)
{
    int t = a; a = b; b = t;
}

void main(void)
{
    int x = 10, y = 40;
    Change1(x, y);      // результат - x=10, y=40
    Change2(x, y);      // результат - x=40, y=10;
}
```

Листинг 3.1.3. Фрагмент кода на языке C++ для демонстрации передачи параметров по значению и по ссылке

```

class Program
{
    // функция с передачей параметров по значению
    static void Change1(int a, int b)
    {
        int t = a; a = b; b = t;
    }

    // функция с передачей параметров по ссылке
    static void Change2(ref int a, ref int b)
    {
        int t = a; a = b; b = t;
    }

    static public void Main(string [] args)
    {
        int x = 10, y = 40;
        Change1(x, y);           // результат - x=10, y=40
        Change2(ref x, ref y);   // результат - x=40, y=10;
    }
}

```

Листинг 3.1.4. Фрагмент кода на языке C# для демонстрации передачи параметров по значению и по ссылке

Параметрами функции (входными и выходными данными) могут являться массивы и матрицы. Напомним, что массив – это набор переменных одинакового типа, которые расположены в непрерывной области памяти друг за другом. Обращение к этим переменным осуществляется с помощью указания имени массива и порядкового номера переменной в нем. В языке C++ имя массива является указателем, содержащим адрес его первого элемента. На рис 3.1.1 изображен массив a , содержащий n элементов, имеющих порядковые номера от 0 до $n - 1$.

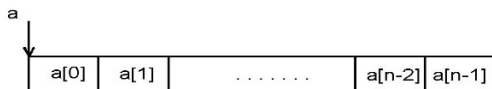


Рис. 3.1.1. Схема расположения элементов массива в памяти

Таким образом, при передаче массива в функцию в качестве параметров передаются имя массива и количество элементов в нем.

В языке C# трактовка массива похожа, однако непосредственного доступа к памяти здесь нет.

Пусть требуется написать функцию, определяющую, является ли массив симметричным. Прототип этой функции на языке C++ может

выглядеть следующим образом, когда имя массива передается как адрес памяти:

```
bool Symmetry(int * a, int n);
```

или следующим образом, когда мы указываем явно, что передается параметр-массив:

```
bool Symmetry(int a[], int n);
```

Функция будет возвращать логическое значение *true*, если массив симметричный, *false* – в противном случае.

Приведем также определение этой функции (листинг 3.1.5):

```
// определение функции проверки симметричности массива
bool Symmetry(int a[], int n)
{
    for (int i = 0; i < n / 2; i++)
        if (a[i] != a[n - 1 - i])
            return false; // нашлась несимметричная пара
    return true;
}

void main(void)
{
    int a[] = { 1,2,3,4,5,6,7 };
    cout << Symmetry(a, 7);
}
```

Листинг 3.1.5. Функция проверки симметричного массива на языке программирования C++

В листинге 3.1.6 приведем аналогичный программный код на языке C#. Заметим отличие в определении формального параметра-массива. Говорят, что в C++ указание о массиве относится к переменной, а в языке C# – к типу данных.

В случае передачи в качестве параметра в функцию двумерного массива (матрицы) также следует передать имя матрицы и ее размер, который задается двумя параметрами – количеством строк и столбцов (в случае квадратной матрицы можно указать только один размер).

Организовать хранение матрицы в памяти можно так, как представлено на рис. 3.1.2. При таком способе хранения матрица представляет собой набор массивов, каждый из которых является строкой матрицы. Адреса (ссылки) всех строк матрицы образуют отдельный массив указателей, имя которого и является именем матрицы. Размер этого массива соответствует количеству строк матрицы.

```

class Program
{
    // определение функции проверки симметричности массива
    static bool Symmetry(int[] a, int n)
    {
        for (int i = 0; i < n / 2; i++)
            if (a[i] != a[n - 1 - i])
                return false;    // нашлась несимметричная пара
        return true;
    }

    static public void Main(string [] args)
    {
        int x = 10, y = 40;
        int[] a = { 1, 2, 3, 4, 5, 6, 7 };
        Console.WriteLine(Symmetry(a, 7));
    }
}

```

Листинг 3.1.6. Функция проверки симметричного массива на языке программирования С#

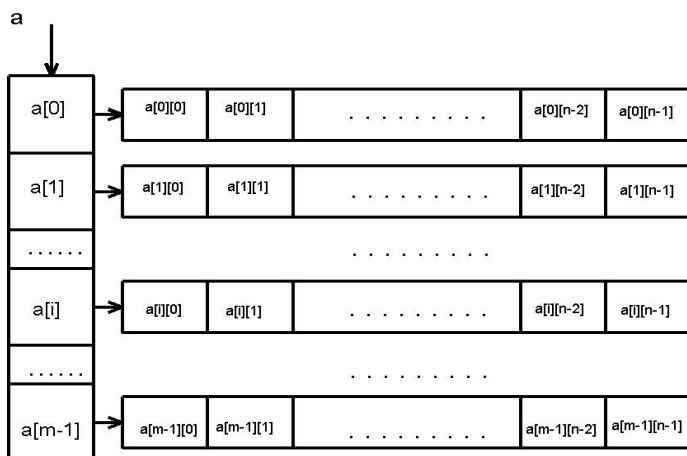


Рис. 3.1.2. Схема расположения элементов матрицы в памяти

На рис. 3.1.2 изображена схема хранения матрицы a размера $m \times n$. Здесь a – имя матрицы, указывающее на первый элемент массива адресов строк, $a[i]$ ($i = 0, \dots, m-1$) – указатель на первый элемент i -й строки матрицы.

Программный код выделения памяти под матрицу, хранение которой осуществляется по данной схеме, выглядит похожим образом в языках С++ и С# (листинги 3.1.7 и 3.1.8).

```

. . .
double ** a = new double*[m];
for (int i = 0; i < m; i++)
    a[i] = new double[n];
. . .

```

Листинг 3.1.7. Фрагмент кода для выделения памяти под матрицу на языке C++

```

. . .
double [][] a = new double[m] [];
for (int i = 0; i < m; i++)
    a[i] = new double[n];
. . .

```

Листинг 3.1.8. Фрагмент кода для выделения памяти под матрицу на языке C#

Тем не менее, как мы уже отмечали в разделе 2.3.3 «Двумерные массивы (матрицы)», память под матрицу в языке C# может быть выделена более простым способом:

```
double[,] a = new double[m, n];
```

Рассмотрим теперь пример передачи матрицы в функцию в качестве параметра.

Пусть требуется проверить, является ли квадратная матрица симметричной относительно главной диагонали. Определение этой функции на языке программирования C++ представлено в листинге 3.1.9. Аналогичным образом будет выглядеть функция и для языка C# за исключением типа параметра-матрицы (*double[][]*).

```

// определение функции проверки симметричности матрицы
bool Symmetry(double** a, int n)
{
    for (int i = 0; i < n - 1; i++)
        for (int j = i + 1; j < n; j++)
            if (a[i][j] != a[j][i])
                return false;
    return true;
}

```

Листинг 3.1.9. Определение функции проверки симметричности матрицы на языке C++

Стоит заметить также, что если функция должна менять адрес расположения массива или матрицы, которые передаются в функцию в качестве параметров, то адрес массива (матрицы) в случае языка C++ или ссылка на него в случае языка C# должны передаваться по ссылке, т. е. с указанием символа '&' в C++ и с ключевым словом *ref* в C#.

Задача 1. Дано четное число $n > 2$. Проверить, выполняется ли для этого числа гипотеза Гольдбаха: каждое четное число $n > 2$ может быть представлено в виде суммы двух простых чисел.

Для решения задачи напомним отдельную функцию, определяющую, является ли число простым. Если число простое, функция *PrimeNumber()* возвращает *true*, иначе – *false*. Само число передается в функцию как входной параметр.

В другой функции (*Goldbakh()*) организуем проверку на «простоту» всех возможных слагаемых заданного целого числа, которое в функцию передается в качестве параметра. Если найдем пару простых чисел, составляющих в сумме заданное число, значит, гипотеза Гольдбаха подтвердилась. В этом случае функция выведет на печать все найденные пары слагаемых и вернет значение *true*. Если же таких чисел нет, то мы опровергли гипотезу и функция возвращает *false*.

Приведем далее в листинге 3.1.10 определение функций проверки простоты числа и проверки гипотезы Гольдбаха на языке C++.

```
// определение функции проверки, является ли число k простым
bool PrimeNumber(int k)
{
    if (k < 2) return false;
    for (int i = 2; i <= sqrt(k); i++)
        if (k % i == 0)    // найден делитель числа k
            return false;
    return true;
}
// определение функции проверки гипотезы Гольдбаха
bool Goldbakh(int k)
{
    // переменная-флажок - найдена ли пара простых слагаемых
    int f = 0;
    for (int i = 2; i <= k / 2; i++)
        if (PrimeNumber(i) && PrimeNumber(k - i))
        {
            f = 1; cout << k << " = " << i << " + " << (k - i);
        }
    return f == 1;
}
```

Листинг 3.1.10. Определение функций для проверки гипотезы Гольдбаха на языке C++

Задача 2. Определить, принадлежит ли точка треугольнику, заданному с помощью координат его вершин.

Для решения этой задачи используем следующее свойство: если точка принадлежит треугольнику, то сумма площадей треугольников, образованных этой точкой и вершинами треугольника, совпадает с площадью исходного треугольника.

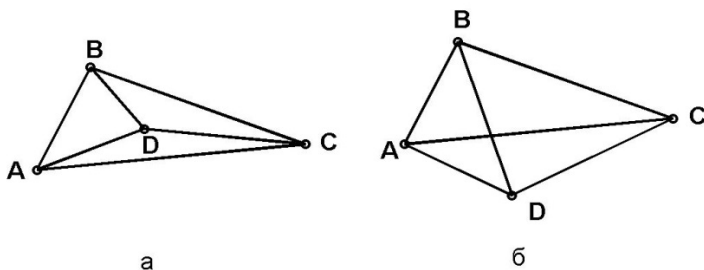


Рис. 3.1.3. Два случая расположения точки D относительно треугольника ABC

На рис. 3.1.3 представлен треугольник ABC и случаи, когда точка D принадлежит ему (рис. 3.1.3а) и не принадлежит (рис. 3.1.3б). В случае (а) площадь треугольника ABC равна сумме площадей треугольников ABD, BDC и ADC, а в случае (б) она меньше суммы площадей треугольников ABD, BDC и ADC. Поэтому решение задачи сводится к вычислению площадей разных треугольников.

Площадь треугольника будем вычислять по формуле Герона. Для ее программной реализации создадим функцию *AreaTriangle()*. Передаваемые в функцию параметры соответствуют координатам трех вершин треугольника. Возвращаемое значение – величина площади. Другая функция *PointInTriangle()* будет вызывать функцию *AreaTriangle()* четыре раза и определять принадлежность точки треугольнику. В качестве параметров функция должна получать координаты исходного треугольника и координаты точки, нахождение которой в треугольнике требуется проверить.

Следует сделать одно важное замечание. При вычислении площади используется стандартная функция вычисления квадратного корня. Она не дает точного значения, а вычисляет значение корня приближенно. В случае когда точка принадлежит треугольнику, сумма площадей внутренних треугольников может незначительно отличаться от площади исходного треугольника. Поэтому сравнение на равенство площадей происходит с некоторой точностью ε .

Далее в листинге 3.1.11 приведены определения функций на языке программирования C#.

```

// определение функции вычисления площади треугольника
// по формуле Герона
double AreaTriangle(double x1, double y1,
                    double x2, double y2,
                    double x3, double y3)
{
    double a = Math.Sqrt((x2 - x1) * (x2 - x1) +
                          (y2 - y1) * (y2 - y1));
    double b = Math.Sqrt((x3 - x1) * (x3 - x1) +
                          (y3 - y1) * (y3 - y1));
    double c = Math.Sqrt((x2 - x3) * (x2 - x3) +
                          (y2 - y3) * (y2 - y3));

    double p = (a + b + c) / 2;
    double s = Math.Sqrt(p * (p - a) * (p - b) * (p - c));
    return s;
}

// определение функции проверки принадлежности
// заданной точки треугольнику
// сначала указываются координаты вершин треугольника
// последние два параметра - координаты заданной точки
bool PointInTriangle(double x1, double y1,
                    double x2, double y2,
                    double x3, double y3,
                    double x, double y)
{
    double eps = 0.0001;

    // вычисление площадей треугольников
    // площадь исходного треугольника
    double s = AreaTriangle(x1, y1, x2, y2, x3, y3);
    // площади треугольников, в которых
    // одна из вершин - точка (x,y),
    // две другие - вершины треугольника
    double s1 = AreaTriangle(x, y, x2, y2, x3, y3);
    double s2 = AreaTriangle(x1, y1, x, y, x3, y3);
    double s3 = AreaTriangle(x1, y1, x2, y2, x, y);

    // проверка того, что сумма площадей треугольников
    // с вершиной (x,y) примерно равна площади
    // исходного треугольника
    if (Math.Abs(s1 + s2 + s3 - s) < eps)
        return true;
    else
        return false;
}

```

Листинг 3.1.11. Определение функций для проверки принадлежности точки треугольнику на языке C#

Задача 3. Дана квадратная матрица A . Вычислить A^p ($p > 1$ – целое число).

В листинге 3.1.12 для решения этой задачи определим несколько функций на языке C++:

- вспомогательную функцию выделения памяти под матрицу заданного размера (*CreateMatrix()*);
- вспомогательную функцию для ввода данных о матрице (*InputMatrix()*);
- вспомогательную функцию для вывода матрицы на печать (*PrintMatrix()*);
- вспомогательную функцию освобождения памяти, занимаемой матрицей (*DestroyMatrix()*);
- для реализации алгоритма необходима функция перемножения матриц (*MatrixProduct()*). Эту функцию не будем рассматривать в общем случае, так как для алгоритма необходимо перемножать только квадратные матрицы;
- функция возведения матрицы в степень p (*MatrixPower()*) является основной функцией данной задачи. В ней $(p - 1)$ раз будет вызываться функция *MatrixProduct()*;
- также приведем главную функцию, чтобы продемонстрировать принцип декомпозиции «в действии», поскольку главная функция будет состоять из вызовов перечисленных ранее функций.

Далее функции *CreateMatrix()*, *InputMatrix()*, *PrintMatrix()*, *DestroyMatrix()* можно оформить в виде библиотеки для работы с матрицами и подключать к любому проекту, в котором требуется работать с матрицами.

```
#include <iostream>
#include <locale>

using namespace std;

// прототипы функций работы с матрицами
double** CreateMatrix(int, int);
void InputMatrix(double **, int, int);
void PrintMatrix(double**, int, int);
void DestroyMatrix(double**, int, int);
double** MatrixProduct(double**, double**, int);
double** MatrixPower(double**, int, int);
```

Листинг 3.1.12. Определение функций для вычисления матрицы в степень на языке C++ (Начало)

```

int main(void)
{
    setlocale(LC_CTYPE, "rus");
    int n, p;
    cout << "Введите размер матрицы:"; cin >> n;
    cout << "Введите степень (p > 1):"; cin >> p;
    double** a = CreateMatrix(n, n);
    InputMatrix(a, n, n);
    double** res = MatrixPower(a, n, p);
    cout << "Матрица-результат:" << endl;
    PrintMatrix(res, n, n);
    DestroyMatrix(res, n, n);
    DestroyMatrix(a, n, n);
    return 0;
}

// функция выделения памяти под матрицу
double ** CreateMatrix(int m, int n)
{
    double** a = new double*[m];
    for (int i = 0; i < m; i++)
        a[i] = new double[n];
    return a;
}

// функция ввода элементов матрицы
void InputMatrix(double** a, int m, int n)
{
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            cin >> a[i][j];
}

// функция вывода элементов матрицы на печать
void PrintMatrix(double** a, int m, int n)
{
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
            cout << a[i][j] << "\t";
        cout << endl;
    }
}

// функция освобождения памяти матрицы
void DestroyMatrix(double** a, int m, int n)
{
    for (int i = 0; i < m; i++)
        delete[] a[i];
    delete[] a;
}

```

**Листинг 3.1.12. Определение функций для вычисления матрицы
в степень на языке C++ (Продолжение)**


```

// функция перемножения двух квадратных матриц
// одинакового размера
double** MatrixProduct(double** a, double** b, int n)
{
    double ** res = CreateMatrix(n, n);
    for(int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
        {
            res[i][j] = 0;
            for (int k = 0; k < n; k++)
                res[i][j] += a[i][k] * b[k][j];
        }
    return res;
}

// функция возведения квадратной матрицы в степень p
double** MatrixPower(double** a, int n, int p)
{
    double** b; // вспомогательная матрица, в которой
    // запоминается результат возведения матрицы
    // a в промежуточную степень, меньшую p
    b = CreateMatrix(n,n);
    // начальная инициализация матрицы
    // вначале матрица b совпадает с матрицей a
    // в первой степени
    for (int i = 0; i<n; i++)
        for (int j = 0; j<n; j++)
            b[i][j] = a[i][j];
    // возведение в степень матрицы a
    for (int k = 0; k < p - 1; k++)
    {
        //вызов функции перемножения двух матриц
        double** c = MatrixProduct(a,b,n);
        // сохранение результата перемножения в матрице b
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                b[i][j] = c[i][j];
        // освобождение памяти вспомогательной матрицы c
        DestroyMatrix(c, n, n);
    }
    return b;
}

```

Листинг 3.1.12. Определение функций для вычисления матрицы в степень на языке C++ (Окончание)

Задача 4. Дана прямоугольная матрица. Написать функцию удаления из этой матрицы строк, содержащих только нулевые элементы.

Для решения этой задачи рассмотрим два варианта алгоритма и соответствующих наборов функций. Заметим, что в данном случае удобно будет использовать вариант хранения матрицы, как показано на рис. 3.1.2.

Первый вариант решения этой задачи предполагает создание новой матрицы для хранения результата. Приведем программный код этого варианта решения в листинге 3.1.13 на языке C#. Программа содержит три функции:

1) функция *IsNullRow()* проверяет, состоит ли строка только из нулевых элементов. Ее параметрами являются размер строки матрицы (количество столбцов) и ссылка на проверяемую строку. Функция возвращает значение *true*, если строка является «нулевой», *false* – в противном случае;

2) функция *CountNullRows()* осуществляет подсчет «нулевых» строк матрицы. В нее передается исходная матрица. Функция возвращает целое число – количество «нулевых» строк;

3) функция *RemoveNullRowMatrix()* формирует новую матрицу, не включая в нее «нулевые» строки исходной матрицы. Входным параметром является матрица. Для определения размера и адреса расположения новой матрицы в функцию в качестве параметров передаются еще и две переменные-ссылки: ссылка на матрицу-результат и на количество строк в ней, так как именно функция *RemoveNullRowMatrix()* должна определять их значения.

Как и при разборе предыдущей задачи, приведем полный код программы, дополнив его вспомогательными функциями выделения памяти, освобождения памяти, ввода и печати матрицы.

```
using System;

namespace Example
{
    class Program
    {
        static void Main (string [] args)
        {
            int m, n;
            Console.WriteLine("Введите размеры матрицы:");
            m = int.Parse(Console.ReadLine());
            n = int.Parse(Console.ReadLine());
            int [][] a = CreateMatrix(m, n);
            InputMatrix(a, m, n);
            int [][] res;    // матрица-результат
            int k;          // количество строк в матрице-результате
            RemoveNullRowMatrix(a, m, n, out res, out k);
            PrintMatrix(res, k, n);
        }
    }
}
```

Листинг 3.1.13. Определение функций для удаления «нулевых» строк матрицы на языке C# (Начало)

```

// функция выделения памяти под матрицу
static int[][] CreateMatrix(int m, int n)
{
    int[][] a = new int[m][];
    for (int i = 0; i < m; i++)
        a[i] = new int[n];
    return a;
}

// функция ввода элементов матрицы
static void InputMatrix (int[][] a, int m, int n)
{
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            a[i][j] = int.Parse(Console.ReadLine());
}

// функция печати элементов матрицы
static void PrintMatrix(int[][] a, int m, int n)
{
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
            Console.Write(a[i][j] + "\t");
        Console.WriteLine();
    }
}

// функция проверки строки матрицы на то,
// что она состоит только из 0
static bool IsNullRow(int[] a, int n)
{
    for (int i = 0; i < n; i++)
        if (a[i] != 0)
            return false;
    return true;
}

// функция подсчета количества "нулевых" строк матрицы
static int CountNullRows(int[][] a, int m, int n)
{
    int k = 0;
    for (int i = 0; i < m; i++)
        if (IsNullRow(a[i], n))
            k++;
    return k;
}

```

Листинг 3.1.13. Определение функций для удаления «нулевых» строк матрицы на языке C# (Продолжение)

```

// функция удаления строк только с нулевыми элементами
static void RemoveNullRowMatrix(int[] [] a, int m, int n,
                                out int[] [] res, out int k)
{
    // k - количество «ненулевых» строк матрицы
    k = m - CountNullRows(a, m, n);
    if (k == 0)
    {
        // все строки исходной матрицы «нулевые»
        res = null;
        return;
    }
    // выделение памяти под матрицу-результат
    res = new int[k] [];
    for (int i = 0; i < k; i++)
        res[i] = new int[n];
    int t = 0;           // номер строки исходной матрицы
    for (int i = 0; i < k; i++)
    {
        // находим номер первой "ненулевой" строки
        // исходной матрицы, начиная с t-й
        while (IsNullRow(a[t], n)) t++;
        // найденная "ненулевая" строка копируется на
        // i-е место в матрицу-результат
        for (int j = 0; j < n; j++)
            res[i][j] = a[t][j];
        // переход к следующей строке исходной матрицы
        t++;
    }
}
}
}
}

```

Листинг 3.1.13. Определение функций для удаления «нулевых» строк матрицы на языке C# (Окончание)

Рассмотрим второй вариант решения этой задачи – исходная матрица преобразуется посредством удаления из нее нулевых строк. Данный вариант решения возможен и эффективен только в случае использования схемы хранения матрицы, изображенной на рис. 3.1.2. Преобразование матрицы происходит путем переприсвоения указателей (ссылок) на строки. Когда встречается «нулевая» строка, освобождается занимаемая ею память, а указателю (ссылке) на эту строку в дальнейшем будет присвоен адрес (ссылка) начала первой встретившейся после нее «ненулевой» строки, если такова существует.

Этот вариант функции *RemoveNullRowMatrix()* в качестве параметров получает исходную матрицу и ее размеры, которые передаются по ссылке, поскольку их значения могут измениться. Программный код приведен в листинге 3.1.14.

```

// функция удаления строк только с нулевыми элементами
static void RemoveNullRowMatrix(ref int[] [] a, ref int m,
                                int n)
{
    // k - количество "ненулевых" строк матрицы
    int k = m - CountNullRows(a, m, n);
    if (k == 0)
    {
        // все строки исходной матрицы "нулевые"
        a = null; m = 0;
        return;
    }
    // вводим переменную для хранения
    // количества удаленных "нулевых" строк
    int t = 0;
    for (int i = 0; i < m; i++)
        // если строка "нулевая", учитываем ее в переменной t
        if (IsNullRow(a[i], n))
            t++;
        else
            // сдвиг "ненулевых" строк
            a[i - t] = a[i];
    // установка значения нового количества строк матрицы
    m = k;
}

```

Листинг 3.1.14. Другой вариант функции удаления «нулевых» строк матрицы на языке C#

Задача 5. Написать функцию сложения двух многочленов.

Правила суммирования многочленов можно сформулировать следующим образом:

1) если степени суммируемых многочленов не равны, степень многочлена-суммы совпадает с наибольшей степенью многочленов-слагаемых, а коэффициенты вычисляются как сумма коэффициентов при соответствующих степенях;

2) если степени суммируемых многочленов равны, степень многочлена-суммы вычисляется следующим образом. Начиная с наибольших степеней, осуществляется поиск первой ненулевой суммы коэффициентов многочленов-слагаемых при соответствующих степенях. Степень данных слагаемых и будет степенью многочлена-результата.

Первый вариант функции использует представление многочлена в виде массива. Многочлен степени n можно представить в виде массива размерности $(n + 1)$. Элемент массива с индексом i хранит коэффициент слагаемого, степень которого равна i .

В функцию сложения многочленов передаем два массива коэффициентов и степени исходных многочленов. С помощью параметра-ссылки

функция будет сохранять результат – массив коэффициентов многочлена-суммы, а его степень можно будет получить как возвращаемое значение функции. Приведем программный код функции сложения многочленов на языке C++ в листинге 3.1.15.

```
// определение функции сложения двух многочленов
int SumPolynom(double* p1, int n1, double* p2, int n2,
               double*& p)
{
    if (n1 > n2)
    {
        // степень первого многочлена больше
        p = new double[n1 + 1];
        // считаем коэффициенты при степенях, которые
        // есть в обоих массивах
        for (int i = 0; i < n2 + 1; i++)
            p[i] = p1[i] + p2[i];
        // перенос коэффициентов первого многочлена при
        // степенях, больших степени второго многочлена
        for (int i = n2 + 1; i < n1 + 1; i++)
            p[i] = p1[i];
        // степень многочлена-результата -
        // степень первого многочлена
        return n1;
    }
    else
        if (n1 < n2)
        {
            // степень второго многочлена больше
            p = new double[n2 + 1];
            // считаем коэффициенты при степенях, которые
            // есть в обоих массивах
            for (int i = 0; i < n1 + 1; i++)
                p[i] = p1[i] + p2[i];
            // перенос коэффициентов второго многочлена при
            // степенях, больших степени первого многочлена
            for (int i = n1 + 1; i < n2 + 1; i++)
                p[i] = p2[i];
            // степень многочлена-результата -
            // степень второго многочлена
            return n2;
        }
    else
    {
        // случай, когда n1 = n2
        // поиск степени многочлена-результата
        int j = n1;
    }
}
```

Листинг 3.1.15. Первый вариант функции сложения
двух многочленов на языке C++ (Начало)

```

    // поиск большего ненулевого коэффициента
    for (; j >= 0; j--)
        if (p1[j] + p2[j] != 0)
            break;
    p = new double[j + 1];
    // остальные коэффициенты при младших степенях
    // определяются как сумма соответствующих
    // коэффициентов обоих массивов
    for (int i = 0; i < j + 1; i++)
        p[i] = p1[i] + p2[i];
    return j;
}
}

```

Листинг 3.1.15. Первый вариант функции сложения двух многочленов на языке C++ (Окончание)

Приведем еще один вариант решения задачи с использованием пользовательского типа данных, определяющего многочлен. Такие типы данных вводятся в программу для описания множества объектов, определяемых одинаковым набором атрибутов. Это дает эффект более легкого чтения программного кода, особенно когда набор атрибутов объектов достаточно сложный. Например, пользовательский тип данных для описания множества многочленов может содержать два атрибута – адрес (ссылку) массива коэффициентов многочлена и его степень.

В большинстве языков программирования существует специальная конструкция, позволяющая определить пользовательский тип данных, в котором набор переменных различных типов хранится как единое целое. Эту конструкцию называют структурой (C++, C#, Java) или записью (Pascal). Определим структуру для хранения многочлена на языке C++ следующим образом:

```

struct Polynom
{
    double* a;    // массив коэффициентов многочлена
    int n;        // степень многочлена
};

```

Такое представление многочленов позволяет передавать в функцию и возвращать из функции объекты пользовательского типа *Polynom*, при этом осуществляется передача всех его атрибутов. При обращении к атрибутам (полям) структуры в языке C++ существуют две операции '.' и '->'. Операция '.' применяется к переменной типа структуры, а операция '->' – к указателю на структуру. Для других языков, например C# или Java, для этих целей используется только одна операция '.'. Это позволяет существенно упростить чтение программного кода.

Покажем в листинге 3.1.16 правила синтаксиса при создании переменной типа *Polynom*:

```
Polynom p1; // создание переменной типа структуры
p1.n = 10; // задание значения поля n многочлена p1
p1.a = new float[p1.n + 1]; // выделение памяти под массив
// коэффициентов многочлена p1
// динамическое выделение памяти под хранение многочлена и
// сохранение адреса в переменной-указателе p2
Polynom * p2 = new Polynom;
p2 -> n = 10;
p2 -> a = new float[p2 -> n + 1];
```

Листинг 3.1.16. Два способа создания переменной типа структуры.
Демонстрация операций доступа к полям структуры

Далее в листинге 3.1.17 приведем другой вариант решения задачи суммирования многочленов. Создадим также вспомогательные функции, которые помогут упростить программный код обработки многочленов:

- функция *CreatePolynom()* предназначена для создания объекта структуры типа *Polynom*, т. е. для выделения памяти под массив коэффициентов многочлена. Степень многочлена передается в функцию в качестве параметра;
- функция *InputPolynom()* предназначена для ввода коэффициентов многочлена с клавиатуры. Объект типа *Polynom*, коэффициенты которого требуется ввести, передается в функцию в качестве параметра;
- функция *PrintPolynom()* предназначена для вывода формулы многочлена на экран. Объект типа *Polynom* передается в функцию в качестве параметра;
- функция *DestroyPolynom()* предназначена для освобождения памяти, занимаемого массивом коэффициентов многочлена. Объект типа *Polynom* передается в функцию в качестве параметра;
- функция *SumPolynom()* осуществляет суммирование двух многочленов. Параметрами функции являются два объекта типа *Polynom*, которые являются операндами суммирования. Функция возвращает объект типа *Polynom*, хранящий результат суммирования.


```

#include <iostream>
#include <locale>

using namespace std;

// описание структуры для многочлена
struct Polynom
{
    double* a; // массив коэффициентов многочлена
    int n;      // степень многочлена
};

// прототипы функций
Polynom CreatePolynom(int);
void InputPolynom(Polynom);
void PrintPolynom(Polynom);
void DestroyPolynom(Polynom);
Polynom SumPolynom(Polynom, Polynom);

void main(void)
{
    setlocale(LC_CTYPE, "rus");
    int m;
    cout << "Введите степень первого полинома:";
    cin >> m;
    int n;
    cout << "Введите степень второго полинома:";
    cin >> n;
    Polynom p1 = CreatePolynom(m);
    cout << "Введите коэффициенты первого полинома:";
    InputPolynom(p1);
    Polynom p2 = CreatePolynom(n);
    cout << "Введите коэффициенты второго полинома:";
    InputPolynom(p2);
    Polynom res = SumPolynom(p1, p2);
    cout << "Полином-результат:" << endl;
    PrintPolynom(res);
    DestroyPolynom(p1);
    DestroyPolynom(p2);
    DestroyPolynom(res);
}

// функция создания объекта структуры для многочлена
Polynom CreatePolynom(int n)
{
    Polynom p; // создание объекта для хранения многочлена
    p.n = n;    // определение степени многочлена
    p.a = new double[n + 1]; // выделение памяти
    return p;
}

```

Листинг 3.1.17. Программный код решения задачи суммирования многочленов с использованием структуры *Polynom* (Начало)

```

// функция ввода данных в объект структуры для многочлена
void InputPolynom(Polynom p)
{
    for ( int i = p.n; i >= 0; i--)
        cin >> p.a[i];
}

// функция печати многочлена
void PrintPolynom(Polynom p)
{
    for (int i = p.n; i > 0; i--)
        cout << p.a[i] <<" * x ^ " << i << " + ";
    cout << p.a[0] << endl;
}

// функция освобождения памяти,
// занимаемой коэффициентами многочлена
void DestroyPolynom(Polynom p)
{
    delete[] p.a;
}

// функция получения суммы двух многочленов
Polynom SumPolynom(Polynom p1, Polynom p2)
{
    if (p1.n > p2.n)
    {
        // степень многочлена p1 больше, чем у p2
        Polynom p = CreatePolynom(p1.n);
        for (int i = 0; i < p2.n + 1; i++)
            p.a[i] = p1.a[i] + p2.a[i];
        for (int i = p2.n + 1; i < p1.n + 1; i++)
            p.a[i] = p1.a[i];
        return p;
    }
    else
    {
        if (p1.n < p2.n)
        {
            // степень многочлена p2 больше, чем у p1
            Polynom p = CreatePolynom(p2.n);
            for (int i = 0; i < p1.n + 1; i++)
                p.a[i] = p1.a[i] + p2.a[i];
            for (int i = p1.n + 1; i < p2.n + 1; i++)
                p.a[i] = p2.a[i];
            return p;
        }
    }
}

```

Листинг 3.1.17. Программный код решения задачи суммирования многочленов с использованием структуры *Polynom* (Продолжение)

```

else
{
    // степени многочленов-операндов совпадают
    int j = p1.n;
    for (; j >= 0; j--)
        if (p1.a[j] + p2.a[j] != 0)
            break;
    Polynom p = CreatePolynom(j);
    for (int i = 0; i < j + 1; i++)
        p.a[i] = p1.a[i] + p2.a[i];
    return p;
}
}

```

Листинг 3.1.17. Программный код решения задачи суммирования многочленов с использованием структуры *Polynom* (Окончание)

Задача 6. Найти один из корней уравнения вида $f(x) = 0$ методом хорд на отрезке $[a, b]$ с точностью ε (точность по функционалу). Если корней на отрезке не существует, выдать сообщение об этом. Для применения метода хорд необходимо, чтобы:

- 1) функция являлась непрерывной, а также выпуклой или вогнутой,
- 2) значения функции $f(x)$ на концах отрезка имели противоположные знаки.

Для решения задачи создаем две функции: функцию вычисления значения $f(x)$ и собственно функцию нахождения корня уравнения $f(x) = 0$ методом хорд.

Согласно методу хорд, требуется построить отрезок, соединяющий точки $(a, f(a))$ и $(b, f(b))$, и найти точку пересечения этого отрезка с осью абсцисс. Обозначим точку пересечения через x . Далее сужается отрезок поиска корня путем замены одного из концов на значение x . В точку x переносится тот конец отрезка, значение функции в котором совпадает по знаку со значением $f(x)$.

На рис. 3.1.4 изображены варианты нахождения точки пересечения построенной хорды с осью абсцисс. На рис. 3.1.4а приведено изменение левого конца отрезка (на следующей итерации $a = x$), на рис. 3.1.4б – правого конца ($b = x$).

Далее в листинге 3.1.18 будет приведен программный код функции поиска корня уравнения методом хорд на языке программирования C#. Функция принимает в качестве параметров координаты концов отрезка, точность решения и ссылку на переменную, в которую будет помещен найденный корень. Успешным ли был поиск корня, можно будет узнать с

помощью возвращаемого значения, которое представлено значением-флагом (*true* – если корень был найден, *false* – в противном случае). Функции вычисления значения $f(x)$ и главная функций здесь очевидны.

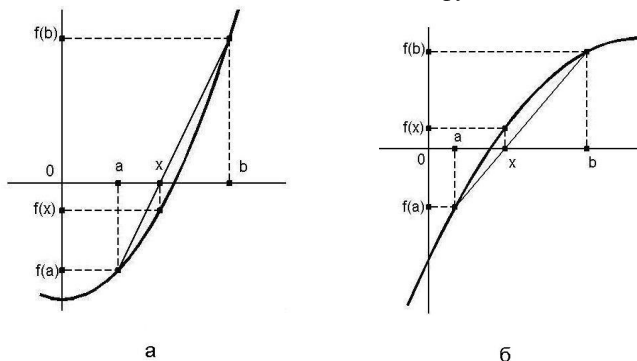


Рис. 3.1.4. Демонстрация одной итерации метода хорд

```
// функция решения уравнения методом хорд
static bool RootEquation(double a, double b, double eps,
                          ref double x)
{
    // вычисление значений f(x) на концах отрезка
    double fa = f(a), fb = f(b);
    // a = b
    if (a == b && fa == 0)
    {
        x = a; return true; // корень найден
    }
    if (fa * fb < 0)
    {
        x = a;
        // условие выхода из цикла - значение функции в
        // одном из концов отрезка близко к нулю
        while (Math.Abs(fa) > eps && Math.Abs(fb) > eps)
        {
            // вычисление точки пересечения хорды
            // с осью абсцисс
            x = (a * fb - b * fa) / (fb - fa);
            // изменение отрезка поиска корня
            if (fa * f(x) > 0)
                a = x;
            else
                b = x;
        }
        return true; // корень найден
    }
    return false; // корень не найден
}
```

Листинг 3.1.18. Функция вычисления корня уравнения методом хорд

Задача 7. Дана символьная строка, состоящая из строчных букв латинского алфавита и пробелов. Набор символов, не разделенных пробелами, образует слово. Требуется написать функцию шифрования символьной строки по следующему алгоритму:

1) определяется количество букв в слове минимальной длины. Обозначим это количество через D ;

2) каждый символ строки за исключением пробела, заменяется символом, расположенным в алфавите через D символов после. Например, при $D = 3$ символ 'a' заменяется символом 'd', символ 'b' – символом 'e' и т. д. Для тех символов, после которых не существует D символов до конца алфавита, поиск замены продолжается, начиная с буквы 'a'. Например, при $D = 3$ символ 'x' заменяется символом 'a', символ 'y' – символом 'b' и символ 'z' – символом 'c'.

Поскольку количество букв в слове минимальной длины может быть больше размера алфавита, смещение для нахождения символа-замены будет вычисляться как остаток от деления длины минимального слова на 26 (количество букв латинского алфавита): $shift = D \% 26$.

Приведем программный код на языке программирования C++ в листинге 3.1.19 с выделением функции вычисления длины символьной строки (*Length()*), вычисления длины слова минимальной длины (*MinLength()*) и функции шифрования символьной строки (*EncodeString()*).

```
#include <iostream>
#include <clocale>
using namespace std;
// прототип функции вычисления длины символьной строки
int Length(char* str);
// прототип функции вычисления количества букв в слове
// минимальной длины
int MinLength(char* str);
// прототип функции шифрования символьной строки
char* EncodeString(char* str);

void main(void)
{
    setlocale(LC_CTYPE, "rus");
    char str[100];
    cin.getline(str, 100);
    char* res = EncodeString(str);
    cout << res << endl;
    delete[] res;
}
```

Листинг 3.1.19. Программа шифрования символьной строки (Начало)

```

// определение функции вычисления длины символьной строки
int Length(char* str)
{
    int i = 0;
    // считаем символы, пока не встретился
    // символ конца строки '\0'
    while (str[i] != '\0')
        i++;
    return i;
}

// определение функции вычисления количества букв
// в слове минимальной длины
int MinLength(char* str)
{
    int min = 0, // длина слова минимальной длины
        i = 0, k;
    while (str[i] != '\0')
    {
        // пропускаем пробелы
        while (str[i] == ' ') i++;
        k = 0; // длина текущего слова
        // проход по символам текущего слова
        while (str[i] != ' ' && str[i] != '\0')
        {
            k++; i++;
        }
        // если встретилось первое слово (min=0)
        // или встретилось слово с меньшей длиной,
        // меняем значение переменной min
        if (min == 0 || min > k) min = k;
    }
    return min;
}

// определение функции шифрования символьной строки
char* EncodeString(char* str)
{
    // определение длины исходной символьной строки
    int len = Length(str);
    // выделение памяти под строку-результат
    char* res = new char[len + 1];
    // определение размера слова минимальной длины
    int D = MinLength(str);
    // определение смещения при шифровании
    int shift = D % 26;

```

Листинг 3.1.19. Программа шифрования символьной строки
(Продолжение)

```

// цикл замены символов
for (int i = 0; i < len; i++)
{
    if (str[i] != ' ')
    {
        // если символ не является пробелом,
        // вычисляем код символа-замены
        int ch = str[i] + shift;
        if (ch > (int)'z')
            // нахождение символа-замены,
            // начиная с символа 'a'
            res[i] = 'a' + (ch - 'z' - 1);
        else
            res[i] = ch;
    }
    else res[i] = ' ';
}
res[len] = '\0';
return res;
}

```

Листинг 3.1.19. Программа шифрования символьной строки
(Окончание)

Домашнее задание

1. Дано натуральное число, меньшее 1 000 000. Написать функцию его перевода в римскую запись (результат представить в виде строки).
2. Даны координаты четырех точек на плоскости. Написать функцию определения, являются ли эти точки вершинами параллелограмма.
3. Написать функцию вычисления площади выпуклого многоугольника. Координаты вершин многоугольника заданы в виде двух массивов – массива координат по оси абсцисс и массива координат по оси ординат. Вершины в массиве перечислены в порядке обхода по часовой стрелке.
4. Написать функцию вычисления радиуса описанной окружности выпуклого многоугольника, если она существует. Координаты вершин многоугольника заданы в виде двух массивов – массива координат по оси абсцисс и массива координат по оси ординат.

Вершины в массиве перечислены в порядке обхода по часовой стрелке.

5. Даны координаты N точек на плоскости в виде двух массивов – массива координат по оси абсцисс и массива координат по оси ординат. Написать функцию нахождения трех точек из набора, которые образуют треугольник максимальной площади.
6. Даны координаты N точек на плоскости в виде двух массивов – массива координат по оси абсцисс и массива координат по оси ординат. Написать функцию, определяющую, существуют ли в наборе четыре точки, образующие трапецию.
7. Дана прямоугольная матрица. Написать функцию нахождения седловой точки матрицы. Седловой точкой называется пара индексов (i, j) такая, что a_{ij} – минимальный элемент из i -й строки и максимальный элемент из j -го столбца.
8. Решить задачу 7 с использованием структуры *Matrix*, которую необходимо определить.
9. Написать функцию вычисления значения многочлена в заданной точке. Задачу решить с использованием структуры *Polynom*.
10. Написать функцию перемножения двух многочленов. Задачу решить с использованием структуры *Polynom*.
11. Написать функцию дифференцирования многочлена. Задачу решить с использованием структуры *Polynom*.
12. Написать функцию сортировки массива методом вставок. Метод вставок вкратце можно описать так. Пусть дан массив размера n : $a[0], a[1] \dots a[n - 1]$. Каждый элемент $a[i]$ (индекс i изменяется от 1 до $n - 1$) вставляется на подходящее место в уже упорядоченную совокупность $a[0], a[1] \dots a[i - 1]$. Это место определяется при обратном проходе по массиву от $(i - 1)$ -го элемента до элемента с меньшим значением, чем $a[i]$, или до начала массива.
13. Написать функцию решения системы линейных уравнений третьего порядка методом Крамера.

14. Написать функцию решения системы линейных уравнений n -го порядка ($3 < n < 10$, n – целое число) методом Гаусса.

15. Написать функцию вычисления обратной матрицы с помощью метода Гаусса.

16. Написать функцию вычисления корня уравнения $f(x) = 0$ на отрезке $[a, b]$ с точностью $\varepsilon > 0$ методом деления отрезка пополам.

Полагаем, что отрезок задан так, что $f(a) \cdot f(b) < 0$, а функция $f(x)$ непрерывна на нем. В этом случае гарантируется, что корень на отрезке $[a, b]$ будет существовать (поскольку на границах отрезка функция принимает разные знаки, то она обязательно пересечет ось абсцисс, т. е. обязательно найдется x из $[a, b]$ такое, что $f(x) = 0$).

Метод деления отрезка пополам заключается в следующем:

1) вычислить $f(a)$ и $f(b)$;

2) вычислить середину отрезка $c = \frac{(a+b)}{2}$ и $f(c)$;

3) если $f(c) = 0$, то c – корень. Иначе,

4) если $f(a) \cdot f(c) < 0$, то $b = c$, иначе $a = c$;

5) если $b - a > \varepsilon$, то переход к п. 1. Иначе решение получено (любая точка из полученного отрезка, например точка $\frac{(a+b)}{2}$).

17. Написать функцию вычисления с заданной точностью корня уравнения $f(x) = 0$ на отрезке $[a, b]$ методом касательных с точностью $\varepsilon > 0$, где $f(x)$ – некоторая непрерывная дифференцируемая выпуклая функция. Полагаем, что отрезок задан так, что $f(a) \cdot f(b) < 0$.

Метод касательных заключается в следующем:

1) вычислить x_1 и x_2 – точки пересечения касательных к графику функции $f(x)$, построенных в точках a и b , с осью абсцисс. Выбрать ту точку, которая принадлежит отрезку $[a, b]$. Запомнить ее в x ;

2) вычислить $f(x)$;

3) если $f(a) \cdot f(x) < 0$, то $b = x$, иначе $a = x$;

4) если $|f(a)| < \varepsilon$, то решением является точка a ; если $|f(b)| < \varepsilon$, то решением является точка b ; в противном случае осуществляется переход к п. 1.

-
18. Написать функцию выравнивания символьной строки до заданного количества символов (если исходная строка короче, равномерно добавить между словами пробелы, в противном случае удалить лишние символы).
 19. Дана символьная строка. Написать функцию определения количества вхождений в нее заданного слова.
 20. Дана символьная строка. Написать функцию нахождения в ней симметричного слова максимальной длины.
 21. Дана символьная строка. Написать функцию нахождения в ней слова максимальной длины, состоящего только из гласных букв.
 22. Дана символьная строка. Написать функцию удаления из нее всех слов, имеющих заданную длину.
 23. Дана зашифрованная символьная строка по алгоритму, приведенному в задаче 7. Написать функцию расшифровки данной символьной строки.

Рекурсивные функции – это функции, в теле которых содержится вызов самих себя. Обычно их используют в ситуациях, когда легко свести исходную задачу к задаче того же вида, но с другими исходными данными, например за счет уменьшения размерности задачи, перехода в новую точку и пр.

Каждый новый вызов рекурсивной функции предполагает сохранение состояния всех переменных функций в специальной области памяти, называемой стеком. После выполнения рекурсивной функции, управление передается оператору, следующему за ее вызовом. При этом состояние всех переменных восстанавливается из стека. Поэтому применение таких функций считается очень затратным с точки зрения памяти и не подходит для решения задач большой размерности. Тем не менее существует множество задач, в которых рекурсивные алгоритмы более просты и позволяют писать компактные программы.

Следует помнить, что любая рекурсивная функция должна иметь обязательное условие выхода из рекурсии. При выполнении этого условия управление передается в место предыдущей рекурсивной итерации, откуда произошел рекурсивный вызов, или в вызывающую функцию.

Задача 1. Вычислить $n!$.

Когда рекурсия заключается в сведении к задаче меньшей размерности, алгоритм аналогичен обратному ходу метода математической индукции.

Очевидно, что $n! = n \cdot (n - 1)!$. Эта формула и определяет основное рекуррентное соотношение – $(n - 1)!$ вычисляется с помощью вызова функции с фактическим параметром, принимающим значение $(n - 1)$. Условие выхода из рекурсии – это сведение задачи к тривиальному случаю – $1! = 0! = 1$.

Приведем программный код рекурсивной функции вычисления факториала в листинге 3.2.1. Программный код напишем на языке программирования C++. Заметим, что программный код на языке C# отличаться не будет.

```

// определение функции вычисления факториала
long int Factorial(int n)
{
    // для отрицательных чисел факториал не определен
    if (n < 0)
        return -1;
    // условие выхода из рекурсии
    if (n == 1 || n == 0)
        return 1;
    // рекурсивный вызов
    return n * Factorial(n - 1);
}

```

Листинг 3.2.1. Рекурсивная функция вычисления $n!$ на языке C++

На примере решения этой задачи рассмотрим, каким образом выполняется рекурсивный вызов функции. Допустим, пользователь ввел значение переменной n , равное 4.

Главная функция будет содержать вызов функции *Factorial(4)*. Для выполнения вызова функции в оперативной памяти выделяется некоторая область, необходимая для запоминания адреса возврата, создания копий параметров и копий локальных переменных.

Свободная область памяти в момент первого вызова *Factorial(4)* на рис. 3.2.1 изображена в виде пустого прямоугольника. Внутри прямоугольника показано значение параметра n .

Вычисление факториала в функции *Factorial(4)* начинается с проверки условия выхода из рекурсии ($n = 1$ или $n = 0$). Так как оно не выполняется, происходит второй вызов *Factorial(3)*, который является первым рекурсивным вызовом функции. Для второго вызова также выделяется некоторое количество памяти (рис. 3.2.2).

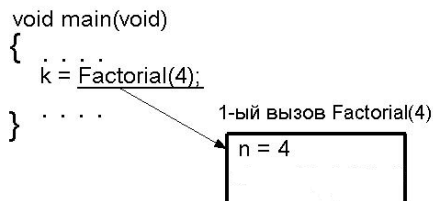


Рис. 3.2.1. Демонстрация первого вызова функции *Factorial(4)*

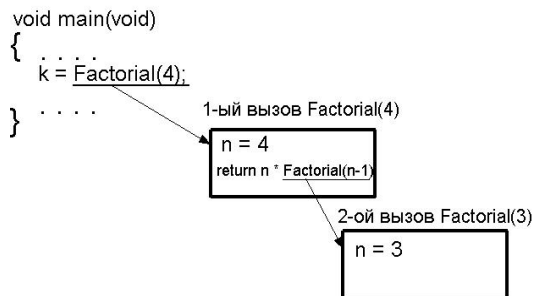


Рис. 3.2.2. Демонстрация первого рекурсивного вызова функции *Factorial(3)*

Далее процесс повторяется до тех пор, пока на четвертом вызове *Factorial(1)* не будет выполнено условие выхода из рекурсии (рис. 3.2.3). Поэтому после четвертого вызова происходит возврат в вызывающую функцию (т. е. в третий экземпляр *Factorial(2)*, рис. 3.2.4).

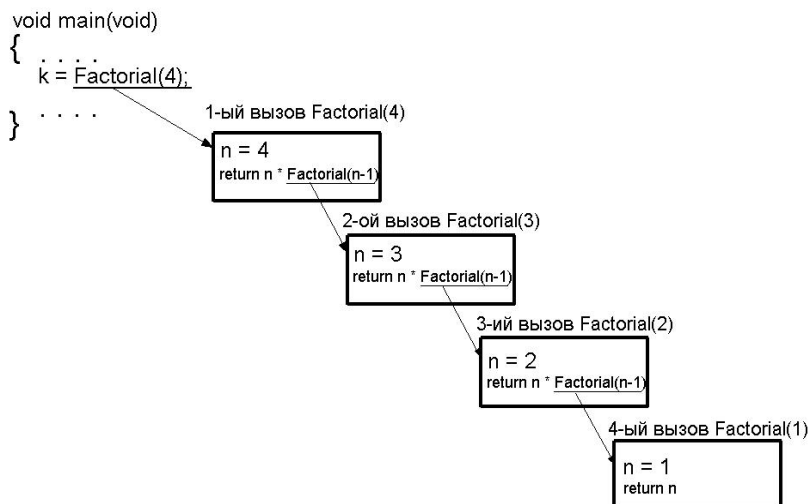


Рис. 3.2.3. Демонстрация рекурсивного вызова функции *Factorial(1)*

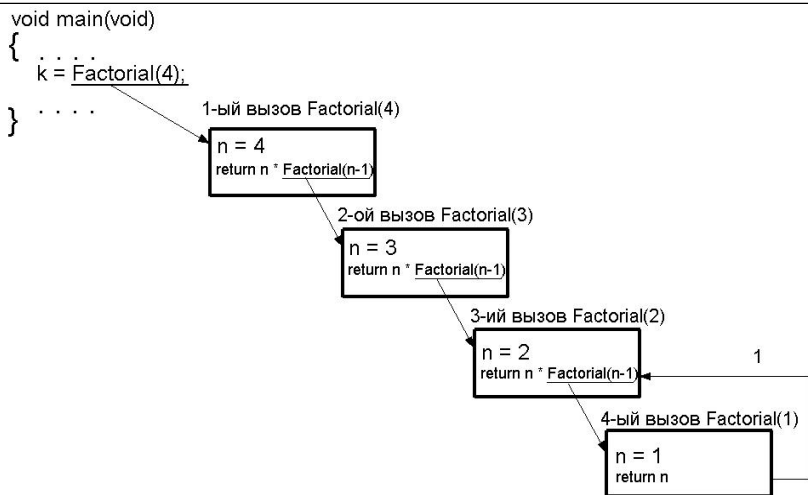


Рис. 3.2.4. Возврат из рекурсивного вызова функции *Factorial(1)*

Третий экземпляр *Factorial(2)* завершается, но перед завершением возвращает значение $2 \cdot 1$ (n *результат 4-го вызова) и т. д. Первый экземпляр функции перед завершением работы возвратит значение $4 \cdot$ – результат 2-го вызова (рис. 3.2.5).

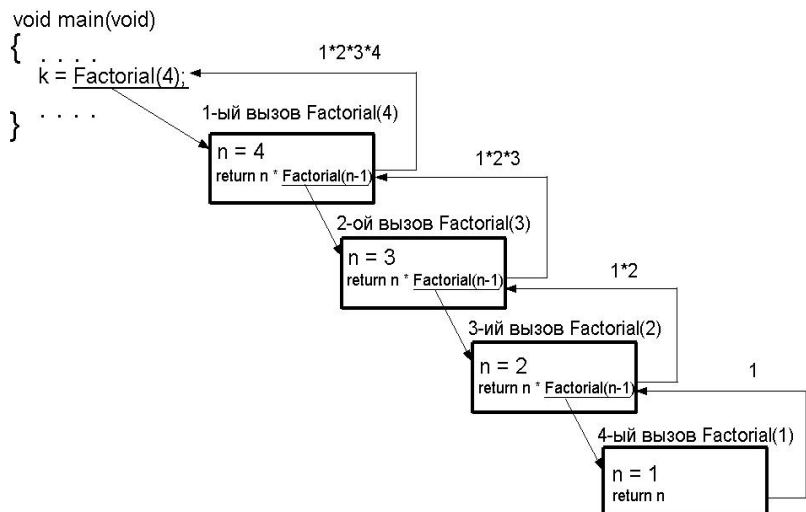


Рис. 3.2.5. Возврат из вызова функции *Factorial(4)*

Задача 2. Вычислить n -е число Фибоначчи. Известно, что $F_1 = F_2 = 1, F_i = F_{i-1} + F_{i-2}$ для $i = 3, 4, \dots$, т.е. каждый член последовательности чисел Фибоначчи, начиная с третьего, равен сумме двух предыдущих.

Рекурсивный характер вычисления чисел Фибоначчи очевиден. Приведем программный код рекурсивной функции вычисления чисел Фибоначчи в листинге 3.2.2.

```
// определение функции вычисления n-го числа Фибоначчи
int Fibonacci(int n)
{
    // числа Фибоначчи вычисляются только для n > 0
    if(n <= 0)
        return -1;
    // условие выхода из рекурсии
    if(n == 2 || n == 1)
        return 1;
    else
        // рекурсивный вызов функции
        return Fibonacci(n - 2) + Fibonacci(n - 1);
}
```

Листинг 3.2.2. Рекурсивная функция вычисления
 n -го числа Фибоначчи на языке C++

Это пример задачи, в которой рекурсивный алгоритм очевиден, но его использование приводит к большим вычислительным расходам и излишним затратам памяти. В теле функции *Fibonacci()* несколько раз осуществляется рекурсивный вызов. Каждый вызов требует выделения памяти. Например, для вычисления F_{10} необходимо вызвать функцию *Fibonacci()* для вычисления F_9 и F_8 . В свою очередь, для вычисления F_9 снова понадобится вызвать функцию *Fibonacci()* для вычисления F_8 и т. д., согласно приведенной ниже схеме:

$$\begin{array}{ccccccc} & & & & F_{10} = F_9 + F_8 & & \\ & & & & & & F_8 = F_7 + F_6 \\ F_9 = F_8 + F_7 & & & & & & \\ F_8 = F_7 + F_6 & F_7 = F_6 + F_5 & & F_6 = F_5 + F_4 & & & \\ & \cdot & \cdot & \cdot & \cdot & & \end{array}$$

Из схемы видно, что для некоторых аргументов приходится осуществлять повторные вызовы функции *Fibonacci()*, т.е. при больших n будет происходить много лишних вычислений.

Задача 3. Задача о Ханойской башне.

Согласно легенде, в одном из буддийских монастырей монахи уже тысячу лет занимаются перекладыванием колец. Они располагают тремя стержнями, на которых надеты кольца разных размеров, образующих пирамиды. В начальном состоянии 64 кольца были надеты на первую пирамиду и упорядочены по размеру.

Монахи должны переложить все кольца с первой пирамиды на вторую, выполняя единственное условие – никакое кольцо нельзя положить на кольцо меньшего размера. При перекладывании можно использовать все три пирамиды.

Монахи перекладывают одно кольцо за одну секунду. Как только они закончат свою работу, наступит конец света.

Количество перекладываний в зависимости от количества колец вычисляется по формуле $2^n - 1$. Для 64 колец эта величина составляет 18 446 744 073 709 551 615 перекладываний, и если принять скорость «одно перекладывание в секунду», получится около 584 942 417 355 лет, то есть апокалипсис наступит нескоро.

Сформулируем условие задачи, которую должна решить рекурсивная функция. Даны три стержня a , b , c . На одном нанизана пирамида из n колец разного радиуса. Требуется перенести всю пирамиду на другой стержень, используя в качестве вспомогательного стержня третий.

Если требуется перенести пирамиду, состоящую из одного кольца, то функция печатает сообщение о переносе этого кольца на нужный стержень.

На рис. 3.2.6 показана ситуация переноса пирамиды из n колец при $n > 1$ (рис. 3.2.6а – исходное состояние). Для такого переноса требуется сначала перенести пирамиду размера $(n - 1)$ на вспомогательный стержень (рис. 3.2.6б). Далее перенести оставшееся кольцо на нужный стержень (рис. 3.2.6в), а затем перенести пирамиду из $(n - 1)$ кольца со вспомогательного стержня на нужный (рис. 3.2.6г). Этот алгоритм приведен в листинге 3.2.3 и реализован в виде рекурсивной функции *HanoiTower()*, которая имеет следующие параметры: первый параметр – номер стержня источника, второй параметр – номер стержня назначения, третий параметр – вспомогательный стержень, четвертый параметр – количество колец в пирамиде.

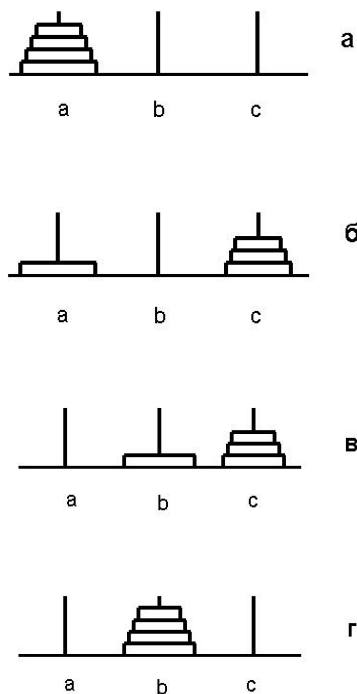


Рис. 3.2.6. Процесс переноса пирамиды со стержня *a* на стержень *b*

```
// определение функции решения задачи о Ханойской башне
void HanoiTower(int a, int b, int c, int n)
{
    if(n == 1) // перенос одного кольца
        cout << a << " -> " << b << endl;
    else
    {
        // перенос пирамиды из n-1 кольца с a на c
        HanoiTower(a, c, b, n - 1);
        // перенос одного кольца самого большого радиуса
        cout << a << " -> " << b << endl;
        // перенос пирамиды из n-1 кольца из c на b
        HanoiTower(c, b, a, n - 1);
    }
}
```

Листинг 3.2.3. Рекурсивная функция для задачи о Ханойских башнях на языке C++

Задача 4. Лабиринт задан в виде прямоугольной матрицы, в которой используются следующие обозначения: 0 – возможен проход, 1 – стена. Даны координаты позиции путника в лабиринте. Требуется определить, сможет ли путник выйти из лабиринта. Считается, что путник вышел из лабиринта, если он находится на первой или последней строке матрицы лабиринта или на первом или последнем столбце. Путник может передвигаться по лабиринту, перемещаясь только по горизонтали или по вертикали.

Алгоритм решения данной задачи связан с использованием рекурсивных функций. Находясь в некоторой ячейке, путник может двигаться в четырех направлениях, включая то, откуда он попал в эту ячейку. Осуществляется перебор всех этих направлений. Если по направлению можно сдвинуться (соседняя ячейка по направлению содержит 0), то делаем шаг по этому направлению. Если ни по одному направлению сдвинуться нельзя (все соседние ячейки содержат 1, кроме той, откуда путник попал в текущую ячейку, или все пути из соседних ячеек привели в тупик), то осуществляется обратный ход в ту ячейку, откуда путник попал в текущую. Если в результате работы алгоритма будет осуществлен возврат в начальную ячейку, делается вывод о том, что путник не может выйти из лабиринта.

В листинге 3.2.4 приведен программный код рекурсивной функции поиска путником выхода из лабиринта. Рекурсивная функция получает в качестве параметров матрицу, задающую лабиринт, координаты текущей позиции путника (x, y), координаты предыдущей позиции путника ($xprev, yprev$). В момент первого вызова функции можно принять в качестве предыдущей позиции исходную позицию путника. При движении по лабиринту позиции, в которых путник уже был, будут помечены значением 2. При возврате данная пометка снимается.

```
// определение функции поиска выхода из лабиринта
bool Labirint(int** a, int m, int n, int x, int y,
              int xprev, int yprev)
{
    // помечаем пройденную ячейку
    a[x][y]=2;
    // печать координат пройденной ячейки
    cout << "( " << x << " , " << y << " ) " << endl;
    // условие выхода из рекурсии - выход из лабиринта найден
    if(x==0 || y==0 || x==m-1 || y==n-1)
        return 1;
```

Листинг 3.2.4. Рекурсивная функция для задачи о выходе из лабиринта на языке C++ (Начало)

```

bool f = 0; // значение f равно true, если выход найден, и
           // false в противном случае

// попытка перейти в ячейку, расположенную внизу
if(a[x + 1][y] == 0 && !(x + 1 == xprev && y == yprev))
    f = Labirint(a, m, n, x + 1, y, x, y); // рекурсивный вызов

// попытка перейти в ячейку, расположенную справа,
// если выход еще не найден
if(!f && a[x][y + 1] == 0 && !(x == xprev && y + 1 == yprev))
    f = Labirint(a, m, n, x, y + 1, x, y); // рекурсивный вызов

// попытка перейти в ячейку, расположенную сверху,
// если выход еще не найден
if(!f && a[x - 1][y] == 0 && !(x - 1 == xprev && y == yprev))
    f = Labirint(a, m, n, x - 1, y, x, y); // рекурсивный вызов

// попытка перейти в ячейку, расположенную слева
// если выход еще не найден
if(!f && a[x][y - 1] == 0 && !(x == xprev && y - 1 == yprev))
    f = Labirint(a, m, n, x, y - 1, x, y); // рекурсивный вызов

// возврат в предыдущую ячейку, снятие пометки
a[x][y] = 0;
return f;
}

```

Листинг 3.2.4. Рекурсивная функция для задачи о выходе из лабиринта на языке C++ (Окончание)

Домашнее задание

1. Написать рекурсивную функцию вычисления

$$n!! = n \cdot (n - 2) \cdot (n - 4) \dots$$

Последний сомножитель в произведении равен 2, если n – четное число, 1 – в противном случае.

2. Написать рекурсивную функцию вычисления числа сочетаний из n по k (C_n^k) с помощью следующих рекуррентных соотношений:

$$C_n^0 = C_n^n = 1, \\ C_n^k = C_{n-1}^k + C_{n-1}^{k-1} \text{ где } 0 < k < n.$$

-
3. Написать рекурсивную функцию вычисления наибольшего общего делителя двух положительных целых чисел m и n при $m > n$:

$$\text{НОД}(m, n) = \text{НОД}(n, m \% n), \text{ если } n \neq 0, \text{НОД}(m, 0) = n.$$

4. Написать рекурсивную функцию вычисления суммы цифр из записи целого положительного числа, не используя операторов цикла.
5. Написать рекурсивную функцию вычисления максимального элемента в массиве целых чисел, не используя операторов цикла.
6. Написать рекурсивную функцию проверки, образуют ли элементы массива симметричную последовательность, не используя операторов цикла.
7. Написать рекурсивную функцию нахождения индекса заданного элемента k в массиве целых чисел методом бинарного поиска, не используя операторов цикла.
8. Написать рекурсивную функцию, формирующую все перестановки n элементов массива.
9. Написать рекурсивную функцию поиска корня уравнения $f(x) = 0$ на отрезке $[a, b]$:
- а) методом деления отрезка пополам;
 - б) методом хорд;
 - в) методом касательных.

Описание методов приводится в разделе 3.2.

10. Написать рекурсивную функцию быстрой сортировки массива (алгоритм Хоара).

Алгоритм заключается в следующем. Находится такой элемент массива, который разбивает массив на два подмножества. Первое подмножество составляют элементы, которые по величине меньше разбивающего элемента, второе подмножество – элементы, которые по величине не меньше его. Далее применяется этот же алгоритм разбиения для каждого из полученных подмножеств.

Элемент, разделяющий массив, может быть определен следующим образом. Вводятся два индекса (i и j) для просмотра элементов слева и справа. Сравниваются i -й и j -й элементы, и если обмен не требуется, то $j = j - 1$, и этот процесс

повторяется. После первого обмена $i = i + 1$. Сравнение продолжается с увеличением i до тех пор, пока не произойдет еще один обмен. Тогда опять уменьшается j и т. д., пока не станет $i = j$.

11. Написать рекурсивную функцию вычисления определителя n -го порядка по правилу Лапласа с разложением по произвольной строке.
12. Написать рекурсивную функцию обхода всей шахматной доски размером 8×8 конем из заданной позиции. В каждой клетке конь должен побывать только один раз. Если такого обхода не существует, выдать соответствующее сообщение, иначе показать путь следования коня.
13. Дан набор предметов, имеющих определенный вес и ценность. Требуется поместить часть этого набора в рюкзак заданной грузоподъемности так, чтобы ценность полученного набора была максимальной (задача о рюкзаке). Написать рекурсивную функцию решения этой задачи.
14. Пусть имеется n городов. Дана матрица стоимостей проезда между городами (a_{ij} – стоимость проезда из города i в город j , если пути из i -го города в j -й не существует, $a_{ij} = -1$). Требуется найти замкнутый маршрут, т. е. такой, который начинается и заканчивается в одном и том же городе, например в городе № 1. При этом каждый город должен быть посещен ровно один раз и стоимость проезда по маршруту должна быть минимальной (задача коммивояжера). Написать рекурсивную функцию решения этой задачи.

Глава 4. Линейные динамические структуры данных

4.1. Односвязные списки

Связные списки – это способ хранения данных, который применяется в тех случаях, когда заранее неизвестно количество элементов или когда размер хранимых данных постоянно меняется. Например, списки удобно использовать в приложении, имитирующем работу телефонного автоответчика. Сообщения, которые поступают на автоответчик, должны сохраняться в некоторой структуре данных. Прослушанные сообщения требуется из этой структуры удалять.

Если использовать для этих целей массив, то нужно сразу указывать его размер достаточным для хранения максимального количества возможных элементов, что не всегда получается оценить. Если выделенной памяти не хватает для хранения всех элементов, приходится производить реорганизацию массива – выделение новой памяти необходимого размера, перенос всех элементов из прежней памяти в новую и освобождение той области памяти, которую ранее занимал массив.

Главное достоинство динамических структур данных, самым простым примером которых является связный список, состоит в том, что только в момент добавления нового элемента под него выделяется память. Соответственно в момент удаления элемента память, занятая им, освобождается. Часто элемент списка называют также узлом.

Однако по сравнению с массивами здесь имеется и недостаток – заранее неизвестно местоположение каждого элемента. Поэтому приходится вместе с самими данными хранить дополнительную информацию об их местоположении. Таким образом, для хранения динамических структур данных требуется больше памяти.

Поскольку связный список должен хранить вместе данные и информацию об их местоположении, при применении списков в программе используют структуры как тип данных, в который может входить несколько различных значений. Так, нам в структуре необходимы переменные, хранящие информационные поля списка, и переменная, содержащая адрес памяти следующего (или предыдущего) узла списка. Для хранения адресов памяти в большинстве языков программирования существуют специальные типы данных. Например, в языке C++ для этих целей применяются указатели, для языка C# их роль выполняют ссылки на объекты классов – типы данных, аналогичные структурам, предназначенным для разработки программ в стиле объектно-ориентированного программирования.

Односвязным называют список, каждый узел которого содержит адрес следующего элемента списка. Предположим, что для реализации автоответчика необходимо хранить следующие данные: номер телефона звонившего, его имя и само сообщение. Схематично вид этого списка будет выглядеть так, как это представлено на рис. 4.1.1. В поле, из которого исходит стрелка, хранится адрес следующего элемента. Стрелка как раз и указывает на этот элемент.

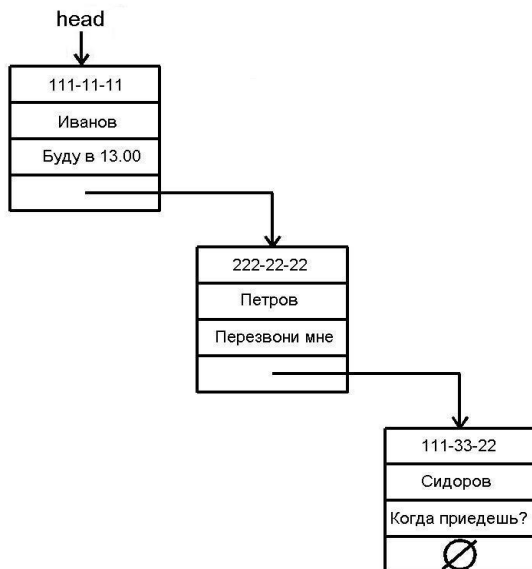


Рис. 4.1.1. Вид односвязного списка

Структуру для хранения элемента такого списка можно определить так:

```
struct PhoneMessage
{
    char phone[12];           // номер телефона
    char name[30];            // имя
    char message[300];        // сообщение
    PhoneMessage * next;      // адрес следующего сообщения
};
```

Чтобы правильно работать с односвязным списком, необходимо отдельно хранить адрес его первого элемента (переменная *head*). Тогда, двигаясь по цепочке, можно получить доступ к любому узлу. Признаком конца списка является пустая (нулевая) ссылка в поле адреса следующего элемента. Например, в языке C++ для обозначения пустой ссылки используется константа *NULL*, в C# и Java – *null*, в Pascal – *nil*.

Далее приводится описание процесса решения некоторых задач с применением пользовательских функций для построения, реорганизации и обработки односвязных списков.

Задача 1. Пусть имеется непустой односвязный список, который содержит целые числа (рис. 4.1.2). Написать функцию нахождения суммы элементов списка.



Рис. 4.1.2. Исходный список

```
// структура для хранения узла списка
struct List
{
    int info;           // информационное поле
    List* next;         // указатель на следующий узел
                       // списка
};
```

Чтобы найти сумму элементов списка, нужно пройти по всем его узлам, добавляя значение информационного поля (переменная *info*) к переменной, хранящей сумму. Для этого заводим переменную *current*, которая сначала хранит адрес первого элемента списка. В цикле последовательно по цепочке сдвигаем ее значение на следующий элемент (*current = current -> next*). Условие окончания цикла – значение *NULL* в переменной *current*. В листинге 4.1.1 приведем код функции поиска суммы элементов односвязного списка на языке C++.

При реализации на языке C# следует использовать вместо структуры класс с открытыми членами класса. Это обусловлено тем фактом, что в языке C# ссылочным типом данных является именно класс, а не структура. Именно ссылочный тип данных обеспечивает передачу параметров и хранения адреса памяти, по которому хранятся данные. Структуры же в C# относятся к значимым типам данных, т. е. к тем, которые передаются по своему значению путем копирования. Так, класс для хранения одного узла списка будет выглядеть так:

```
// класс для хранения узла списка
class List
{
    public int info;     // информационное поле
    public List next;    // ссылка на следующий узел списка
};
```

Программный код функции получения суммы элементов списка на языке C# будет приведен в листинге 4.1.2.


```
// определение функции подсчета суммы элементов списка
int SummaOfList(List* head)
{
    int s = 0;
    // просмотр элементов списка начинается
    // с его первого элемента
    List* current = head;
    while(current != NULL) // пока список не закончен
    {
        s += current -> info; // добавление значения текущего
                               // элемента к сумме
        current = current -> next; // переход на следующий
                                   // элемент списка
    }
    return s;
}
```

Листинг 4.1.1. Функция получения суммы элементов односвязного списка на языке C++

```
// определение функции подсчета суммы элементов списка
int SummaOfList(List head)
{
    int s = 0;
    // просмотр элементов списка начинается
    // с его первого элемента
    List current = head;
    while(current != null) // пока список не закончен
    {
        s += current.info; // добавление значения текущего
                           // элемента к сумме
        current = current.next; // переход на следующий
                                // элемент списка
    }
    return s;
}
```

Листинг 4.1.2. Функция получения суммы элементов односвязного списка на языке C#

Как видно из листингов 4.1.1 и 4.1.2, существенных синтаксических различий в реализации односвязных списков на языках C++ и C# нет. Но стоит заметить, что в языке C# программный код является легче читаемым. Это связано как раз с тем, что при разработке языка C# старались устранить все технические сложности, связанные с применением переменных-указателей.

Задача 2. Пусть односвязный список содержит целые числа. Написать функцию добавления нового элемента в список в заданную позицию. Эту же функцию можно использовать для создания списка.

При решении задачи можно выделить три частных случая добавления элемента: вставка первого узла, вставка в середину и в конец списка. Если происходит вставка первого узла, требуется изменить значение переменной *head* (рис. 4.1.3). При вставке элемента в другую позицию требуется переместиться по списку на элемент, предшествующий месту вставки. Например, если требуется вставить элемент на пятую позицию, нужно переместиться на четвертый узел списка. Затем перенаправить адреса согласно рис. 4.1.4.

Пусть происходит вставка первого элемента. Сначала выделяем память под хранение первого узла и его адрес записываем в переменную *help* (рис. 4.1.3а). Затем заносим в информационное поле нового узла значение *key*. Теперь требуется связать новый узел со всем списком. Для этого в поле *next* нового узла *help* заносим адрес прежнего первого элемента списка (*head*) (рис. 4.1.3б). Теперь осталось только изменить начало списка, т. е. *head* = *help* (рис. 4.1.3в). Заметим, что вставка пройдет корректно, даже если список был пустым. В этом случае перед вставкой *head* = *NULL*. После вставки *head* будет ссылаться на новый элемент, а поле *next* примет значение *NULL*.

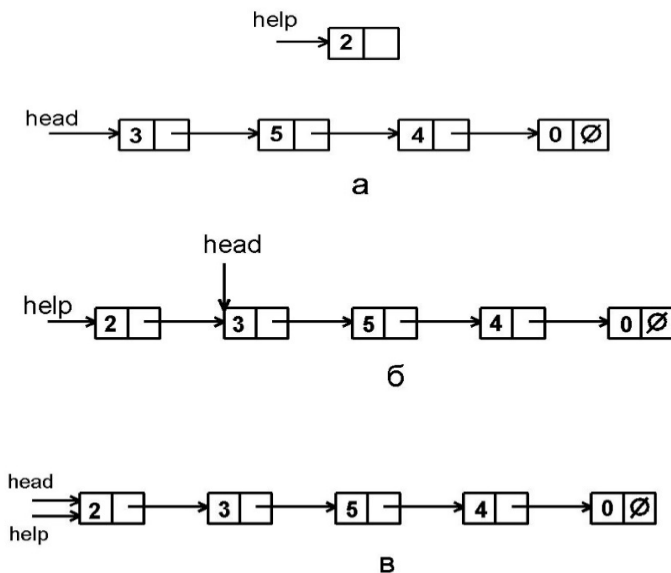


Рис. 4.1.3. Вставка элемента в начало списка

При вставке элемента в середину (в конец) списка надо переместиться на элемент, предшествующий месту вставки. Для этого заводим переменную *current*, которая сначала хранит адрес первого элемента списка. В цикле последовательно по цепочке сдвигаем ее значение на следующий элемент (*current = current -> next*). Это нужно делать до тех пор, пока не получим адрес элемента в нужной позиции (*i = pos - 1*) или пока не закончится список (в этом случае *current = NULL*). Если список закончится, то вставка не производится. Если же остановка цикла произошла, когда *current* хранит адрес существующего элемента списка, выполняем следующие действия. Сначала выделяем память под хранение нового узла и его адрес записываем в переменную *help* (рис. 4.1.4а). Затем заносим в информационное поле нового узла значение *key*. Теперь «встраиваем» новый элемент в середину списка. Для этого сначала указываем, что следующим за новым элементом должен быть элемент *current -> next* (*help -> next = current -> next*) (рис. 4.1.4б) и делаем ссылку из *current* на новый элемент (*current -> next = help*) (рис. 4.1.4в).

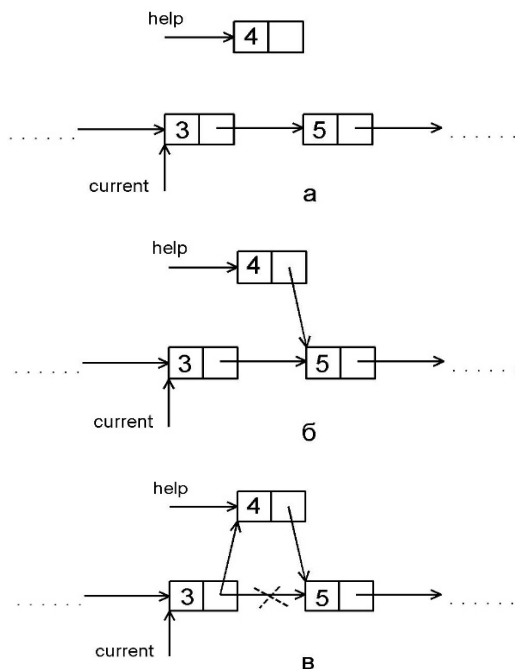


Рис. 4.1.4. Вставка в середину списка

Заметим, что это нужно сделать именно в этом порядке. Если поменять эти действия местами, будет потеряян «хвост» списка (рис. 4.1.5).

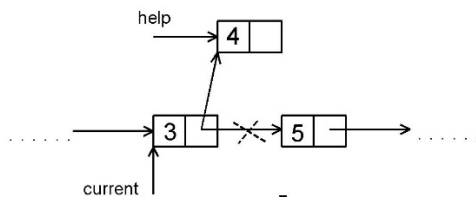


Рис. 4.1.5. Случай потери «хвоста» списка

Приведем в листинге 4.1.3 программный код функции добавления нового элемента в список на языке C++, а в листинге 4.1.4 – программный код функции на языке C#. Входными параметрами функции является список, заданный с помощью адреса первого элемента списка (*head*) (ссылки на первый элемент в C#), а также вводимое значение (*key*) и место вставки (*pos*). Отметим, что так как адрес первого элемента списка может измениться, параметр *head* передается в функцию по ссылке (тип параметра *List*&*) в языке C++ и объявлен явно как выходной параметр (с модификатором *ref*) в языке C#.

```
// определение функции добавления в список
// элемента key на позицию pos
void AddList(List*& head, int key, int pos)
{
    List* current, *help;
    // добавление в начало списка
    if(pos == 1)
    {
        // выделение памяти под новый элемент списка
        help = new List;
        // заполнение нового элемента списка данными
        help -> info = key;
        help -> next = head;
        // изменение адреса первого элемента списка
        head = help;
        return;
    }
}
```

Листинг 4.1.3. Функция вставки нового элемента в односвязный список на языке C++ (Начало)

```

// поиск элемента, после которого требуется
// вставить новый элемент
current = head;
int i = 1;
// проход по списку до позиции pos - 1
while (i != pos - 1 && current != NULL)
{
    i++;
    current = current -> next;
}

// позиция не найдена, список оказался коротким
if(current == NULL)
{
    cout << "Некорректная позиция." << endl;
    return;
}

// вставка элемента в позицию pos
// выделение памяти под новый элемент списка
help = new List;
// заполнение нового элемента данными
help -> info = key;
help -> next = current -> next;
// перенаправление указателей в списке
current -> next = help;
}

```

Листинг 4.1.3. Функция вставки нового элемента в односвязный список на языке C++ (Окончание)

Далее будем приводить программный код только на одном из языков программирования, так как серьезной разницы в программном коде на C++ и C# нет.

Задача 3. Пусть односвязный список содержит целые числа. Написать функцию удаления всех элементов списка с заданным значением.

Решение задачи можно разделить на две части. Сначала удаляем все элементы с заданным значением (оно хранится в переменной *key*), находящиеся в начале списка. Это нужно для того, чтобы при удалении первого элемента изменялся заголовок списка (переменная *head*). Удаление первого элемента по шагам показано на рис. 4.1.6.

Если новый первый элемент будет хранить заданное значение, то процедура удаления повторяется с новым значением переменной *head*.

```

// определение функции добавления в список
// элемента key на позицию pos
void AddList(ref List head, int key, int pos)
{
    List current, help;
    // добавление в начало списка
    if(pos == 1)
    {
        // выделение памяти под новый элемент списка
        help = new List();
        // заполнение нового элемента списка данными
        help.info = key;
        help.next = head;
        // изменение адреса первого элемента списка
        head = help;
        return;
    }
    // поиск элемента, после которого требуется
    // вставить новый элемент
    current = head;
    int i = 1;
    // проход по списку до позиции pos - 1
    while (i != pos - 1 && current != null)
    {
        i++;
        current = current.next;
    }
    // позиция не найдена, список оказался коротким
    if(current == null)
    {
        Console.WriteLine("Некорректная позиция.");
        return;
    }
    // вставка элемента в позицию pos
    // выделение памяти под новый элемент списка
    help = new List();
    // заполнение нового элемента данными
    help.info = key;
    help.next = current.next;
    // перенаправление указателей в списке
    current.next = help;
}

```

Листинг 4.1.4. Функция вставки нового элемента в односвязный список на языке C#

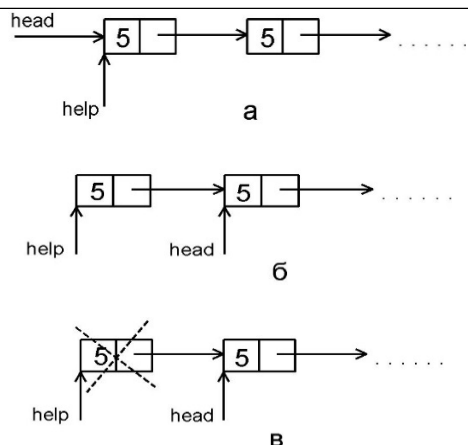


Рис. 4.1.6. Удаление первого элемента из начала списка

Если после первого цикла список не окажется пустым, удаляем элементы с заданным значением из середины списка. Переменную *current* устанавливаем на начало списка. Удаляться будет следующий за *current* элемент, адрес которого будет храниться в переменной *help* (рис. 4.1.7а). Если нужный элемент найден, то после удаления следующим за *current* должен быть элемент, идущий за *help*, иначе будет потерян весь остаток списка ($current \rightarrow next = help \rightarrow next$) (рис. 4.1.7б). Теперь элемент *help* можно удалить (*delete help*) (рис. 4.1.7в). В этом случае значение *current* не меняется, поскольку после удаления за ним может идти элемент, который, возможно, имеет заданное значение. Если же в *help* хранится значение, отличное от значения переменной *key*, сдвигаем *current* на следующий узел.

Отметим, что для реализации на языке C# шаг, изображенный на рис. 4.1.7в, не требуется, так как освобождением памяти в C# занимается компонента «сборщик мусора». Память будет им освобождена после окончания выполнения функции, так как больше не будет переменных, хранящих ссылку на этот элемент.

Программный код функции удаления элемента по ключу на языке C++ представлен в листинге 4.1.5. Функция имеет входные параметры – ссылка на адрес первого элемента списка (так как элементы могут удаляться из начала списка) (*head*) и заданное значение удаляемого элемента (*key*).

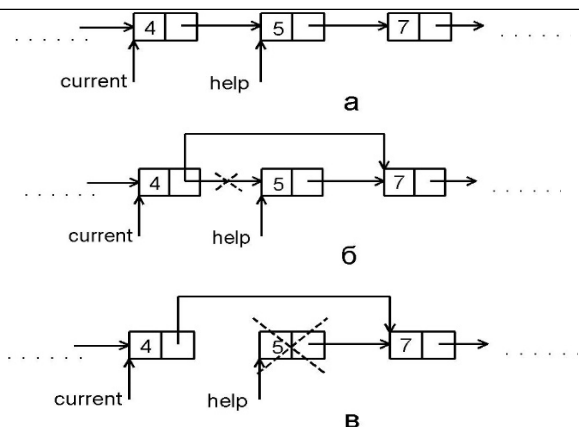


Рис. 4.1.7. Удаление из середины списка

```
// определение функции удаления элементов списка по ключу
void DeleteKey(List*& head, int key)
{
    List* current, *help;
    // удаление всех элементов с заданным значением,
    // расположенных в начале списка
    while(head != NULL && head -> info == key)
    {
        help = head;
        head = head -> next;
        delete help;
    }
    // если элементы в списке еще остались, продолжаем поиск
    if(head != NULL)
    {
        current = head;
        // поиск и удаление элементов с заданным значением,
        // расположенных в середине или в конце списка
        while(current -> next != NULL)
        {
            help = current -> next;
            if(help -> info == key)
            {
                current -> next = help -> next;
                delete help;
            }
            else current = current -> next;
        }
    }
}
```

Листинг 4.1.5. Функция удаления заданного элемента из односвязного списка на языке C++

Задача 4. Дан односвязный список. Написать функцию удаления всех элементов списка.

Заметим сразу, что для C# такая задача не является актуальной, так как потеря ссылки на первый элемент списка будет основанием автоматического удаления всех элементов списка «сборщиком мусора». Поэтому далее в листинге 4.1.6 будет представлен только программный код функции на языке C++.

Функция является довольно простой. Будем удалять элементы последовательно с начала списка. В переменной *head* будем хранить адрес текущего элемента. Чтобы не потерять «хвост» списка, следующий за удаляемым элементом, надо сохранять адрес следующего элемента в переменной *help*. Удалив текущий элемент, в переменную *head* записываем значение из *help*. Эти действия осуществляются до тех пор, пока список не станет пустым.

```
// определение функции уничтожения списка
void DestroyList(List* head)
{
    List*help;
    // пока список не пуст, удаляем head
    while(head != NULL)
    {
        // запоминаем «хвост» списка
        help = head -> next;
        // удаляем первый элемент списка
        delete head;
        // устанавливаем head в начало «хвоста» списка
        head = help;
    }
}
```

Листинг 4.1.6. Функция удаления всех элементов из односвязного списка на языке C++

Задача 5. Дан односвязный список, содержащий целые числа. Написать функцию разбиения списка на два таких, что в одном будут содержаться только четные числа, в другом – нечетные.

Для разнообразия приведем пример функции на языке программирования C# (листинг 4.1.7). Функция имеет три параметра: *head* – содержит ссылку на начало исходного списка, *head1* – списка с нечетными элементами, *head2* – с четными. Отметим, что первый параметр передается как *ref*-параметр, так как после работы функции исходного списка больше не будет и переменная *head* должна стать равной *null*. Две ссылки на списки-результаты передаются как *out*-параметры.

Переменные *current1*, *current2* используются для хранения текущих элементов создаваемых списков. Начальные значения *head1*, *head2*, *current1*, *current2* будут пустыми ссылками, т. е. устанавливаются в *null*, поскольку результирующие списки еще не созданы.

Двигаясь по элементам исходного списка, проверяем, к какой группе относится очередной элемент. Если значение информационного поля четное, добавляем элемент в список с заголовком *head2*. Для этого если в списке еще нет элементов, то значения переменных *head2* и *current2* становятся равными ссылке на этот элемент. Если же в списке уже имеются элементы, то он добавляется в конец:

```
current2.next = head;
current2 = current2.next;
```

Аналогично поступаем и в том случае, когда встречается нечетный элемент, только используем переменные *head1* и *current1*.

```
// определение функции разбиения списка на два -
// в одном четные элементы, в другом - нечетные
static void Decompose(ref List head,
                      out List head1,out List head2)
{
    List current1 = null, current2 = null;
    head1 = null; head2 = null;
    // просмотр элементов исходного списка
    while(head != NULL)
    {
        // если значение текущего элемента четное
        if(head.info % 2 == 0)
        {
            // добавляем элемент в список
            // с четными значениями элементов
            if(head2 == null)
                // список еще пуст
                // формируем заголовок списка
                head2 = current2 = head;
            else
            {
                // список не пуст,
                // добавляем в конец новый элемент
                current2.next = head;
                current2 = current2.next;
            }
        }
    }
}
```

Листинг 4.1.7. Функция разбиения односвязного списка на языке C#
(Начало)

```

else
{
    // добавляем элемент в список
    // с нечетными значениями элементов
    if(head1 == NULL)
        // список еще пуст
        // формируем заголовок списка
        head1 = current1 = head;
    else
    {
        // список не пуст,
        // добавляем в конец новый элемент
        current1.next = head;
        current1 = current1.next;
    }
}
// переходим на следующий элемент
// в исходном списке
head = head.next;
}
// установка признаков конца построенных списков
if(current1 != null)
    current1.next = null;
if(current2 != null)
    current2.next = null;
}

```

Листинг 4.1.7. Функция разбиения односвязного списка на языке C#
(Окончание)

Списки часто используются как удобный способ хранения многочленов. Каждое слагаемое полинома задается одним элементом списка. В нем хранится значение коэффициента и показатель степени. Структура для хранения отдельного слагаемого имеет следующий вид (язык C++):

```

struct Summand
{
    double a;        // коэффициент
    int n;           // степень
    Summand* next;   // указатель на следующее слагаемое
};

```

Приведем в листинге 4.1.8 функцию создания односвязного списка, хранящего многочлен, посредством ввода с клавиатуры. Параметром функции является ссылка на первый элемент создаваемого списка. Оставляем алгоритм работы функции для самостоятельного разбора читателем.

```

// определение функции создания многочлена
void CreatePolynom(Summand*& head)
{
    double koef;
    int pow;
    int m = 0; // количества слагаемых в многочлене
    Summand* current, *help;
    cout << "Введите полином:" << endl;
    cout << "Введите коэффициент и степень:" << endl;
    // завершение цикла произойдет по нажатию клавиш Ctrl+Z
    // операция ввода может интерпретироваться как условие
    // продолжения цикла при успешном вводе
    while(cin >> koef >> pow)
    {
        if(m == 0)
        {
            // вводится первое слагаемое многочлена
            head = new Summand;
            head -> a = koef;
            head -> n = pow;
            head -> next = NULL;
            current = head;
        }
        else
        {
            // вводится очередное слагаемое многочлена
            current -> next = new Summand;
            current = current -> next;
            current -> a = koef;
            current -> n = pow;
            current -> next = NULL;
        }
        m++; // подсчет очередного слагаемого
        cout << "Введите коэффициент и степень:" << endl;
    }
}

```

Листинг 4.1.8. Функция создания списка для хранения многочлена на языке C++ (Начало)

Задача 6. Имеется односвязный список, определяющий многочлен. Требуется вычислить значение этого многочлена в заданной точке.

Используя прием прохода по всем элементам списка, рассмотренный ранее (например, в задаче 1), вычисляем значение каждого слагаемого многочлена и добавляем его к переменной, хранящей сумму.

Функция вычисления значения многочлена представлена в листинге 4.1.9 (язык C++).

```

// определение функции вычисления значения многочлена
// в заданной точке x
double CalculatePolynom(Summand* head, double x)
{
    double p_x = 0;
    if (head != NULL)
    {
        Summand* current = head;
        while (current != NULL)
        {
            p_x += current -> a * pow(x, current -> n);
            current = current -> next;
        }
    }
    return p_x;
}

```

Листинг 4.1.8. Функция вычисления значения многочлена в точке на языке C++ (Окончание)

Задача 7. Имеется односвязный список, задающий многочлен. Требуется привести подобные слагаемые (предполагается, что многочлен может состоять из нескольких слагаемых с одинаковыми показателями степени).

Алгоритм решения данной задачи заключается в следующем. Берется очередной элемент списка (переменная *current*) и просматриваются все элементы, следующие за ним (переменная *help*). Если показатели степеней у двух рассматриваемых элементов совпадают, то коэффициенты складываются и помещаются в поле *info* узла, на который указывает *current*, а элемент *help* из списка удаляется.

Представим программный код функции приведения подобных слагаемых многочлена в листинге 4.1.9 (язык C++). Отметим, что не требуется передавать адрес первого элемента по ссылке, так как он не может измениться. Может быть изменено значение коэффициента (поле *a* структуры *Summand*) первого слагаемого за счет нахождения подобного ему, но оно не может быть удалено.

```

// определение функции приведения
// подобных слагаемых многочлена
void SimilarSummand(Summand* head)
{
    Summand* current, *help, *prev;
    current = head;
    while(current != NULL)
    {
        // поиск всех подобных слагаемых
        // для слагаемого, хранящегося в current,
        // начиная со следующего за ним;
        // адрес предыдущего слагаемого необходим
        // для последующего удаления
        prev = current;
        help = current -> next;
        while(help != NULL)
        {
            if(help -> n == current -> n)
            {
                // нашли подобное слагаемое
                // приводим его и удаляем из списка
                current -> a += help -> a;
                prev -> next = help -> next;
                delete help;
                help = prev -> next;
            }
            else
            {
                // ищем следующее подобное слагаемое
                prev = help;
                help = help -> next;
            }
        }
        // смещаем current для поиска подобных слагаемых для
        // слагаемых с другой степенью
        current = current -> next;
    }
}

```

Листинг 4.1.9. Функция приведения подобных слагаемых многочлена

Домашнее задание

1. Дан односвязный список. Написать функцию печати его элементов.
2. Дан односвязный список. Написать функцию удаления элемента списка, расположенного в заданной позиции.

-
3. Написать функцию создания копии имеющегося односвязного списка.
 4. Дан односвязный список. Написать функцию, которая:
 - формирует новый список, расположив элементы исходного списка в обратном порядке;
 - располагает элементы в обратном порядке в исходном списке.
 5. Дан односвязный список. Написать функцию добавления в начало списка нового элемента, значение которого равно количеству вхождений минимального элемента в список.
 6. Дан односвязный список. Написать функцию добавления двух новых элементов – с минимальным значением в начало списка и с максимальным значением в конец списка.
 7. Дан односвязный список. Написать функцию удаления из него всех элементов с минимальным значением.
 8. Написать функцию сортировки элементов односвязного списка методом вставок.
 9. Написать функцию слияния двух отсортированных односвязных списков.
 10. Даны два односвязных списка. Написать функцию, проверяющую, содержится ли первый список во втором.
 11. Даны два односвязных списка. Написать функцию, формирующую новый список, состоящий из элементов, которые содержатся в обоих исходных списках.
 12. Даны два односвязных списка. Написать функцию формирования нового списка из тех элементов первого списка, которые не содержатся во втором.
 13. Дан односвязный список. Написать функцию нахождения суммы элементов, расположенных между минимальным и максимальным элементами списка.
 14. Даны два односвязных списка. Написать функцию формирования нового списка, состоящего из тех элементов первого списка, которые входят во второй список более одного раза.

15. Написать функцию печати формулы многочлена.

16. Написать функции, реализующие операции с многочленами: сложение, вычитание, умножение на число, умножение двух полиномов, возведение многочлена в целую степень.

17. Написать приложение для работы с односвязным списком, узлы которого содержат информацию о полученных сообщениях (автоответчик, SMS и пр.). Требуется реализовать возможности добавления нового сообщения, просмотра всех сообщений, поиска сообщений по заданному отправителю, поиска всех сообщений по заданной дате, удаления выбранного сообщения, удаления всех сообщений, сортировки сообщений списка по дате или по отправителю.

Стек и очередь – это такие динамические структуры данных (подобно спискам), в которых добавление и извлечение элементов происходит по четко определенным правилам. Стек работает по правилу LIFO (Last In First Out), т. е. «последним вошел, первым вышел». Очередь работает по правилу FIFO (First In First Out), т. е. «первым вошел, первым вышел». Для реализации стеков и очередей можно использовать массивы или динамические списки. Добавление нового элемента в стек осуществляется в конец массива или списка, а извлечение элемента – из конца массива или списка. Добавление элемента в очередь осуществляется в конец массива или списка, а извлечение элемента – из начала массива или стека.

Стеки и очереди – это всего лишь частные случаи односвязного списка, однако они находят большой спектр применения как вспомогательные структуры хранения при построении различных алгоритмов обработки данных. Особое место здесь отводится алгоритмам решения задач разбора математических выражений, обхода деревьев и графов. Некоторые из этих алгоритмов будут изложены при описании соответствующих заданий, предназначенных для самостоятельного решения.

Задача 1. Написать функции добавления и извлечения элемента из стека. Стек реализовать в виде односвязного списка.

По аналогии с реализацией списка требуется отдельная структура для хранения одного элемента стека. Эта структура в языке C++ будет иметь следующий вид:

```
struct Node
{
    int info;           // информационное поле
    Node* next;         // указатель на следующий элемент
};
```

Для хранения стека вводится другая структура, в которой хранится адрес начала списка:

```
// структура стека
struct Stack
{
    Node* head; // указатель на вершину стека
                // (начало списка)
};
```

Указатель на последний элемент, вошедший в стек, называется его вершиной. Заголовок (начало) списка в приведенном примере указывает на вершину стека. Поэтому добавление и извлечение элементов стека будет осуществляться с его вершины (работаем только с началом списка).

Для добавления элемента в стек создается новый элемент списка (переменная *p*). Новый элемент добавляется в начало списка. Для этого он связывается с прежним первым элементом (*p -> next = s.head*) и заголовок списка изменяется, указывая на новый элемент (*s.head = p*).

При извлечении элемента из стека возможны два случая: стек пуст или стек не пуст. Поэтому кроме извлекаемого значения приходится формировать еще и код возврата (в нашем примере это возвращаемое значение функции). Код возврата будет равен -1, если стек был пуст, в противном случае он будет равен 1. Извлекаемое из стека значение будет возвращаться через параметр функции *n*, который передается в функцию по ссылке. Вершиной стека станет следующий элемент списка, а бывшая вершина стека будет удалена.

Программный код функций работы со стеком на языке C++ представим в листинге 4.2.1.

```
// определение структуры для одного элемента стека
struct Node
{
    int info;    // информационное поле
    Node* next; // указатель на следующий элемент
};

// структура стека
struct Stack
{
    Node* head; // указатель на вершину стека
               //(начало списка)
};

// определение функции добавления в стек нового элемента
void PushStack(Stack& s, int key)
{
    // создание нового элемента стека и его заполнение
    Node* p = new Node;
    p -> next = s.head;
    p -> info = key;
    // новый элемент будет новой вершиной стека
    s.head = p;
}
```

Листинг 4.2.1. Функции работы со стеком на языке C++
(Начало)

```

// определение функции извлечения элемента из стека
// результат работы функции - флаг - 1, если операция удалась,
// -1 - если операция не удалась (стек был пуст)
int PopStack(Stack& s, int& n)
{
    // проверка стека на пустоту
    if(s.head == NULL)
        return -1;
    // получаем значение, находящееся на вершине стека
    n = s.head -> info;
    // удаляем элемент с вершины стека и смещаем вершину на
    // следующий элемент по списку
    Node* p=s.head;
    s.head=s.head->next;
    delete p;
    return 1;
}

```

Листинг 4.2.1. Функции работы со стеком на языке C++
(Окончание)

Напомним, что, в отличие от предложенной реализации, при написании функций работы со стеком на языке C# требуется создать не структуру, а класс с открытыми элементами (*public*-элементами). В остальном общая логика написания функций будет аналогичной.

Задача 2. Написать функции добавления и извлечения элемента из очереди. Очередь реализовать в виде односвязного списка.

Решение данной задачи приведем на языке программирования C# (листинг 4.2.2).

Структура хранения элемента очереди будет той же, что и для односвязного списка, и стека, – должны храниться информационное поле и ссылка на следующий узел. Однако, так как при выполнении операций с очередью требуется работа с обоими концами списка, представим очередь в виде двух полей – ссылки на начало (*head*) и конец (*tail*) списка.

При добавлении элемента в очередь придется отдельно рассмотреть ситуацию, когда очередь пуста. В этом случае нужно формировать ее начало и конец – они будут указывать на единственный элемент очереди. Если же в очереди имеются элементы, то новый элемент добавляется в конец (*q.tail.next = new Node()*) и конец очереди сдвигается.

Если необходимо извлечь элемент из очереди, то необходимо сообщить о возможности извлечения элемента. Мы будем это делать путем формирования кода возврата, который будет равен -1, если очередь пуста, и 1 – в противном случае. Извлекаемое из очереди значение будет возвращаться через *ref*-параметр функции *n*.

Из очереди всегда извлекается первый элемент. Поэтому его значение сохраняется в параметр n , а затем он удаляется (мы должны обеспечить потерю ссылки на этот элемент). Началом очереди становится следующий элемент списка. Если в очереди был один элемент (после удаления $q.head = null$), то одновременно следует обнулить и $q.tail$.

```
// класс для хранения элемента очереди
class Node
{
    public int info;    // информационное поле
    public Node next;  // ссылка на следующий элемент очереди
}

// класс для представления очереди
class Queue
{
    public Node head;  // ссылка на начало очереди
    public Node tail; // ссылка на конец очереди
}

// функция добавления в очередь нового элемента
static void PushQueue(Queue q, int key)
{
    // очередь пуста - создаем ее первый элемент
    if (q.head == null)
    {
        q.head = q.tail = new Node();
        q.head.info = key;
        q.head.next = null;
        return;
    }
    // очередь не пуста - добавляем элемент в конец очереди
    q.tail.next = new Node();
    q.tail = q.tail.next;
    q.tail.info = key;
    q.tail.next = null;
}

// определение функции извлечения элемента из очереди
static int PopQueue(Queue q, ref int n)
{
    // если очередь пуста, формируем код возврата
    if (q.head == null)
        return -1;
    n = q.head.info;    // получаем элемент из начала очереди
    q.head = q.head.next; // смещаем начало очереди
    if (q.head == null) // если очередь стала пуста,
        q.tail = null; // меняем ссылку на «хвост» очереди
    return 1;
}
```

Листинг 4.2.2. Функции работы с очередью на языке C#

Задача 3. Дан односвязный список. Написать функцию, определяющую, образуют ли его элементы симметричную последовательность.

Функция будет возвращать значение *true*, если список является симметричным, *false* – в противном случае.

По определению пустой список является симметричным. Поэтому если список пуст, то возвращаем значение *true*.

Для проверки симметричности списка нужно проверить на равенство все пары элементов, равноотстоящих от середины списка. Каждая пара содержит один элемент из первой половины списка и один из второй. Поэтому будем элементы первой половины списка последовательно заносить в очередь, второй половины – в стек. Если количество элементов списка будет нечетным, то срединный элемент будет симметричен сам себе и не будет помещен ни в очередь, ни в стек.

Элементы, находящиеся в начале очереди и в вершине стека, образуют пару расположенных симметрично относительно середины списка элементов. Поэтому, извлекая элементы из стека и очереди и проверяя их на равенство, можно определить, симметричен ли заданный список.

Приведем программный код функции проверки односвязного списка на симметричность на языке C++ в листинге 4.2.3.

Задача 4. Дано положительное целое число в десятичной системе счисления. Перевести его в заданную систему счисления.

Как известно, при переводе числа в заданную систему счисления производится последовательное получение остатков от деления числа на основание системы счисления, которые при записи в обратном порядке образуют требуемое число. Поэтому вычисленные остатки от деления заносятся в стек, а затем извлекаются из него для печати. Если основание системы счисления будет больше 10, будем использовать для записи числа буквенные обозначения – на печать будет выводиться соответствующий символ (10 – ‘A’, 11 – ‘B’ и т. д.). Функция должна будет вернуть строку с записью числа в указанной системе счисления.

Программный код функции перевода числа в заданную систему счисления приведем на языке программирования C# в листинге 4.2.4.

```

// определение функции проверки симметричности
// последовательности чисел в односвязном списке
bool Symmetry(List* head)
{
    int n = 0, a, b;
    if (head == NULL)
    {
        cout << "Список пуст!" << endl;
        return true;
    }
    List* current;
    // подсчитываем количество элементов в списке
    for (current = head; current != NULL; current = current->next)
        n++;
    // создание вспомогательных стека и очереди
    Stack s; s.head = NULL;
    Queue q; q.head = q.tail = NULL;
    current = head;
    // запись первой половины элементов списка в очередь
    for (int i = 0; i < n / 2; i++)
    {
        PushQueue(q, current -> info);
        current = current -> next;
    }
    // при нечетном количестве элементов в списке
    // пропускаем элемент, расположенный в середине
    if (n % 2 == 1)
        current = current -> next;
    // запись второй половины элементов списка в стек
    for (int i = 0; i < n / 2; i++)
    {
        PushStack(s, current -> info);
        current = current -> next;
    }
    bool f=true;
    // одновременное извлечение значений элементов
    // из очереди и стека и их сравнение
    for (int i = 0; i < n / 2; i++)
    {
        PopQueue(q, a);
        PopStack(s, b);
        if (a != b)
            f = false;
    }
    return f;
}

```

Листинг 4.2.3. Функция проверки симметричности списка на языке C++

```

// определение функции перевода положительного целого числа
// из десятичной системы счисления в любую заданную
// с основанием от 2 до 16 (параметр b)
string PrintChislo(int number, int b)
{
    Stack s;
    s.head = null;
    int n;           // переменная для разряда числа
    string number_b = ""; // строковое представление записи
                        // в системе счисления с основанием b
    // вычисленные остатки от деления числа
    // на основании системы счисления помещаются в стек
    while(number != 0)
    {
        PushStack(s, number % b);
        number = number / b;
    }
    // формирование разрядов полученного числа в строку number_b
    // путем извлечения их значений из стека
    while(PopStack(s, n) == 1)
    {
        if(n < 10)
            // разряд представляется цифрой
            number_b = number_b + n;
        else
            // разряд представляется буквой
            number_b = number_b + ('A' + n - 10);
    }
    return number_b;
}

```

**Листинг 4.2.4. Функция перевода положительного целого числа
из десятичной системы счисления в заданную с основанием
от 2 до 16 на языке C#**

Домашнее задание

1. Написать функции уничтожения стека и очереди. Уничтожение понимается как очистка стека и очереди от их элементов.
2. Написать функции печати стека и очереди. Алгоритмы этих функций не должны извлекать элементы из стека и очереди.
3. Дан односвязный список, содержащий целые числа. Написать функцию, которая за один проход по списку распечатывает эти числа следующим образом: сначала все четные числа в обратном порядке их следования в списке, затем все нечетные числа в

прямом порядке их следования. Для решения задачи использовать стек и очередь.

4. Дан односвязный список, содержащий целые числа. Написать функцию, которая за один проход по списку распечатывает эти числа таким образом, что сначала распечатываются все отрицательные числа, затем – все положительные. Элементы, равные нулю, печататься не должны. Для решения задачи использовать очередь.
5. Лабиринт задан в виде прямоугольной матрицы, в которой используются следующие обозначения: 0 – возможен проход, 1 – стена. Даны координаты позиции путника в лабиринте. С помощью стека распечатать путь, по которому путник может выйти из лабиринта, если выход существует. Считается, что путник вышел из лабиринта, если он находится на первой или последней строке матрицы или на первом или последнем столбце. Путник может двигаться только по горизонтали или по вертикали.
6. Дана символьная строка с некоторым выражением, в котором могут содержаться скобки трех видов – $()$, $\{\}$, $[]$. Написать функцию проверки правильности расстановки скобок в этой строке.

Правильное скобочное выражение определяется следующим образом:

- 1) строка, не содержащая скобок, является правильным скобочным выражением;
- 2) если строка A – правильное скобочное выражение, то (A) , $[A]$ и $\{A\}$ – правильные скобочные выражения;
- 3) если строки A и B – правильные скобочные выражения, то AB (конкатенация строк A и B) – правильное скобочное выражение.

Например, в выражениях $[(A+B)*(A-B)+B]$ и $[(A+B)]$ нарушается второй пункт определения правильного скобочного выражения. В первом выражении правильное скобочное выражение $“(A+B)*(A-B)+B”$ некорректно заключено в $[]$ (нет закрывающей скобки). Во втором выражении внутри $[]$ содержится неправильное скобочное выражение $“(A+B)”$ (нет закрывающей круглой скобки).

Для решения задачи применяется следующий алгоритм синтаксического анализа, основанный на использовании стека. Выражение анализируется посимвольно слева направо, пока не

будет обнаружен признак его конца или не нарушится пункт 2 определения. В анализируемом выражении рассматриваются только скобки. При нахождении открывающей скобки заносится в стек информация о ее типе. При обнаружении закрывающей скобки любого типа анализируется содержание стека. Если стек пуст, то считается, что скобки в выражении расставлены неправильно. Если стек не пуст, то из него извлекается последняя открывающая скобка и проверяется соответствие ее типа типу обнаруженной закрывающей скобки. Если в рассматриваемой паре скобок открывающая и закрывающая скобки имеют различный тип, выражение считается некорректным, в противном случае продолжается поиск следующей скобки во входном выражении. При достижении признака конца строки вновь анализируется состояние стека. Пустой стек соответствует правильной расстановке скобок. Наличие скобок в стеке соответствует ошибке расстановки скобок.

Синтаксический анализ расстановки скобок иллюстрируется на примере следующего алгебраического выражения:

$$\begin{array}{cccccccccccc} \{ & X & + & [& Y & - & (& A & + & B &) &] & * & C & - & [& (& D & + & E &) &] & \} \\ \wedge & & & \wedge & & & \wedge & & & \wedge & \wedge & & \wedge & & \wedge & \wedge & \wedge & \wedge & \wedge & \wedge & \wedge & \wedge \\ 1 & & & 2 & & & 3 & & & 4 & 5 & & 6 & 7 & & 8 & 9 & 10 \end{array}$$

Под строкой алгебраического выражения указаны номера состояний стека при разборе скобок. Состояния стека приведены на следующей диаграмме состояний (рис. 4.2.1).

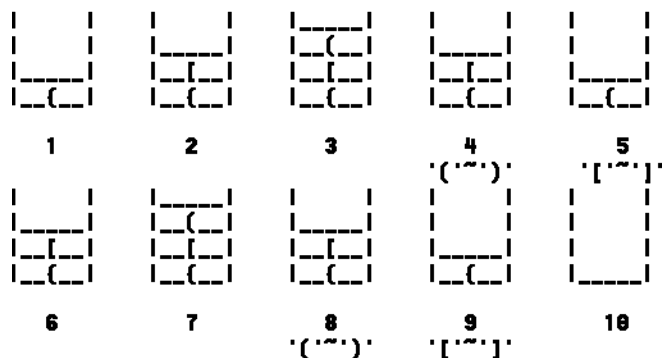


Рис. 4.2.1. Диаграммы состояний стека

7. Дана символьная строка, которая содержит правильное скобочное выражение. Для каждой пары скобок (открывающей и соответствующей ей закрывающей) распечатать номера их позиций в строке, упорядочив пары:

-
- а) по возрастанию номеров открывающих скобок;
 - б) по возрастанию номеров закрывающих скобок.

8. Дана символьная строка, содержащая правильно записанное математическое выражение следующего вида:

$$\langle \text{формула} \rangle ::= \langle \text{цифра} \rangle | M(\langle \text{формула} \rangle, \langle \text{формула} \rangle) \\ | m(\langle \text{формула} \rangle, \langle \text{формула} \rangle)$$

M – операция вычисления максимума из двух выражений, m – операция вычисления минимума из двух выражений. Написать функцию вычисления значения этого выражения. Здесь следует использовать два стека – для обработки последовательности выполняемых операций и для обработки операндов этих операций.

9. Дана символьная строка, содержащая правильно записанное логическое выражение следующего вида:

$$\langle \text{формула} \rangle ::= T \mid F \mid \text{And}(\langle \text{формула} \rangle, \langle \text{формула} \rangle) \mid \\ \text{Or}(\langle \text{формула} \rangle, \langle \text{формула} \rangle) \mid \text{Not}(\langle \text{формула} \rangle)$$

And – операция логического И, Or – операция логического ИЛИ, Not – операция логического НЕ.

Написать функцию вычисления этого выражения (функция должна возвращать true, если значение выражения равно T, false – в противном случае).

10. Дана символьная строка, содержащая правильно записанное арифметическое выражение. Написать функцию перевода выражения в постфиксную форму. Постфиксной формой выражения называется такая запись, в которой знак операции следует за операндами. Такая запись не содержит скобок. Например, выражение $a * b$ в постфиксной форме имеет вид $ab *$, $a * b + c$ имеет вид $ab * c +$, $a * (b + c)$ имеет вид $abc + *$.

Для решения задачи можно использовать следующий алгоритм. Рассматриваются поочередно все символы строки. В стек записывается открывающая скобка, и выражение далее анализируется посимвольно слева направо. Если встречается операнд (число или переменная), то он сразу помещается в строку-результат. Если встречается открывающая скобка, то она заносится в стек, а если встречается закрывающая скобка, то из стека извлекаются находящиеся там знаки операций до ближайшей открывающей

скобки, которая также удаляется из стека. Все эти знаки в порядке их извлечения помещаются в строку-результат. Когда же встречается знак операции, то из стека извлекаются знаки операций, приоритет которых больше или равен приоритету данной операции, и они помещаются в строку-результат, после чего рассматриваемый знак переносится в стек. Когда выражение заканчивается, выполняются те же действия, что и при встрече закрывающей скобки.

Рассмотрим алгоритм на примере простейшего выражения:

$$a + (b - c) * d.$$

Таблица 3
Перевод выражения в постфиксную форму

Символ	Действие	Состояние выходной строки	Состояние стека
a	'a' – переменная. Помещаем ее в строку-результат	a	Пуст
+	'+' – знак операции. Помещаем его в стек (поскольку стек пуст, приоритеты можно не проверять)	a	+
(	'(' – открывающая скобка. Помещаем в стек	a	(+
b	'b' – переменная. Помещаем ее в выходную строку	a b	(+
-	'-' – знак операции, который имеет приоритет 2. Проверяем стек: на вершине находится символ '(', приоритет которого равен 1. Следовательно, мы должны просто поместить текущий символ '-' в стек	a b	- (+
c	'c' – переменная. Помещаем ее в выходную строку	a b c	- (+
)	')' – закрывающая скобка. Извлекаем из стека в выходную строку все символы, пока не встретим открывающую скобку. Затем уничтожаем обе скобки	a b c -	+

Символ	Действие	Состояние выходной строки	Состояние стека
*	'*' – знак операции, который имеет приоритет 3. Проверяем стек: на вершине находится символ '+', приоритет которого равен 2, т. е. меньший, чем приоритет '*'. Следовательно, мы должны поместить текущий символ '*' в стек	a b c -	* +
d	d' – переменная. Помещаем ее в выходную строку	a b c - d	* +

Теперь вся входная строка разобрана, но в стеке еще остаются знаки операций, которые мы должны просто извлечь в строку-результат. Итак, мы получили конечный результат: $abc - d * +$.

- Дана символьная строка, содержащая постфиксную форму правильно записанного арифметического выражения, операндами которого являются цифры. Написать функцию вычисления этого выражения.

Можно использовать следующий алгоритм. Выражение просматривается слева направо. Если встречается цифра, то она заносится в стек. Если встречается знак операции, то из стека извлекаются два операнда, над ними выполняется операция и ее результат записывается в стек. Когда выражение заканчивается, в стеке остается одно число – значение выражения.

- Решить задачу 7 при условии, что операндами выражения являются натуральные числа.

4.3. Двусвязные списки

Двусвязные списки, в отличие от односвязных, характеризуются тем, что дают возможность просмотра элементов списка в обоих направлениях. Это достигается посредством хранения в каждом узле двух адресов (ссылок): следующего и предыдущего элементов. Первый элемент имеет пустую ссылку на предыдущий элемент, а последний элемент имеет пустую ссылку на следующий элемент (рис. 4.3.1).

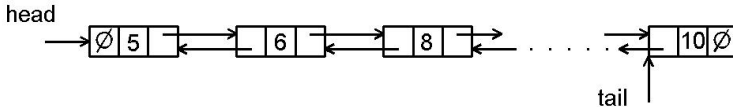


Рис. 4.3.1. Пример двусвязного списка

Для возможности доступа к элементам с обоих концов списка стоит хранить две ссылки – на первый (*head*) и на последний (*tail*) элементы списка.

В листинге 4.3.1 приведено описание фрагмента описания структуры хранения одного элемента двусвязного списка и самого объекта описания списка на языке C++. В листинге 4.3.2 аналогичное описание дано для языка программирования C#.

```
// элемент списка
struct Element
{
    int info;           // информационное поле
    Element* prev;      // указатель на предыдущий элемент
    Element* next;      // указатель на следующий элемент
};

// структура для двусвязного списка
struct TwoWayList
{
    Element* head;      // адрес первого элемента списка
    Element* tail;      // адрес последнего элемента списка
};
```

Листинг 4.3.1. Структуры представления двусвязного списка на языке программирования C++

```

// элемент списка
class Element
{
    public int info;           // информационное поле
    public Element prev;      // ссылка на предыдущий элемент
    public Element next;      // ссылка на следующий элемент
}

// структура для двусвязного списка
class TwoWayList
{
    // ссылка на первый элемент списка
    public Element head;
    // ссылка на последний элемент списка
    public Element tail;
}

```

Листинг 4.3.2. Структуры представления двусвязного списка на языке программирования C#

Задача 1. Дан текстовый файл, содержащий целые числа. Написать функцию построения двусвязного списка из этих чисел.

Приведем в листинге 4.3.3 программный код функции для решения этой задачи на языке программирования C++. Параметрами функции являются имя файла (*file_name*) и ссылка на структуру для хранения начала и конца списка (переменная *L*). Код возврата функции будет равен 1, если список создан успешно, 0 – если создан пустой список, –1 – если файл не был открыт.

Общая схема алгоритма заключается в следующем. Отдельно обрабатывается создание первого узла – выделение памяти и заполнение данными элемента *L.head*. Если файл пуст, то *L.head* и *L.tail* принимают значения *NULL*.

Для добавления каждого из последующих узлов используется переменная *current*, которая будет хранить адрес последнего созданного элемента списка. Ее начальное значение – адрес первого элемента. Далее для каждого считанного из файла числа создается новый узел (адрес запоминается в переменной *help*). Заполняются поля этого узла: *info* – число из файла, *prev* содержит адрес *current*. Элемент добавляется в конец списка. В завершение устанавливается признак конца списка (*current -> next = NULL*) и адрес последнего элемента сохраняется в *L.tail*.

```

// определение функции построения двусвязного списка
// из чисел, хранящихся в текстовом файле
int CreateTwoWayList(char* file_name, TwoWayList& L)
{
    Node *current, *help;
    int k;
    // открытие файла
    ifstream in(file_name);
    if(!in.is_open())
    {
        // Ошибка открытия файла
        return -1;
    }
    // считывание первого числа из файла и
    // создание первого элемента списка
    if(in >> k)
    {
        L.head = new Element;
        L.head -> info = k;
        L.head -> prev = NULL;
    }
    else
    {
        // Файл пуст, следовательно, список тоже пуст
        L.head = L.tail = NULL;
        return 0;
    }
    current = L.head;
    // считывание еще непрочитанных чисел из файла и
    // добавление их в конец списка
    while(in >> k)
    {
        help = new Element;
        help -> info = k;
        current -> next = help;
        help -> prev = current;
        current = current -> next;
    }
    // установка признака конца двусвязного списка
    L.tail = current;
    current -> next = NULL;
    in.close();
    return 1;
}

```

Листинг 4.3.3. Функция для построения двусвязного списка на основании данных из текстового файла на языке C++

Задача 2. Дан двусвязный список, содержащий целые числа. Определить, является ли он симметричным.

При решении задачи используются две переменные-указатели b и e для просмотра элементов с начала и с конца списка соответственно. Для этого их начальные значения устанавливаются на $L.head$ и $L.tail$. Смещая их одновременно (b на следующий элемент, e – на предыдущий), проверяем значения в этих узлах на равенство. Смещение заканчивается в одном из двух случаев:

- 1) встретилась неравная пара значений;
- 2) были просмотрены все элементы списка. При нечетном количестве узлов адреса b и e совпадут (рис. 4.3.2).

При четном количестве узлов адрес b будет указывать на первый элемент второй половины списка, а e – на последний элемент первой половины. В этом случае в качестве условия выхода из цикла можно использовать одно из следующих равенств: «предыдущим элементом для b является e » или «следующим элементом для e является b » (рис. 4.3.3).

Если встретилась неравная пара значений, то смещение указателей прекращается и функция возвращает значение *false*. Если просмотрены все элементы, то список является симметричным и функция возвращает значение *true*. По определению пустой список является симметричным.

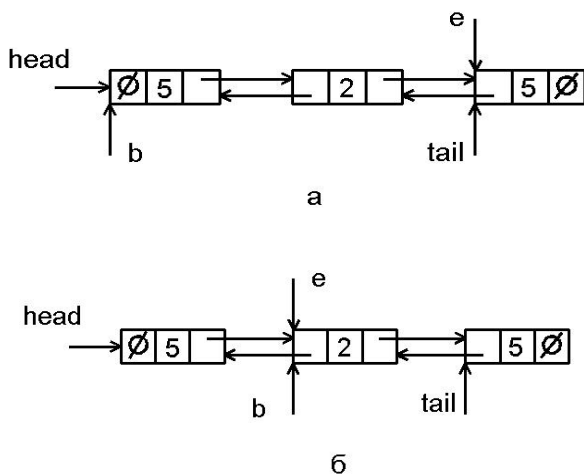


Рис. 4.3.2. Условие окончания просмотра элементов списка при их нечетном количестве

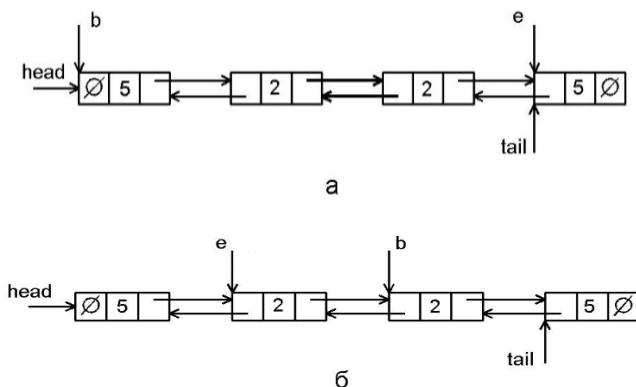


Рис. 4.3.3. Условие окончания просмотра элементов списка при их четном количестве

В листинге 4.3.4 приведем программный код функции проверки двусвязного списка на симметричность, написанный на языке программирования C#.

```
// определение функции проверки симметричности списка
bool Symmetry(TwoWayList L)
{
    Element b = L.head,    // ссылка на начало списка
    e = L.tail;            // ссылка на конец списка
    if(b == NULL) return true; // пустой список симметричен
    // просмотр элементов с начала и с конца списка
    // до тех пор, пока значения соответствующих элементов
    // равны и не просмотрены все элементы списка
    while(b.info == e.info && b != e && b.prev != e)
    {
        b = b.next;
        e = e.prev;
    }
    // определение условия, по которому произошел
    // выход из цикла
    if(b.info != e.info)
        // найдена несимметричная пара элементов
        return false;
    // список симметричен
    return true;
}
```

Листинг 4.3.4. Функция проверки симметричности двусвязного списка на языке программирования C#

Задача 3. Дан двусвязный список, содержащий целые числа. Отсортировать его методом вставок.

Метод вставок применительно к массиву можно описать так. Пусть дан массив $a[1] \dots a[n]$. Каждый элемент $a[i]$ (i изменяется от 2 до n) вставляется на подходящее место в уже упорядоченной совокупности $a[1] \dots a[i-1]$. Это место определяется при обратном проходе по массиву от $(i-1)$ -го элемента до элемента с меньшим значением, чем $a[i]$, или, если такого элемента не нашлось, до начала массива.

Если список пуст или состоит из одного элемента, сортировать его не нужно. Для просмотра всех элементов списка, начиная со второго, создается переменная *current*. Если *current* стоит на своем месте, т. е. предыдущий элемент имеет меньшее значение информационного поля, то *current* либо сдвигается на следующий узел, либо происходит выход из цикла, когда достигнут конец списка.

Если требуется изменить позицию элемента *current*, то выполняются следующие действия:

- 1) удалить *current* из списка, присоединив «хвост» списка к предыдущему элементу;
- 2) определить его место в уже отсортированной части списка;
- 3) вставить *current* на найденное место.

При удалении сохраняем адреса предыдущего и следующего элементов в переменных *help1* и *help2* соответственно. Далее соединяем эти два узла. Если следующего узла не существует, т. е. *current* был последним узлом, *L.tail* устанавливается на *help1*.

Для определения места вставки элемента, двигаемся от узла *help1* к началу списка до тех пор, пока не встретим узел со значением, меньшим «вставляемого», или пока не достигнем начала списка. Если *current* нужно вставить на первое место, то связываем элементы *current* и *help1* и меняем начало списка ($L.head = current$). Если же позиция вставки *current* не начальная, то сохраняем адрес элемента, перед которым нужно вставить *current*, в переменную *help3*, и организуем связи с учетом вставки *current*. Далее повторяем те же действия для следующего элемента, адрес которого был сохранен в переменной *help2*.

Программный код функции сортировки двусвязного списка методом вставок приведем далее в листинге 4.3.5. Программный код представим на языке программирования C++.

```

// определение функции сортировки списка методом вставок
void SortList(TwoWayList& L)
{
    Element* current, *help1, *help2, *help3;
    // если список пуст или состоит из одного элемента,
    // сортировать его не нужно
    if(L.head == L.tail)
        return;
    // устанавливаем в качестве текущего второй элемент списка
    current = L.head -> next;
    bool f = true; // булева переменная, которая принимает
                  // значение true, если конец списка
                  // не достигнут, а иначе - false

    while(f)
    {
        if(current -> prev -> info > current -> info)
        {
            // значение элемента current меньше,
            // чем значение последнего элемента в уже
            // упорядоченной части списка
            // извлекаем элемент current из списка
            help2 = current -> next;
            help1 = current -> prev;
            help1 -> next = current -> next;
            // если элемент current был
            // последним в списке
            if(current == L.tail)
            {
                // переносим конец списка
                // на предыдущий элемент
                L.tail = current -> prev;
                // устанавливаем признак достижения
                // конца списка
                f = false;
            }
        }
        else
        {
            help2 -> prev = help1;
            // двигаясь в обратном порядке от предыдущего
            // положения current, определяем место
            // вставки current
            while(help1 != L.head && help1 -> info > current -> info)
                help1 = help1 -> prev;
            // вставка элемента current
            if(help1 -> info > current -> info)
            {
                // в начало списка
                help1 -> prev = current;
                current -> next = help1;
                L.head = current;
            }
        }
    }
}

```

Листинг 4.3.5. Функция сортировки двусвязного списка методом вставок на языке программирования C++ (Начало)

```

        else
        {
            // в середину упорядоченной части списка
            help3 = help1 -> next;
            help3 -> prev = current;
            current -> next = help3;
            current -> prev = help1;
            help1 -> next = current;
        }
        // продолжаем просмотр неупорядоченной части списка
        current = help2;
    }
    else
    {
        // если список просмотрен и current стоит
        // на своем месте, устанавливаем признак
        // достижения конца списка
        if(current == L.tail)
            f = false;
        // переход к следующему элементу
        current = current -> next;
    }
}
}

```

Листинг 4.3.5. Функция сортировки двусвязного списка методом вставок на языке программирования C++ (Окончание)

Домашнее задание

1. Дан двусвязный список. Написать функцию печати списка в обратном порядке.
2. Дан двусвязный список. Написать функцию добавления элемента в заданную позицию.
3. Дан двусвязный список. Написать функцию удаления элемента из заданной позиции.
4. Дан двусвязный список. Написать функцию добавления в начало списка нового элемента, значение которого равно количеству вхождений минимального элемента в список.
5. Дан двусвязный список. Написать функцию добавления двух новых элементов – с минимальным значением в начало списка и с максимальным значением в конец списка.

6. Дан двусвязный список. Написать функцию удаления из него всех элементов с минимальным значением.

7. Дан двусвязный список. Написать функцию печати всех симметричных подпоследовательностей элементов списка.

8. Дан двусвязный список из n элементов (n – четное). Вычислить выражение

$$\max\{a_1 + a_n; a_2 + a_{n-1}; \dots a_{n/2} + a_{n/2+1}\}.$$

9. Дан двусвязный список из n элементов (n – четное). Вычислить выражение:

$$\max\{\min\{a_1, a_n\}, \min\{a_2, a_{n-1}\} \dots \min\{a_{n/2}, a_{n/2+1}\}\}.$$

10. Дан двусвязный список. Написать функцию, которая:

- 1) формирует новый список, содержащий элементы исходного списка, располагая их в обратном порядке;
- 2) располагает элементы списка в обратном порядке, не создавая нового списка.

11. Написать функцию слияния двух отсортированных по неубыванию двусвязанных списков.

12. Даны два двусвязных списка. Написать функцию формирования нового двусвязного списка, состоящего из элементов, входящих одновременно в оба исходных списка.

13. Даны два двусвязных списка. Написать функцию формирования нового двусвязного списка из тех элементов первого списка, которые не содержатся во втором.

14. Дан двусвязный список. Написать функцию нахождения суммы элементов, расположенных между минимальным и максимальным элементами списка.

Глава 5. Нелинейные структуры данных

5.1. Деревья

Еще одним из наиболее важных видов динамических структур данных являются деревья. Они задают иерархический порядок следования элементов. Существует большое количество видов деревьев – свободные, ориентированные, упорядоченные, бинарные деревья, В-деревья. В данном разделе мы будем рассматривать только бинарные деревья. Формально бинарное дерево можно представить в виде набора узлов (вершин), один из которых называется корнем дерева, а остальные узлы образуют два непересекающихся подмножества, каждое из которых также образует дерево (рис. 5.1.1). Узлы соединены между собой дугами. От каждого узла бинарного дерева могут исходить две дуги к левому и правому узлам. При этом начальный узел называется родительским, два других узла – его потомками.

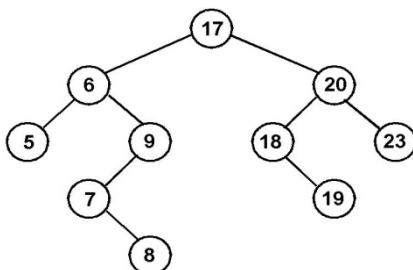


Рис. 5.1.1. Вид бинарного дерева

На рис. 5.1.1 узел «17» является корнем дерева – узлом, у которого нет узла-родителя. Остальные узлы образуют два непересекающихся подмножества, называемых левым и правым поддеревьями. Узлы «6» и «20» являются потомками узла «17».

В свою очередь, каждое поддерево также является деревом, например корнем левого поддерева является узел «6», который имеет двух потомков (рис. 5.1.2).

Узлы, расположенные на одинаковом расстоянии от корня дерева, образуют один уровень дерева. Листом дерева называют узел, не имеющий потомков. Глубина дерева – это номер максимального уровня дерева или максимальная длина пути от корня до какого-либо листа.

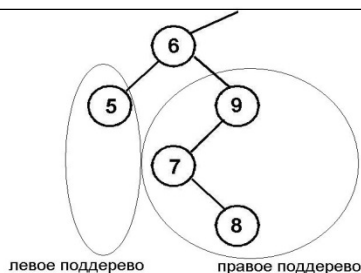


Рис. 5.1.2. Левое и правое поддерева

Например, на рис. 5.1.3 узел «17» образует нулевой уровень, узлы «6», «20» – первый уровень и т. д. Узлы «5», «8», «19», «23» являются листовыми, глубина дерева равна 4 (соответствует пути из узла «17» в листовую узел «8»).

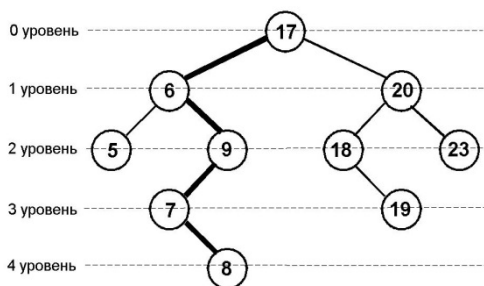


Рис. 5.1.3. Уровни дерева. Глубина дерева

Вид структуры хранения узла бинарного дерева такой (язык C++):

```
// структура узла бинарного дерева
struct Tree
{
    int info;                // информационное поле
    Tree* left;              // указатель на левое поддерево
    Tree* right;              // указатель на правое поддерево
};
```

Подвидом бинарных деревьев является дерево сортировки. Это такое дерево, в котором слева от любой вершины находятся вершины с элементами, меньшими значения информационного поля данной вершины, а справа – с большими или равными значениями. Именно пример дерева сортировки представлен на рис. 5.1.1.

Задача 1. Дано дерево сортировки. Написать функцию добавления в дерево нового элемента.

При добавлении нового узла в дерево используются следующие правила:

1) если дерево было пустым, то значение помещается в корень дерева;

2) в противном случае просматриваются узлы дерева, начиная с корня, и осуществляется переход к левому поддереву, если новый элемент имеет меньшее значение, чем значение в текущей вершине. В противном случае – переход к правому поддереву.

Как только достигается вершина, от которой нет возможности передвигаться в нужном направлении, т. е. указатель на соответствующее поддерево равен *NULL*, создаем новый узел и присоединяем его к этой вершине в соответствии с определением дерева сортировки.

Рассмотрим вставку новых элементов в дерево сортировки, представленного на рис. 5.1.1. Например, сначала добавим элемент «12», затем «10».

Просматриваем узлы дерева, начиная с корня. Поскольку 17 больше 12, то переходим к левому потомку – узлу «6». 6 меньше 12, следовательно, переходим на правый потомок узла «6» – узел «9». 9 меньше 12, поэтому следует перейти к правому потомку, которого не существует. Таким образом, определилось место вставки нового элемента – справа от узла «9» (рис. 5.1.4).

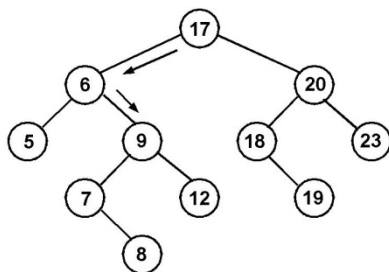


Рис. 5.1.4. Вставка в дерево значения 12

При вставке узла со значением 10 аналогично переходим по узлам «17», «6», «9», «12», и поскольку 10 меньше 12, то связываем узел «10» с узлом «12» слева.

Для решения задачи рассмотрим два варианта функции – рекурсивный и нерекурсивный. Разберем сначала рекурсивный вариант (листинг 5.1.1). Поскольку каждое поддерево по определению также является деревом, то каждый узел дерева можно обрабатывать как корень.

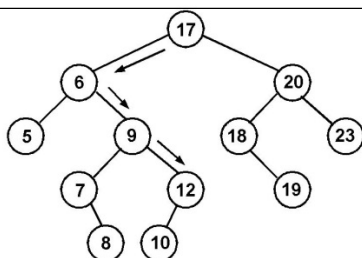


Рис. 5.1.5. Вставка в дерево значения 10

Параметрами функции будут ссылка на адрес корневого элемента поддерева (переменная *root*) и добавляемое значение (переменная *key*).

Если дерево пусто, формируем корень дерева. Для этого адрес вновь созданного элемента сохраняется в переменной *root*, заполняется информационное поле и ссылки на левое и правое поддерева обнуляются. На этом функция завершает свою работу.

Если дерево непусто, вызываем эту же функцию для левого (со значениями аргументов *root -> left* и *key*) или правого поддеревья (*root -> right* и *key*) в зависимости от добавляемого значения.

```

// определение рекурсивной функции
// добавления в дерево нового элемента
void AddTree(Tree*& root, int key)
{
    // если дерево пусто, формируем корень дерева
    if(root == NULL)
    {
        Tree* help = new Tree;
        help -> info = key;
        help -> left = NULL; help -> right = NULL;
        root = help;
        return;
    }
    // определяем, в левое или в правое поддерево узла root
    // добавляется новый элемент
    if(root -> info > key)
        // рекурсивный вызов функции
        // добавления в левое поддерево
        AddTree(root -> left, key);
    else
        // рекурсивный вызов функции
        // добавления в правое поддерево
        AddTree(root -> right, key);
}
  
```

Листинг 5.1.1. Рекурсивная функция добавления нового элемента в дерево сортировки на языке программирования C++

При реализации рекурсивной функции на языке программирования С# отличия заключаются, как и при работе со списками, в организации обращения к данным по ссылке (в использовании класса вместо структуры для хранения информации, в использовании операции '.' для объекта класса и передачи параметра по ссылке с помощью ключевого слова *ref*). Для сравнения приведем рекурсивную функцию добавления элемента в дерево, написанную на языке программирования С#, в листинге 5.1.2.

```
// класс для хранения узла бинарного дерева
class Tree
{
    public int info;           // информационное поле
    public Tree left;         // ссылка на левое поддерево
    public Tree right;        // ссылка на правое поддерево
}

class Program
{
    // определение рекурсивной функции
    // добавления в дерево нового элемента
    static void AddTree(ref Tree root, int key)
    {
        // если дерево пусто, формируем корень дерева
        if(root == null)
        {
            Tree help = new Tree();
            help.info = key;
            help.left = null;
            help.right = null;
            root = help;
            return;
        }
        // определяем, в левое или в правое поддерево узла root
        // добавляется новый элемент
        if(root.info > key)
            // рекурсивный вызов функции
            // добавления в левое поддерево
            AddTree(ref root.left, key);
        else
            // рекурсивный вызов функции
            // добавления в правое поддерево
            AddTree(ref root.right, key);
    }
    . . .
}
```

Листинг 5.1.2. Рекурсивная функция добавления нового элемента в дерево сортировки на языке программирования С#

Теперь рассмотрим нерекурсивный вариант решения задачи (для языка программирования C++) (листинг 5.1.3).

В случае когда дерево пусто, поступаем так же, как и в рекурсивном варианте. Иначе перемещаемся по узлам дерева в соответствии с правилами, используя вспомогательную переменную *help*, до тех пор, пока не определим место вставляемого элемента. Если значение информационного поля *help -> info* больше *key*, то добавление нового узла осуществляется при условии, что *help -> left = NULL*. Если же значение *help -> info* меньше или равно *key*, то добавление нового узла осуществляется в правое поддерево, когда *help -> right = NULL*. Новый узел связывается с узлом *help* с помощью надлежащего указателя (*help -> left* или *help -> right*).

```
// определение нерекурсивной функции добавления
// в дерево нового элемента
void AddTree(Tree*& root, int key)
{
    Tree *help, *newNode;
    // если дерево пусто, формируем корень дерева
    if(root == NULL)
    {
        root = new Tree;
        root -> info = key;
        root -> left = NULL;
        root -> right = NULL;
        return;
    }
    help = root;
    // двигаемся по узлам дерева, начиная с корня,
    // пока не найдем место вставки нового элемента
    while (true)
    {
        if(help -> info > key)
            // в текущем узле help значение больше нового
            if(help -> left == NULL)
            {
                // если у узла help нет левого поддерева, новый узел
                // «подцепляется» к узлу help слева
                newNode = new Tree;
                newNode -> info = key;
                newNode -> left = NULL;
                newNode -> right = NULL;
                help -> left = newNode;
                return;
            }
        else
            // значение info не больше key, идем вправо
            help = help -> right;
    }
}
```

Листинг 5.1.3. Нерекурсивная функция добавления нового элемента в дерево сортировки на языке программирования C++ (Начало)

```

        else
            // если существует левое поддерево,
            // перемещаемся к левому потомку help
            help=help->left;
    else
        // в текущем узле help значение
        // меньше или равно новому
        if(help -> right == NULL)
        {
            // если у узла help нет правого поддерева, новый узел
            // «подцепляется» к help справа
            newNode = new Tree;
            newNode -> info = key;
            newNode -> left = NULL;
            newNode -> right = NULL;
            help -> right = newNode;
            return;
        }
        else
            // если существует правое поддерево,
            // перемещаемся к правому потомку help
            help = help -> right;
    }
}

```

Листинг 5.1.3. Нерекурсивная функция добавления нового элемента в дерево сортировки на языке программирования C++ (Окончание)

Заметим, что функцию добавления элемента в дерево удобно использовать и для построения дерева сортировки. В листинге 5.1.4 приведем пример функции создания дерева сортировки на основе данных из текстового файла.

```

// определение функции создания дерева на основе
//данных текстового файла
int CreateTree(char* FileName, Tree*& root)
{
    int k;        // переменная для ввода данных из файла
    // открытие файла
    ifstream in(FileName);
    if(!in.is_open())
    {
        //Ошибка открытия файла, возвращаем код ошибки
        return -1;
    }
}

```

Листинг 5.1.4. Функция создания дерева сортировки на основе данных текстового файла на языке C++ (Начало)

```

// изначально дерево пусто
root = NULL;
// в цикле считываем числа из файла в переменную k
while(in >> k)
    // добавляем элемент со значением k в дерево
    AddTree(root, k);
in.close();
return 1;
}

```

Листинг 5.1.4. Функция создания дерева сортировки на основе данных текстового файла на языке C++ (Окончание)

Задача 2. Дано дерево сортировки. Написать функцию печати элементов дерева в порядке неубывания.

Согласно правилам построения дерева сортировки, сначала следует распечатать элементы левого поддерева, потом элемент самой вершины и, в завершении, элементы правого поддерева.

Очень просто выглядит решение этой задачи с использованием рекурсивной функции, поскольку само определение дерева является рекурсивным (каждое поддерево также является деревом). Приведем программный код этой функции на языке C# в листинге 5.1.5.

```

// определение рекурсивной функции распечатки
// значений из узлов дерева в порядке неубывания
static void PrintTree(Tree root)
{
    if (root != null)
    {
        // печатаем значения из левого поддерева узла root
        PrintTree(root.left);
        // печатаем информационное поле узла root
        Console.Write("{0} ", root.info);
        // печатаем значения из правого
        // поддерева узла root
        PrintTree(root.right);
    }
}

```

Листинг 5.1.5. Рекурсивная функция печати данных дерева сортировки в порядке неубывания (обход узлов дерева) на языке C#

При написании нерекурсивного варианта функции печати требуется использовать стек. Для возвращения к вершинам верхнего уровня в нем нужно хранить адреса (ссылки) уже пройденных вершин.

В таблице 4 наглядно поясним процесс печати элементов дерева, изображенного на рис. 5.1.1, в порядке неубывания значений элементов.

Таблица 4
Печать элементов дерева

Содержимое по адресу temp	Состояние стека	Выходная строка	Комментарии
17	Пуст		
6	17		Начинаем движение до крайнего левого листового узла
5	17, 6		
NULL	17, 6, 5		Левого потомка узла «5» не существует
5	17, 6	5	Извлекаем из стека элемент 5, выводим на печать. Переходим к правому потомку узла «5»
NULL	17, 6		Правого потомка узла «5» тоже не существует. Поэтому переходим к родительскому узлу «6», извлекая его из стека
6	17	5, 6	Переходим к правому потомку узла «6»
9	17		Начинаем движение влево
7	17, 9		
NULL	17, 9, 7		Движение влево невозможно, извлекаем родительский узел из стека
7	17, 9	5, 6, 7	Переходим к правому потомку узла «7»
8	17, 9		Начинаем движение влево. Левого потомка узла «8» не существует
NULL	17, 9, 8		

Содержимое по адресу temp	Состояние стека	Выходная строка	Комментарии
8	17, 9	5, 6, 7, 8	Переходим к правому потомку узла «8»
NULL	17, 9		Переходим к родительскому узлу «9», извлекая его из стека
..... и т. д.			
23	пуст		Переходим к правому потомку узла «20»
NULL	23		Начинаем движение влево. Левого потомка узла «23» не существует
23	пуст	5, 6, 7, 8, 9, 17, 18, 19, 20, 23	Извлекаем из стека элемент 23, выводим на печать. Переходим к правому потомку узла «23»
NULL	пуст		Все узлы распечатаны

Приведем нерекурсивную версию функции организации печати элементов дерева на языке C++. Для этого нам понадобится использовать стек, который будет хранить в качестве информационного поля адрес элемента дерева. Подробно алгоритмы работы со стеком мы разбирали в разделе 4.2 данного пособия. Поэтому без подробных комментариев приведем только основной программный код, необходимый для реализации такого стека, – структуры для хранения и функции добавления и извлечения элемента из стека (листинг 5.1.6).

```

// структура для хранения одного элемента стека
struct Node
{
    Tree* node;    //адрес узла дерева - информационное поле
    Node* next;    // указатель на следующий элемент списка-стека
};

..// структура для хранения адреса вершины стека
struct Stack
{
    Node* head;    // адрес вершины стека
};

// определение функции добавления в стек нового элемента
void PushStack(Stack& s, Tree* ptr)
{
    Node* p = new Node;
    p -> next = s.head;
    p -> node = ptr;
    s.head = p;
}

// определение функции извлечения элемента из стека
int PopStack(Stack& s, Tree*& ptr)
{
    if(s.head == NULL)
        return -1;
    ptr = s.head -> node;
    Node* p = s.head;
    s.head = s.head -> next;
    delete p;
    return 1;
}

```

Листинг 5.1.6. Реализация стека для хранения адресов узлов дерева

Алгоритм нерекурсивной функции печати элементов списка выглядит так. Создается переменная *temp*, с помощью которой осуществляется просмотр вершин дерева. Адрес текущей вершины помещается в стек, затем осуществляется переход к левому поддереву (*temp = temp -> left*), т. е. текущей вершиной становится левый потомок. И так продолжается до тех пор, пока не достигнем крайнего левого узла и обработаем его (*temp = NULL*). Далее извлекаем узел из стека, печатаем его значение и переходим к его правому потомку. Если правого потомка у узла нет, из стека извлекается очередная вершина. Если же стек пуст, то это означает, что дерево было просмотрено полностью. В противном случае в стеке остались непросмотренные вершины, поэтому снова осуществляется попытка прохода по левой ветке, но уже для другой текущей вершины.

Программный код нерекурсивной функции печати элементов списка приведем в листинге 5.1.7.

```
// определение нерекурсивной функции распечатки элементов
// дерева в порядке неубывания
void PrintTree(Tree* root)
{
    if(root == NULL)
        return;
    // создание вспомогательного стека
    Stack s; s.head = NULL;
    // текущий узел - корень дерева
    Tree* temp = root;
    // обход будет закончен, если все узлы окажутся
    // просмотренными, т. е. temp будет указывать
    // на несуществующий узел, и стек станет пустым
    while(s.head != NULL || temp != NULL)
    {
        // перемещаемся по крайней левой ветке,
        // исходящей из узла temp
        while(temp != NULL)
        {
            // помещаем каждый узел ветки в стек
            PushStack(s, temp);
            temp = temp -> left;
        }
        // «прошли» последний узел левой ветки,
        // осуществляем возврат, т. е. извлекаем из стека
        // предыдущую вершину и сохраняем ее в temp
        PopStack(s, temp);
        // печатаем значение информационного поля
        cout << (temp -> info) << " ";
        // переходим на правого потомка текущего узла temp
        temp = temp -> right;
    }
    cout << endl;
}
```

Листинг 5.1.7. Нерекурсивный вариант функции печати данных дерева сортировки в порядке неубывания (обход узлов дерева) на языке C++

Задача 3. Дана символьная строка, содержащая правильное арифметическое выражение. Операндами выражения являются числа с плавающей точкой, допустимые операции – сложение, вычитание, умножение и деление. Требуется построить дерево разбора этого выражения.

Деревом разбора выражения называется бинарное дерево, построенное по следующим правилам:

1) формула из одного числа представляется деревом из одной вершины, операндом которого является это число;

2) если выражение имеет операции, то листовые узлы (не имеющие потомков) будут хранить операнды, а остальные узлы – операции, причем левое и правое поддеревья узла-операции будут представлять собой выражение его операндов.

Так, дерево разбора выражения $(5 + \frac{16}{2}) * (3 - 7 * 2)$ представлено на рис. 5.1.6.

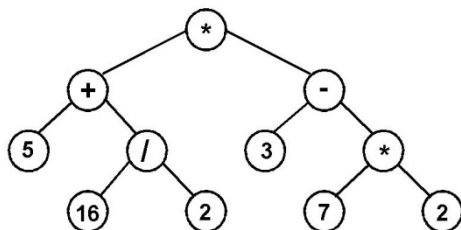


Рис. 5.1.6. Пример дерева разбора выражения

В целях упрощения алгоритма мы рассмотрим случай, когда построенное выражение не использует скобок для изменения приоритета выполняемых операций. Тем не менее основную идею и схему алгоритма можно будет применить и для разбора более сложных арифметических выражений. В листинге 5.1.8 представим программный код функции построения дерева разбора выражения на языке программирования C#.

Узел такого дерева хранит два информационных поля – символ операции и значение операнда. Условимся, что если в узле дерева будет храниться операнд, то поле операции будет содержать символ '@'.

При построении дерева надо учитывать приоритет операций. Поскольку умножение и деление выполняются в первую очередь, то в дереве они будут располагаться на более низких уровнях по отношению к сложению и вычитанию (если таковые имеются в формуле).

Параметрами функции являются символьная строка (*str*), содержащая выражение, признак обрабатываемых операций (*level*) и длина строки (*len*). Особо поясним значение параметра *level*. Если он равен 0, это означает, что строка содержит все возможные операции и операнды, если *level* равно 1, тогда в рассматриваемой подстроке могут содержаться только операции умножения и деления и операнды, при *level*, равном 2, – только операнды.

Возвращаемое значение функции – адрес созданного дерева.

Выражение начинаем разбирать с конца.

Рассмотрим три возможных случая обработки строки, соответствующих трем значениям параметра *level*.

1. Когда значение аргумента *level* равно 0, осуществляется поиск первого знака '+' или '-'. Знак операции помещается в вершину дерева, и рекурсивно вызывается функция построения левого поддерева из подформулы, расположенной в строке до символа найденной операции, а затем правого поддерева, начиная со следующего символа. При этом в правой части строки не содержится операций сложения и вычитания, следовательно, значение аргумента *level* = 1. Левая часть строки может содержать все допустимые операции и операнды. Для нее значение *level* = 0.

Если же в строке не встретилось ни одного знака '+' или '-', то вызывается функция построения дерева-выражения со значением *level* = 1.

2. Когда значение аргумента *level* равно 1, осуществляется поиск первого знака '*' или '/'. Далее происходит построение поддеревьев аналогично первому случаю. В правой части строки не содержится операций умножения и деления, поэтому значение аргумента *level* = 2. Левая часть строки может содержать операции умножения и деления и операнды. Для нее значение *level* = 1.

3. Когда значение аргумента *level* равно 2, создается новый узел с операндом и связывается с узлом соответствующей операции.

```
class Tree
{
    public double operand;    // поле операнда
    public char operation;    // поле операции
    // ссылка на левый операнд операции
    public Tree left;
    // ссылка на правый операнд операции
    public Tree right;
}

class Program
{
    // определение функции создания дерева разбора выражения
    static Tree CreateTree(string str, int level, int len)
    {
        Tree next = null;
        // начинаем анализировать строку-выражение с конца
        int i = len - 1;
```

Листинг 5.1.8. Функция построения дерева разбора арифметического выражения на языке C# (Начало)

```

switch(level)
{
    // выражение может содержать операции '+', '-', '*', '/'
    case 0:
        // ищем позицию первой с конца операции '+' или '-'
        while(i != 0 && str[i] != '+' && str[i] != '-') i--;
        if((str[i] == '+' || str[i] == '-') && i != 0)
        {
            // операция '+' или '-' найдена, создаем
            // узел дерева для этой операции
            next = new Tree();
            next.operation = str[i];
            // левый операнд - выражение, находящееся слева от
            // i-го символа, оно может содержать все виды
            // операций (level=0)
            // осуществляем рекурсивный вызов
            // для построения поддерева этого выражения
            next.left = CreateTree(str, 0, i);
            // правый операнд - выражение,
            // находящееся справа от i-го символа,
            // оно может содержать только
            // операции '*' и '/' (level=1)
            // осуществляем рекурсивный вызов для
            // построения поддерева этого выражения
            next.right = CreateTree
                (str.Substring(i + 1), 1, len - i - 1);
        }
        else
        {
            // операции '+' и '-' не найдены,
            // выражение может содержать только
            // операции '*' и '/' (level=1)
            // осуществляем рекурсивный вызов для
            // построения поддерева выражения
            next = CreateTree(str, 1, len);
            break;
        }
    // выражение может содержать операции '*', '/'
    case 1:
        {
            // ищем позицию первой с конца
            // операции '*' или '/'
            while(i != 0 && str[i] != '*' && str[i] != '/')
                i--;
            if(str[i] == '*' || str[i] == '/')
            {
                // операция '*' или '/' найдена, создаем
                // новый узел дерева для этой операции
                next = new Tree();
                next.operation = str[i];
            }
        }
    }
}

```

Листинг 5.1.8. Функция построения дерева разбора арифметического выражения на языке C# (Продолжение)

```

        // левый операнд - выражение, находящееся
        // слева от i-го символа, оно может содержать
        // операции '*' и '/' (level = 1)
        // осуществляем рекурсивный вызов для
        // построения поддерева этого выражения
        next.left = CreateTree(str, 1, i);
        // правый операнд - выражение, находящееся
        // справа от i-го символа, оно может
        // содержать только операнды (level=2)
        // осуществляем рекурсивный вызов для
        // построения поддерева этого выражения
        next.right = CreateTree
            (str.Substring(i + 1), 2, len - i - 1);
    }
    else
        // операции '*' и '/' не найдены, выражение
        // может содержать только операнды (level=2)
        // осуществляем рекурсивный вызов для
        // построения поддерева выражения
        next = CreateTree(str, 2, len);
        break;
    }
// выражение содержит операнды
case 2:
    // создаем новый листовой узел дерева
    next = new Tree();
    next.operation = '@';
    // поиск следующего символа операции в выражении
    int q = str.IndexOfAny(new char[] { '+', '-', '*', '/' }, 1);
    // если символов операций нет, в строке только
операнд
    if(q < 0) q = str.Length;
    // преобразование операнда в число
    next.operand = Convert.ToDouble(str.Substring(0, q));
    // лист с операндом не имеет потомков
    next.left = null;
    next.right = null;
    break;
}
// возвращаем адрес созданного узла
// первый вызов функции вернет
// адрес корневого узла дерева
return next;
}

```

Листинг 5.1.8. Функция построения дерева разбора арифметического выражения на языке C# (Окончание)

Домашнее задание

1. Дано бинарное дерево. Написать функцию уничтожения дерева.
2. Дано бинарное дерево. Написать функцию вычисления глубины дерева.
3. Дано бинарное дерево. Написать функцию вычисления количества узлов на заданном уровне дерева.
4. Дано дерево сортировки. Написать функцию подсчета количества повторяющихся элементов в узлах дерева.
5. Дано дерево сортировки. Написать функцию нахождения максимального четного значения в дереве.
6. Даны два бинарных дерева. Определить, подобны ли они. Два дерева являются подобными, если либо оба они пусты, либо оба непусты и при этом их левые и правые поддеревья являются подобными, т. е. деревья имеют одинаковую структуру.
7. Дано дерево разбора выражения. Написать функцию вычисления выражения.
8. Дано дерево разбора выражения. Распечатать постфиксную (префиксную) форму этого выражения.
9. Напомним, что постфиксной записью выражения называется такая запись, в которой знак операции следует за операндами. Такая запись не содержит скобок. Например, выражение $a * b$ в постфиксной форме имеет вид $ab *$, $a * b + c$ имеет вид $ab * c +$, $a * (b + c)$ имеет вид $abc + *$.
10. Префиксной записью выражения называется такая запись, в которой знак операции предшествует ее операндам. Такая запись также не содержит скобок. Например, выражение $a * b$ в префиксной форме имеет вид $* ab$, $a * b + c$ имеет вид $+ * abc$, $a * (b + c)$ имеет вид $* a + bc$.

5.2. Графы

На практике часто приходится решать задачи, в которых моделируются схемы расположения объектов и связей между ними. Например, города – это объекты, а дороги, их соединяющие, – это связи. Для таких задач удобно использовать структуру данных, которая называется графом.

Граф – это пара множеств V и E , где V – непустое конечное множество точек, называемых вершинами, а E – множество ребер, соединяющих пары вершин. Так, на рис. 5.2.1 представлен граф с множеством вершин $\{1, 2, 3, 4\}$ и множеством ребер $\{(1, 2), (1, 3), (2, 3), (3, 4)\}$.

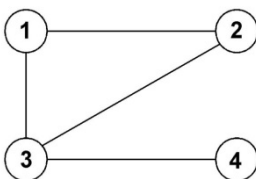


Рис. 5.2.1. Пример графа

Пусть u и v – вершины, а e – ребро, их соединяющее.

В этом случае вершина u и ребро e называются инцидентными (так же как и вершина v и ребро e). На рис. 5.2.1 вершина 1 и ребро $(1, 2)$ инцидентны друг другу. Степень вершины – это число ребер, инцидентных ей, например вершина «1» имеет степень 2. Ребра, инцидентные одной вершине, называются смежными. Вершины, соединенные одним ребром, также называются смежными. Например, смежными являются вершины 1 и 2, смежными ребрами являются $(1, 3)$, $(2, 3)$ и $(3, 4)$.

Граф называется полным, если любые две вершины соединены ребром. Дополнением графа G называется граф G' с тем же множеством вершин, что и в G , причем две вершины в G' смежны тогда и только тогда, когда они несмежны в G . Объединение графов G и G' образует полный граф. Граф, изображенный на рис. 5.2.1, не является полным. Его дополнение изображено на рис. 5.2.2.

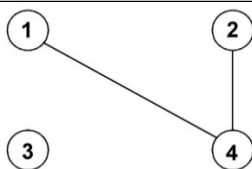


Рис. 5.2.2. Дополнение графа, изображенного на рис. 5.2.1

Если каждому ребру графа ставится в соответствие некоторое число, то оно называется весом ребра, а граф называется взвешенным.

Путь, соединяющий вершины u и v , – это последовательность вершин $v_0, v_1, \dots, v_n$, где $v_0 = u$, а $v_n = v$ и для любых $i = 0..n$ вершины v_i и v_{i+1} соединены ребром. Если граф взвешенный, то длиной пути называется сумма весов его ребер. Путь называется замкнутым, если $v_0 = v_n$. Цикл – это замкнутый путь, в котором ни одно ребро не повторяется дважды. Простой цикл – это цикл, в котором ни одна вершина не повторяется дважды.

Расстояние между двумя вершинами – это длина кратчайшего пути, соединяющего эти вершины.

Деревом называется связный граф без циклов. Пример дерева представлен на рис. 5.2.3.

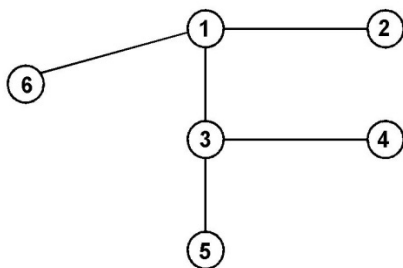


Рис. 5.2.3. Пример дерева – связного графа без циклов

Бинарные деревья сортировки, алгоритмы работы с которыми были рассмотрены в предыдущем разделе, являются частным случаем такого графа с выделенной корневой вершиной и направленными ребрами от родителя к потомку.

Источник графа – это вершина, из которой достижимы все остальные вершины (из этой вершины можно построить путь в любую вершину графа). Сток графа – это вершина, достижимая из всех других вершин.

Каркасом называется дерево, содержащее все вершины исходного графа и часть его ребер. Граф может иметь несколько каркасов.

Каркасами для графа, представленного на рис. 5.2.1, являются следующие графы (деревья) (рис. 5.2.4):



Рис. 5.2.4. Каркасы графа, изображенного на рис. 5.2.1

Приведем часто используемые способы представления графов.

1. Матрица инцидентности

$$A(n, m) = \{a_{ij}\},$$

где n – число вершин, m – число ребер графа.

$$a_{ij} = \begin{cases} 1, & \text{вершина } i \text{ инцидентна ребру } j, \\ 0, & \text{вершина } i \text{ и ребро } j \text{ не инцидентны.} \end{cases}$$

В каждом столбце такой матрицы содержится ровно две единицы. Равных столбцов в матрице нет.

Приведем пример матрицы инцидентности для графа, изображенного на рис. 5.2.1.

$$A = \begin{matrix} & \begin{matrix} (1,2) & (1,3) & (2,3) & (3,4) \end{matrix} \\ \begin{pmatrix} 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix} \end{matrix}$$

Недостаток данного способа представления состоит в том, что требуется $m * n$ ячеек памяти, большинство из которых заполнены нулями.

2. Матрица смежности.

$$A(n, n) = \{a_{ij}\},$$

где n – число вершин.

$$a_{ij} = \begin{cases} 1, & \text{вершина } i \text{ смежна вершине } j, \\ 0, & \text{вершина } i \text{ не смежна вершине } j. \end{cases}$$

В случае взвешенного графа элементу a_{ij} вместо 1 присваивается значение соответствующего веса, а вместо 0 – большое число,

обозначающее бесконечность (в текстах программ большое число обозначается *INT_MAX*). Эта матрица является симметричной с нулями на главной диагонали. Число единиц в строке равно степени соответствующей вершины.

Приведем пример матрицы смежности для графа, изображенного на рис. 5.2.1.

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

Преимуществом данного способа является легкость получения информации о существовании ребра между двумя вершинами. По сравнению с предыдущим способом требуется меньшее количество памяти для графа с большим числом ребер. Недостатком считается постоянный размер матрицы, не зависящий от числа ребер графа.

3. Список ребер.

Граф представляется в виде списка (массива) элементов, хранящих информацию про одно ребро (номер начальной и конечной вершин, длина ребра и пр.).

4. Динамический список смежности (хэш-таблица).

В данном случае создается массив списков по количеству вершин графа. Каждый *i*-й список из массива содержит номера вершин, смежных с *i*-й вершиной.

Приведем пример (рис. 5.2.5) динамического списка смежности для графа, изображенного на рис. 5.2.1:

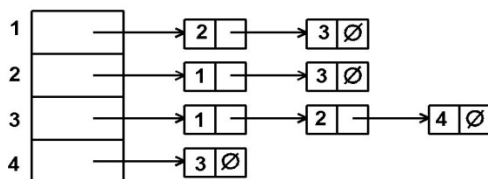


Рис. 5.2.5. Список смежности графа, изображенного на рис. 5.2.1

Для каждого из представлений графа приведем в листинге 5.2.1 описание структур для хранения данных о графе (язык C++).

```

// структура графа в виде матрицы инцидентности
struct Graph1
{
    int n;        // количество вершин графа
    int m;        // количество ребер графа
    int** a;      // матрица инцидентности графа
};

// структура графа в виде матрицы смежности
struct Graph2
{
    int n;        // количество вершин графа
    int m;        // количество ребер графа
    int** a;      // матрица смежности графа
};

// граф в виде массива ребер
// структура для хранения информации об одном ребре графа
struct Edge
{
    int begin;    // номер начальной вершины ребра
    int end;      // номер конечной вершины ребра
};

// структура графа в виде списка (массива) ребер
struct Graph3
{
    int n;        // количество вершин графа
    int m;        // количество ребер графа
    Edge* edges;  // массив элементов типа Edge
};

// граф в виде списков смежности
// структура одного элемента списка,
// хранящего номер смежной вершины
struct ElementList
{
    int number;    // номер вершины
    ElementList* next; // адрес следующего элемента списка
};

// структура графа в виде списков смежности
struct Graph4
{
    int n;        // количество вершин графа
    int m;        // количество ребер графа
    ElementList** nodes; // массив, содержащий списки
                        // смежных вершин
};

```

**Листинг 5.2.1. Программный код структур хранения данных
для представления графов в программе**

Задача 1. Дан граф в виде списка ребер. Преобразовать граф в матрицу смежности.

Задачу преобразования одного представления графа, например удобного для ввода, в другой, удобный для реализации алгоритма обработки данных графа, часто приходится решать на практике. Так, например, представление в виде списка ребер действительно удобно для ввода с клавиатуры, особенно в случаях, когда граф имеет много вершин и очень мало ребер. С другой стороны, представление графа в виде матрицы смежности является наиболее часто используемым для алгоритмов построения кратчайших путей в графе.

Кратко алгоритм можно описать так. Сначала выделяем память под матрицу смежности. Первоначально заполняем все ее элементы нулями. Проходя по всем ребрам из массива, заполняем элементы матрицы единицами исходя из номеров начальной и конечной вершин ребра.

Представим программный код функции преобразования представления графа на языке C++ в листинге 5.2.2. Входными параметрами функции являются ссылки на структуру для представления графа в виде исходного списка ребер (*Graph3*) и на структуру для представления графа в виде матрицы смежности (*Graph2*).

```
// определение функции преобразования представления графа
// из массива дуг в матрицу смежности
void Preobr32(Graph3& g1, Graph2& g2)
{
    g2.n = g1.n; g2.m = g1.m;
    // выделение памяти под матрицу смежности
    g2.a = new int* [g2.n];
    for(int i = 0; i < g2.n; i++)
        g2.a[i] = new int[g2.n];
    for(int i = 0; i < g2.n; i++)
        for(int j = 0; j < g2.n; j++)
            g2.a[i][j] = 0;
    // заполнение элементов матрицы единицами
    // в зависимости от номеров начальной и конечной
    // вершин каждого ребра
    for(int i = 0; i < g1.m; i++)
    {
        g2.a[g1.edges[i].begin - 1][g1.edges[i].end - 1] = 1;
        g2.a[g1.edges[i].end - 1][g1.edges[i].begin - 1] = 1;
    }
}
```

Листинг 5.2.2. Функция преобразования представления графа из списка ребер в матрицу смежности на языке C++

Задача 2. Дан граф в виде списка ребер. Преобразовать граф в динамический список смежности.

Задача 2 аналогична задаче 1, но в качестве целевой формы представления является более сложный по своей структуре динамический список смежности. Для его создания сначала выделяется память под массив заголовков списков смежных вершин. В начальный момент все списки пусты. Далее для каждого ребра из массива ребер (номер начальной вершины сохраняем в переменной *begin*, номер конечной вершины – в переменной *end*) осуществляется добавление новых элементов в списки: элемента с вершиной *end* в конец списка смежности для вершины *begin* и элемента с вершиной *begin* в конец списка смежности для вершины *end*. Приведем программный код функции преобразования в листинге 5.2.3 на языке программирования C# вместе с описанием классов для хранения используемых представлений графа.

```
// граф в виде массива ребер
// класс для хранения информации об одном ребре графа
class Edge
{
    public int begin;    // номер начальной вершины ребра
    public int end;      // номер конечной вершины ребра
}
// класс графа в виде списка (массива) ребер
class Graph3
{
    public int n;        // количество вершин графа
    public int m;        // количество ребер графа
    public Edge[] edges; // массив элементов типа Edge
}

// граф в виде списков смежности
// класс для хранения номера смежной вершины
class ElementList
{
    public int number;    // номер вершины
    public ElementList next; // ссылка на следующий элемент
}
// класс графа в виде списков смежности
class Graph4
{
    public int n;        // количество вершин графа
    public int m;        // количество ребер графа
    public ElementList[] nodes; // массив, содержащий списки
                                // смежных вершин
}
```

Листинг 5.2.3. Функция преобразования представления графа из списка ребер в динамический список смежности на языке C# (Начало)

```

// определение функции преобразования представления графа
// из массива дуг в списки смежности
static void Preobr34(Graph3 g1, Graph4 g2)
{
    g2.n = g1.n; g2.m = g1.m;
    // создание массива списков смежных вершин
    // по количеству вершин графа
    g2.nodes = new ElementList[g2.n];
    for(int i = 0; i < g2.n; i++)    g2.nodes[i] = null;
    for(int i = 0; i < g1.m; i++)
    {
        // создание для каждого ребра двух элементов
        // в списках смежных вершин
        int begin = g1.edges[i].begin, end = g1.edges[i].end;
        // конечная вершина ребра смежна начальной
        if(g2.nodes[begin - 1] == null)
        {
            // создание первого элемента списка смежных
            // вершин, если список пуст
            g2.nodes[begin - 1] = new ElementList();
            g2.nodes[begin - 1].next = null;
            g2.nodes[begin - 1].number = end;
        }
        else {
            // добавление нового элемента в «хвост»
            // списка смежных вершин
            ElementList current = g2.nodes[begin - 1], help;
            while(current.next != null)
                current = current.next;
            help = new ElementList();
            help.next = null; help.number = end;
            current.next = help;
        }
        // начальная вершина ребра смежна конечной
        if(g2.nodes[end - 1] == null)
        {
            g2.nodes[end - 1] = new ElementList();
            g2.nodes[end - 1].next = null;
            g2.nodes[end - 1].number = begin;
        }
        else {
            ElementList current = g2.nodes[end - 1], help;
            while(current.next != null)
                current = current.next;
            help = new ElementList();
            help.next = null; help.number = begin;
            current.next = help;
        }
    }
}

```

Листинг 5.2.3. Функция преобразования представления графа из списка ребер в динамический список смежности на языке С# (Окончание)

Задача 3. Дан граф в виде матрицы смежности. Определить длины всех путей из узла A в узел B .

Построение путей в графе, удовлетворяющих различным условиям, является одной из самых популярных задач на графах. В данном случае мы рассматриваем алгоритм полного перебора путей между двумя вершинами графа.

Разберем два варианта решения этой задачи – рекурсивный и нерекурсивный. Идея алгоритма в обоих случаях заключается в следующем. Находясь в некоторой текущей вершине A , последовательно переходим в каждую из смежных с ней вершин. Каждую пройденную вершину помечаем как вершину, включенную в путь, и одновременно добавляем длину соответствующего ребра к длине строящегося пути. Таким образом, все вершины фактически разделены на две группы: уже включенные в путь и оставшиеся. Теперь задача сводится к нахождению всех путей из последней помеченной вершины в конечную, при этом рассматриваются только те смежные с ней вершины, которые еще не включены в путь. Если на каком-то этапе достигается конечная вершина, печатаем путь. Если для некоторой вершины все пути рассмотрены, снимаем с нее пометку и возвращаемся в предыдущую вершину, продолжая поиск оставшихся путей из нее.

Например, пусть дан граф, изображенный на рис. 5.2.6.

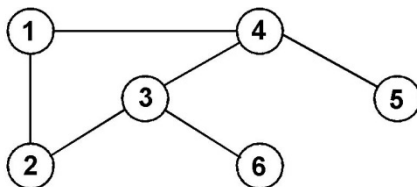


Рис. 5.2.6. Пример графа

Пусть требуется найти все пути из вершины 1 в вершину 6. Помечаем вершину 1 как включенную в путь. Для нее существуют две смежные вершины – 2 и 4. Сначала переходим в вершину 2 и помечаем ее. Теперь задача сводится к поиску всех путей из вершины 2 в вершину 6. Для текущей вершины 2 смежными являются две – 1 и 3. Однако вершина 1 уже включена в путь, поэтому ее не рассматриваем и переходим в вершину 3. Из нее сначала находится путь $3 \rightarrow 4 \rightarrow 5$. Вершина 5 не имеет непомеченных смежных вершин, поэтому возвращаемся в вершину 4. Для нее нерассмотренных возможных путей больше нет. Поэтому возвращаемся в вершину 3, для которой существует еще одна нерассмотренная и непомеченная смежная вершина 6. Таким образом, найден путь $1 \rightarrow 2 \rightarrow 3 \rightarrow 6$ из начальной вершины в конечную.

Аналогично осуществляем возврат до вершины 1. После рассмотрения всех возможных путей через вершину 2, остаются пути, которые проходят через вершину 4. Из всех возможных путей:

$$1 \rightarrow 4 \rightarrow 3 \rightarrow 2, 1 \rightarrow 4 \rightarrow 3 \rightarrow 6, 1 \rightarrow 4 \rightarrow 5$$

искомым путем является $1 \rightarrow 4 \rightarrow 3 \rightarrow 6$. Таким образом, найдены все возможные пути из вершины 1 в вершину 6.

Функция *AllWays()*, представленная в листинге 5.2.4, реализует рекурсивный вариант этого алгоритма. Параметрами функции являются исходный граф в виде матрицы смежности *g*, номера начальной вершины *s* и конечной вершины *t*, массив пометок вершин графа *x*, номер текущей вершины в строящемся пути *k* и длина уже построенного пути *sum*. Пометка в массиве *x* представляет собой номер вершины в построенном пути, например $x[3] = 2$ означает, что вершина 3 в построенном пути является второй.

```
// определение рекурсивной функции поиска всех путей
// в графе, соединяющих две заданные вершины
void AllWays(Graph2 g, int s, int t, int* x, int k, int sum)
{
    s--; t--;
    // указываем, что порядковый номер s-й вершины графа
    // в строящемся пути равен k
    x[s] = k;
    // условие выхода из рекурсии -
    // проверка достижения конечной вершины пути
    if(s == t)
    {
        // цикл распечатки найденного пути
        int nom = 1;
        while (true)
        {
            for(int i = 0; i < g.n; i++)
                if(x[i] == nom)
                {
                    cout << (i + 1); break;
                }
            if(i == b) break;
            cout << "->"; nom++;
        }
        cout << " sum = " << sum << endl;
        x[s] = 0;
        return;
    }
}
```

Листинг 5.2.4. Рекурсивная функция поиска всех путей в графе, соединяющих две заданные вершины, на языке C++ (Начало)


```

// цикл поиска непомяченных вершин, смежных s-й вершине
for(int j = 0; j < g.n; j++)
    if(x[j] == 0 && g.a[s][j] > 0 && g.a[s][j] != INT_MAX)
        // переход в смежную вершину с номером j
        // рекурсивный вызов для поиска всех путей
        // из j-й вершины в t-ю
        AllWays(g, j + 1, t + 1, x, k + 1, sum + g.a[s][j]);
// возврат из s-й вершины, снятие пометки о том,
// что s-я вершина включена в путь
x[s]=0;
}

```

Листинг 5.2.4. Рекурсивная функция поиска всех путей в графе, соединяющих две заданные вершины, на языке C++ (Окончание)

Для нерекурсивного варианта этой функции требуется только граф и номера начальной и конечной вершин. Массив пометок, номер вершины в пути и длина пути будут созданы внутри функции. Приведем программный код для этой функции в листинге 5.2.5 на языке программирования C#. В данном случае для реализации алгоритма помимо класса для хранения графа в виде матрицы смежности нам понадобятся функции работы со стеком.

```

// класс для представления графа в виде матрицы смежности
class Graph2
{
    public int n;        // количество вершин графа
    public int m;        // количество ребер графа
    public int[,] a;     // матрица смежности графа
}

// определение нерекурсивной функции поиска всех путей
// в графе, соединяющих две заданные вершины
void AllWays(Graph2 g, int s, int t, int[] x)
{
    int v = s,          // номер текущей вершины
    k = 2,              // порядковый номер вершины в пути
    jj = -1,            // номер вершины, с которой начинается
                        // поиск смежных вершин
    sum = 0,            // длина построенного пути
    j, f, ii, пом;
    // если начальная и конечная вершины совпадают,
    // задача тривиальна и путей нет
    if(s == t) return;
    // корректируем индексы - пользователь задает, начиная с 1,
    // для массивов требуется начинать нумерацию с 0
    s--; t--;
}

```

Листинг 5.2.5. Нерекурсивная функция поиска всех путей в графе, соединяющих две заданные вершины, на языке C# (Начало)

```

// вспомогательный стек для хранения вершин возврата
Stack st;
st.head = NULL;

// пометка начальной вершины
x[v] = 1;
while(true)
{
    f = 0;
    // поиск смежных вершин с v-й
    for(j = jj + 1; j < g.n; j++)
        if(x[j] == 0 && g.a[v,j] > 0 && g.a[v,j] != int.MaxValue)
        {
            // найдена смежная вершина j
            x[j] = k; // вершина включается в путь,
            //добавляется длина дуги <v,j>
            sum += g.a[v,j];
            if(j == t)
            {
                // если найдена конечная вершина пути
                // печатаем найденный путь
                nom = 1;
                while (true)
                {
                    for(ii = 0; ii < g.n; ii++)
                        if(x[ii]==nom)
                        {
                            Console.Write(ii+1);
                            break;
                        }
                    if(ii == t)
                    {
                        Console.WriteLine("sum = {0}", ii+1);
                        break;
                    }
                    Console.Write("->");
                    nom++;
                }
                // осуществляем возврат в предыдущую
                // вершину, т. е. снимаем пометку с
                // j-й вершины и вычитаем длину
                // ребра <v,j> из длины пути
                x[j] = 0;
                sum -= g.a[v,j];
            }
        }
}

```

Листинг 5.2.5. Нерекурсивная функция поиска всех путей в графе, соединяющих две заданные вершины, на языке C# (Продолжение)

```

else
{
    // конечная вершина не достигнута
    // помещаем вершину v в стек,
    // запоминаем найденную смежную
    // вершину в jj
    k++;
    PushStack(st, v);
    jj = j;
    f = 1;
    break;
}
}
if(f == 1)
{
    // если перешли в очередную вершину пути,
    // запоминаем ее номер в v и указываем,
    // что смежные вершины будем искать,
    // начиная с нулевой вершины
    v = jj; jj = -1;
}
else
{
    // не нашли смежных непомеченных вершин,
    // осуществляем возврат в предыдущую вершину
    jj = v;    // сохраняем номер текущей вершины,
               // чтобы продолжить поиск смежных
               // вершин со следующей за ней
    if(PopStack(st, v) != -1)
    {
        // извлекли из стека предшествующую
        // вершину и поместили ее номер в v,
        // с прежней вершины jj снимаем пометку
        x[jj] = 0;
        k--;
        sum -= g.a[v, jj];
    }
    else
        // стек оказался пустым, т. е. были
        // просмотрены все возможные варианты,
        // осуществляем выход из цикла
        break;
}
}
}
}

```

Листинг 5.2.5. Нерекурсивная функция поиска всех путей в графе, соединяющих две заданные вершины, на языке C# (Окончание)

Задача 4. Дан взвешенный граф в виде матрицы смежности. Найти кратчайший путь между двумя заданными вершинами (s – начальная вершина, t – конечная вершина).

Одним из самых известных алгоритмов решения данной задачи является алгоритм Дейкстры (1959).

Кратчайший путь будет представлен с помощью двух массивов T и H . $T[v]$ – длина кратчайшего пути от начальной вершины к v , $H[v]$ – вершина, непосредственно предшествующая v на кратчайшем пути.

Для решения задачи требуется вспомогательный массив X размера n . С его помощью будут помечаться вершины, которые уже были просмотрены, поэтому начальные значения элементов массива равны нулю. Массив T первоначально должен заполняться бесконечными значениями, поскольку длина кратчайшего пути пока неизвестна.

Для начальной вершины s $H(s) = 0$ (начальной вершине ничего не предшествует), $T(s) = 0$ (путь $s \rightarrow s$ имеет длину 0), $X(s) = 1$ (вершина просмотрена). Фиксируется вершина $v = s$.

Для зафиксированной вершины перебираем все смежные ей непросмотренные вершины (переменная u). Если длина пути из s в u через v ($T[v] + g.a[v, u]$) (длина кратчайшего пути до вершины v плюс вес ребра $v \rightarrow u$) меньше, чем прежнее значение $T[u]$, меняем его и запоминаем в $H[u]$ предшествующую вершину v . Далее находим минимальный элемент массива T , соответствующий непросмотренным вершинам (в переменной v запоминаем его индекс). Если все оставшиеся элементы массива T равны бесконечности, можно сделать вывод, что требуемого пути не существует (в этом случае функция возвращает значение 0). Если найденный минимум соответствует вершине t , то задача решена (в этом случае функция возвращает значение 1). В противном случае помечаем вершину v как пройденную и осуществляем перебор смежных ей вершин.

Рассмотрим процесс решения задачи поиска кратчайшего пути из вершины 1 в вершину 6 на графе, представленном на рис. 5.2.7. Процесс решения приведем в таблице 5.

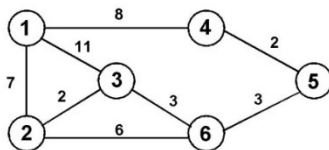


Рис. 5.2.7. Пример взвешенного графа

Таблица 5

Поиск кратчайшего пути алгоритмом Дейкстра

№	Действия с комментариями	Зафикс. вершина	Массив пометок X	Массив длин T	Массив путей H
1	Фиксируем начальную вершину пути	1	{1, 0, 0, 0, 0, 0}	{1, 0, 0, 0, 0, 0}	{0, ∞, ∞, ∞, ∞, ∞}
2	Для вершины 1 смежными непомяченными вершинами являются вершины 2, 3, 4. Меняем для них элементы массивов T и H , т. е. $T[2]$, $H[2]$, $T[3]$, $H[3]$, $T[4]$, $H[4]$ (в T заносим длину ребра, в H – номер зафиксированной вершины)	1	{1, 0, 0, 0, 0, 0}	{0, 7, 11, 8, ∞, ∞}	{0, 1, 1, 1, ∞, ∞}
3	Находим в массиве T минимальный элемент, среди тех, которые соответствуют непомяченным вершинам ($T[2]=7$). Помечаем соответствующую минимальному значению вершину 2 и фиксируем ее	2	{1, 1, 0, 0, 0, 0}	{0, 7, 11, 8, ∞, ∞}	{0, 1, 1, 1, ∞, ∞}
4	Для вершины 2 смежными непомяченными вершинами являются вершины 3, 6. Меняем для них элементы массивов T и H	2	{1, 1, 0, 0, 0, 0}	{0, 7, 9, 8, ∞, 13}	{0, 1, 2, 1, ∞, 2}
5	Находим в массиве T минимальный элемент среди тех, которые соответствуют непомяченным вершинам ($T[4]=8$). Помечаем вершину 4 и фиксируем ее	4	{1, 1, 0, 1, 0, 0}	{0, 7, 9, 8, ∞, 13}	{0, 1, 2, 1, ∞, 2}
6	Для вершины 4 смежной непомяченной вершиной является вершина 5. Меняем $T[5]$ и $H[5]$	4	{1, 1, 0, 1, 0, 0}	{0, 7, 9, 8, 10, 13}	{0, 1, 2, 1, 4, 2}

№	Действия с комментариями	Зафикс. вершина	Массив пометок X	Массив длин T	Массив путей H
7	Находим в массиве T минимальный элемент, среди соответствующих непомеченным вершинам ($T[3]=9$). Помечаем вершину 3 и фиксируем ее	3	{1, 1, 1, 1, 0, 0}	{0, 7, 9, 8, 10, 13}	{0, 1, 2, 1, 4, 2}
8	Для вершины 3 смежной непомеченной вершиной является вершина 6. Меняем элементы $T[6]$ и $H[6]$	3	{1, 1, 1, 1, 0, 0}	{0, 7, 9, 8, 10, 12}	{0, 1, 2, 1, 4, 3}
9	Находим в массиве T минимальный элемент среди тех, которые соответствуют непомеченным вершинам ($T[5]=10$). Помечаем вершину 5 и фиксируем ее	5	{1, 1, 1, 1, 1, 0}	{0, 7, 9, 8, 10, 12}	{0, 1, 2, 1, 4, 3}
10	Для вершины 5 смежной непомеченной вершиной является вершина 6. Минимальным элементом массива T среди тех, которые соответствуют непомеченным вершинам, является $T[6]=12$. Поскольку вершина 6 – конечная вершина, задача решена				

Таким образом, длина кратчайшего пути из вершины 1 в вершину 6 равна 12 ($T[6]$).

Далее в листинге 5.2.6 приведем программный код функции, реализующей решения задачи поиска кратчайшего пути с помощью алгоритма Дейкстры. Функцию напомним на языке программирования C++.

```
// определение функции поиска кратчайшего пути
// между двумя вершинами с помощью алгоритма Дейкстры
int Dijkstra(Graph2& g, int s, int t, int* H, int* T)
{
    // создание вспомогательного массива пометок
    int* X = new int[g.n];
    int flag, i, m, u,
    v = s - 1; // номер текущей вершины
```

Листинг 5.2.6. Функция поиска кратчайшего пути в графе, соединяющего две заданные вершины, на языке C++ (Начало)

```

// начальная инициализация массива пометок X и массива
// длин кратчайших путей T
for(i = 0; i < g.n; i++)
{
    T[i] = INT_MAX; X[i] = 0;
}
// инициализация элементов массивов для начальной вершины s
H[s - 1] = 0; T[s - 1] = 0; X[s - 1] = 1;
while(true)
{
    // цикл поиска непомяченных вершин,
    // смежных с s-й вершиной
    for(u = 0; u < g.n; u++)
        if(g.a[v][u] != 0 && g.a[v][u] != INT_MAX)
        {
            // корректировка длин кратчайших путей происходит,
            // если путь из вершины s в вершину u, проходящий через
            // вершину v, будет короче, чем прежний
            if(X[u] == 0 && T[u] > T[v] + g.a[v][u])
            {
                T[u] = T[v] + g.a[v][u]; H[u] = v + 1;
            }
        }
    // поиск непомяченной вершины, которой соответствует
    // минимальный кратчайший путь из s-й вершины
    m = MAX_INT; v = 0;
    for(u = 0; u < g.n; u++)
        if(X[u] == 0 && T[u] < m)
        {
            v = u + 1; m = T[u];
        }
    // если нет путей во все непомященные вершины,
    // пути в конечную тоже не существует
    if(v == 0)
    {
        delete [] X; return 0;
    }
    // если найденная вершина является конечной, путь найден
    if(v == t)
    {
        delete [] X; return 1;
    }
    // конечной вершины еще не достигли, помечаем
    // вершину с минимальной длиной кратчайшего пути
    // как пройденную, и она становится текущей
    v--; X[v] = 1;
}
}

```

Листинг 5.2.6. Функция поиска кратчайшего пути в графе, соединяющем две заданные вершины, на языке C++ (Окончание)

Особо требуется пояснить, как осуществить печать найденного пути из массива *H*.

Так как элемент $H[i]$ содержит вершину, предшествующую i -й вершине пути, то сам путь можно получить только в обратном порядке (от вершины t к вершине s). Начиная с конечной вершины $v = t$, переходим от вершины v к вершине $H[v]$, пока не дойдем до начальной вершины s (для распечатки пути в прямом порядке можно применить стек). В листинге 5.2.7 представлен фрагмент программного кода, осуществляющий печать пути в обратном порядке следования вершин.

```
. . .  
i = t - 1;  
cout << t << "<-";  
while (H[i] != 0)  
{  
    cout << H[i] << "<-";  
    i = H[i] - 1;  
}  
. . .
```

Листинг 5.2.7. Фрагмент кода печати кратчайшего пути, построенного алгоритмом Дейкстры

Рассмотрим процесс печати кратчайшего пути для примера графа, представленного на рис. 5.2.6. Результатом решения задачи является массив $H[] = \{0, 1, 2, 1, 4, 3\}$. Находясь в конечной вершине 6, получаем номер предыдущей вершины на пути $H[6] = 3$. Далее вершине 3 предшествует $H[3] = 2$, второй – $H[2] = 1$. Таким образом, кратчайший путь из вершины 1 в вершину 6: $1 \rightarrow 2 \rightarrow 3 \rightarrow 6$.

Задача 5. Дан взвешенный граф в виде матрицы смежности. Найти кратчайшие пути между всеми парами вершин.

Для решения этой задачи часто используется алгоритм Флойда (1962).

Результатом работы алгоритма являются две матрицы H и T размера $n \times n$, где n – количество вершин графа. Элемент матрицы $T[i][j]$ соответствует кратчайшему расстоянию между вершинами i и j . В матрице H хранится информация о кратчайших путях:

$$H[i][j] = \begin{cases} k, & \text{если } k \text{ — номер первой вершины, в которую} \\ & \text{необходимо перейти, двигаясь из } i \text{ в } j; \\ 0, & \text{если из вершины } i \text{ в вершину } j \text{ нет пути.} \end{cases}$$

Сначала осуществляется начальная инициализация матриц H и T . Матрица T совпадает с матрицей смежности, т. е. фиксируются пути, состоящие из одного ребра. В матрице H для каждой пары смежных

вершин в качестве начального значения берется номер конечной вершины ребра.

Для любой пары вершин j и k осуществляется попытка включить в путь новую вершину i . Это происходит при выполнении следующих условий:

- 1) вершина i не является концом рассматриваемого пути, т. е. не совпадает с j или с k ;
- 2) существует путь из j в i и из i в k ;
- 3) длина полученного пути через вершину i меньше, чем прежняя длина.

Если все эти условия выполнены, то меняется длина кратчайшего пути в матрице T и первой вершиной на пути из j в k становится первая вершина на пути из j в i .

Приведем в листинге 5.2.8 программный код на языке C++ функции, осуществляющей решение задачи поиска всех кратчайших путей с помощью алгоритма Флойда.

```
// определение функции построения всех кратчайших
// путей в графе с помощью алгоритма Флойда
void Floyd(Graph2 g, int**& H, int**& T)
{
    int i, k, j;
    // выделение памяти под матрицу длин кратчайших путей T
    // и матрицу путей H
    T = new int*[g.n];
    H = new int*[g.n];
    for(i = 0; i < g.n; i++)
    {
        T[i] = new int[g.n];
        H[i] = new int[g.n];
    }
    // начальная инициализация матриц T и H согласно
    // матрице смежности графа
    for(i = 0; i < g.n; i++)
        for(j = 0; j < g.n; j++)
        {
            T[i][j] = g.a[i][j];
            if(g.a[i][j] == INT_MAX) H[i][j] = 0;
            else H[i][j] = j + 1;
        }
}
```

Листинг 5.2.8. Функция получения всех кратчайших путей алгоритмов Флойда на языке C++ (Начало)

```

for(i = 0; i < g.n; i++)
{
    for(j = 0; j < g.n; j++)
        for(k = 0; k < g.n; k++)
            // если можно проехать из j-ой вершины в
            // k-ю через i-ю более коротким путем,
            // меняем длину T[j][k] и путь
            if(i != j && i != k && T[j][i] != INT_MAX &&
                T[i][k] != INT_MAX && T[j][k] > T[j][i] + T[i][k])
            {
                H[j][k] = H[j][i];
                T[j][k] = T[j][i] + T[i][k];
            }
        }
    }
}

```

Листинг 5.2.8. Функция получения всех кратчайших путей алгоритмов Флойда на языке C++ (Окончание)

Задача 6. Дан взвешенный граф в виде матрицы смежности. Найти диаметр графа, т. е. максимальное расстояние между всевозможными парами его вершин.

Параметрами функции являются исходный граф и две ссылки на переменные, в которые будут помещены номера двух вершин (расстояние между этими вершинами является максимальным в графе).

Решить эту задачу можно, используя алгоритм Флойда. Полученная с помощью этого алгоритма матрица T является матрицей расстояний. Находим максимальный элемент матрицы, значение которого и будет являться диаметром графа. Программный код приведен в листинге 5.2.9.

```

// определение функции поиска диаметра графа
int Diametr(Graph2 g, int &s, int &t)
{
    int ** T, **H;
    // получение матрицы расстояний между вершинами графа
    Floyd(g, H, T);
    // поиск максимального расстояния
    int max = -INT_MAX;
    for(int i = 0; i < g.n; i++)
        for(int j = 0; j < g.n; j++)
            if(T[i][j] != INT_MAX && i != j && T[i][j] > max)
            {
                max = T[i][j]; s = i + 1; t = j + 1;
            }
    return max;
}

```

Листинг 5.2.9. Функция получения диаметра графа на языке C++

Задача 7. Дан взвешенный граф в виде матрицы смежности. Построить для него каркас минимального веса.

Для построения каркаса имеется несколько алгоритмов. Мы будем использовать алгоритм Прима (1957).

Опишем процесс создания функции и определим необходимые структуры данных на языке программирования C#.

Сформированный каркас графа будет храниться в массиве ребер. Каждому ребру соответствуют номера начальной и конечной вершин и длина. Для этого создадим класс *Edge* (листинг 5.2.10):

```
class Edge
{
    // номера начальной и конечной вершин ребра
    public int begin, end;
    // длина ребра
    public int len;
};
```

Листинг 5.2.10. Класс для хранения информации об одном ребре (C#)

В алгоритме Прима каркас формируется посредством последовательного добавления вершин. Для определения добавляемой на каждой итерации вершины используются два вспомогательных массива. Элемент массива $\alpha[i]$ – это ближайшая к i -й вершина, уже включенная в каркас, $\beta[i]$ – это длина ребра, соединяющего i -ю вершину с вершиной $\alpha[i]$, $nodes$ – это массив номеров вершин, включенных в каркас.

В $nodes$ включается начальная вершина. Просматривая все смежные с ней вершины (переменная v), запоминаем для них в массиве α начальную вершину, а в массиве β – длину соответствующего ребра. Для несмежных вершин соответственно $\alpha[v] = -1$ и $\beta[v] = \infty$.

До тех пор пока в каркас не будут включены все вершины, выполняем следующие действия:

- 1) среди вершин, не включенных в каркас, находим ближайшую к нему. Обозначим найденную вершину через w ;

- 2) добавляем вершину w в каркас:

- а) в массив $nodes$ заносим вершину w ;

- б) в массив ребер заносим ребро $\langle \alpha[w], w \rangle$;

- 3) пересчитываем те элементы массивов α и β , которые соответствуют не включенным в каркас вершинам. Для них вершина w может стать ближайшей из каркаса.

Рассмотрим процесс решения задачи построения каркаса минимального веса алгоритмом Прима на примере графа, изображенного на рис. 5.2.6 (табл. 6).

Таблица 6
Построение каркаса минимального веса алгоритмом Прима

№	Действия с комментариями	Теку- щая верши на	Массив ребер <i>ostov</i>	Массив вер- шин в каркасе <i>nodes</i>	Массив ближ. вершин <i>alpha</i> и длин <i>beta</i>
1	Включаем в каркас вершину 1	1	пуст	{1}	–
2	Заполняем <i>alpha</i> и <i>beta</i> . Смежными с вершиной 1 являются вершины 2, 3, 4. В <i>beta</i> заносим длины всех ребер, смежных с вершиной 1, в <i>alpha</i> – номер текущей вершиной 1	1	пуст	{1}	$\alpha = \{0, 1, 1, 1, -1, -1\}$ $\beta = \{0, 7, 11, 8, \infty, \infty\}$
3	Находим в <i>beta</i> минимальный элемент среди тех, которые соответствуют вершинам, не включенным в <i>nodes</i> : $\beta[2]=7$. Включаем вершину 2 в массив <i>nodes</i> , а ребро <1, 2> в каркас. Вершина 2 становится текущей	2	{<1,2>}	{1, 2}	$\alpha = \{0, 1, 1, 1, -1, -1\}$ $\beta = \{0, 7, 11, 8, \infty, \infty\}$

№	Действия с комментариями	Текущая вершина	Массив ребер <i>ostov</i>	Массив вершин в каркасе <i>nodes</i>	Массив ближ. вершин <i>alpha</i> и длин <i>beta</i>
4	Изменяем <i>alpha</i> и <i>beta</i> . Смежными с вершиной 2 являются вершины 3, 6. В <i>beta</i> заносим длины ребер из вершины 2 во все смежные, в <i>alpha</i> – номер текущей вершины 2	2	{<1,2>}	{1, 2}	$\alpha = \{0, 1, 2, 1, -1, 2\}$ $\beta = \{0, 7, 2, 8, \infty, 6\}$
5	Находим в <i>beta</i> минимальный элемент среди тех, которые соответствуют вершинам, не включенным в <i>nodes</i> : $\beta[3]=3$. Включаем вершину 3 в массив <i>nodes</i> , а ребро <2, 3> в каркас. Вершина 3 становится текущей	3	{<1,2>, <2,3>}	{1, 2, 3}	$\alpha = \{0, 1, 2, 1, -1, 2\}$ $\beta = \{0, 7, 2, 8, \infty, 6\}$
6	Изменяем <i>alpha</i> и <i>beta</i> . Смежной с вершиной 3 является вершина 6. В <i>beta</i> заносим длину ребра <3,6>, в <i>alpha</i> – номер текущей вершины 3	3	{<1,2>, <2,3>}	{1, 2, 3}	$\alpha = \{0, 1, 2, 1, -1, 3\}$ $\beta = \{0, 7, 2, 8, \infty, 3\}$

№	Действия с комментариями	Теку- щая верши- на	Масси в ребер <i>ostov</i>	Массив вер- шин в каркасе <i>nodes</i>	Массив ближ. вершин <i>alpha</i> и длин <i>beta</i>
7	Находим в <i>beta</i> минимальный элемент среди тех, которые соответствуют вершинам, не включенным в <i>nodes</i> : <i>beta</i> [6]=3. Включаем вершину 6 в массив <i>nodes</i> , а ребро <3, 6> в каркас. Вершина 6 становится текущей	6	{<1,2>, <2,3>, <3,6>}	{1, 2, 3, 6}	<i>alpha</i> = {0, 1, 2, 1, -1, 3} <i>beta</i> = {0, 7, 2, 8, ∞, 3}
8	Изменяем <i>alpha</i> и <i>beta</i> . Смежной с вершиной 6 является вершиной 5. В <i>beta</i> заносим длину ребра из вершины 6 в вершину 5, в <i>alpha</i> – номер текущей вершиной 6	6	{<1,2>, <2,3>, <3,6>}	{1, 2, 3, 6}	<i>alpha</i> = {0, 1, 2, 1, 6, 3} <i>beta</i> = {0, 7, 2, 8, 3, 3}
9	Находим в <i>beta</i> минимальное расстояние до каркаса для не включенных в <i>nodes</i> вершин: <i>beta</i> [5]=3. Включаем вершину 5 в <i>nodes</i> , а ребро <5, 6> в каркас. Вершина 5 стала текущей	5	{<1,2>, <2,3>, <3,6>, <5, 6>}	{1, 2, 3, 6, 5}	<i>alpha</i> = {0, 1, 2, 1, 6, 3} <i>beta</i> = {0, 7, 2, 8, 3, 3}

№	Действия с комментариями	Текущая вершина	Массив ребер <i>ostov</i>	Массив вершин в каркасе <i>nodes</i>	Массив ближ. вершин <i>alpha</i> и длин <i>beta</i>
10	Изменяем <i>alpha</i> и <i>beta</i> . Смежной с вершиной 5 является верш. 4. В <i>beta</i> заносим длину ребра <4,5>, в <i>alpha</i> – номер текущей вершины 5	5	{<1,2>, <2,3>, <3,6>, <5, 6>}	{1, 2, 3, 6, 5}	<i>alpha</i> = {0, 1, 2, 5, 6, 3} <i>beta</i> = {0, 7, 2, 2, 3, 3}
11	Минимальный элемент <i>beta</i> для не включенных в <i>nodes</i> вершин: <i>beta</i> [4]=2. Включаем вершину 4 в <i>nodes</i> , а ребро <4, 5> в каркас. Текущая вершина – 4	4	{<1,2>, <2,3>, <3,6>, <5, 6>, <4, 5>}	{1, 2, 3, 6, 5, 4}	–
В каркас добавлены все вершины. Решение получено					

Далее в листинге 5.2.11 приведем программный код функции построения каркаса минимального веса для графа с помощью алгоритма Прима. Будем использовать язык программирования C#.

```

// определение функции построения каркаса минимального веса
// для графа с помощью алгоритма Прима
void Prim(Graph2 g, out Edge[] ostov)
{
    // выделение памяти под массив ребер каркаса
    ostov = new Edge[g.n - 1];
    // выделение памяти под массив включенных в каркас вершин
    int[] nodes = new int[g.n];
    // выделение памяти под вспомогательные массивы
    // ближайших вершин из каркаса и расстояний до них
    int[] alpha = new int[g.n];
    int[] beta=new int[g.n];
    // включение в каркас нулевой вершины и инициализация
    // вспомогательных массивов согласно матрице смежности
    nodes[0] = 0;
    int count = 1, u = 0, v, w, i, ii, min, flag;
    for(v = 1; v < g.n; v++)
        if(g.a[0, v] != int.MaxValue)
        {
            // если v смежна нулевой вершине
            alpha[v] = 0;
            beta[v] = g.a[0, v];
        }
    else
    {
        // если v не смежна нулевой вершине
        alpha[v] = -1;
        beta[v] = int.MaxValue;
    }
    for(i = 0; i < g.n - 1; i++)
    {
        // поиск ближайшей вершины среди невключенных в каркас
        min = int.MaxValue;
        for(v = 1; v < g.n; v++)
        {
            // проверка на включение вершины в каркас
            flag = 0;
            for(ii = 0; ii < count; ii++)
                if(nodes[ii] == v)
                {
                    flag = 1;
                    if(flag == 0)
                        if(beta[v] < min)
                        {
                            w = v; min = beta[v];
                        }
                }
        }
    }
}

```

Листинг 5.2.11. Функция построения каркаса минимального веса для графа с помощью алгоритма Прима (Начало)


```

// включение найденной вершины в каркас
nodes[count] = w;
count++;
// включение ребра <w, alpha[w]> в каркас
ostov[i] = new Edge();
ostov[i].begin = alpha[w] + 1;
ostov[i].end = w + 1;
ostov[i].len = g.a[alpha[w], w];
// цикл корректировки значений во вспомогательных
// массивах - вершина w может оказаться ближайшей
// из каркаса для еще невключенных вершин
for(v = 0; v < g.n; v++)
{
    flag = 0;
    for(ii = 0; ii < count; ii++)
        if(nodes[ii] == v)
            flag = 1;
    if(flag == 0 && g.a[w, v] != int.MaxValue)
        if(beta[v] > g.a[w, v])
        {
            alpha[v] = w;
            beta[v] = g.a[w, v];
        }
}
}
}
}

```

Листинг 5.2.11. Функция построения каркаса минимального веса для графа с помощью алгоритма Прима (Окончание)

Домашнее задание

1. Написать функции преобразования графа из одного вида представления в другой:
 - а) из матрицы смежности в списки смежности;
 - б) из матрицы смежности в матрицу инцидентности;
 - в) из списков смежности в матрицу смежности.
2. Дан граф. Написать функцию построения его дополнения. Дополнением графа G называется граф G' с тем же множеством вершин, что и в G , причем две вершины в G' смежны тогда и только тогда, когда они несмежны в G .
3. Дан граф. Написать функцию, определяющую, является ли он полным. Граф называется полным, если любые две его вершины соединены ребром.

-
4. Дан граф. Написать функцию поиска всех источников (стоков) графа. Источник графа – это вершина, из которой достижимы все остальные вершины (из этой вершины можно построить путь в любую вершину графа). Сток графа – это вершина, достижимая из всех других вершин.
 5. Дан граф. Написать функцию поиска всех вершин графа, недостижимых из заданной вершины.
 6. Даны граф и номера трех его вершин. Написать функцию поиска всех путей из первой вершины во вторую, которые проходят через третью вершину.
 7. Даны граф и номера двух его вершин. Написать функцию поиска всех путей из первой вершины во вторую, длина которых больше заданной величины.
 8. Дан граф. Написать функцию, которая находит путь максимальной длины, исходящий из заданной вершины.
 9. Дан граф. Написать функцию поиска пути максимальной длины в графе.
 10. Дан граф. Написать функцию поиска медианы графа, т. е. такой его вершины, что сумма расстояний от нее до остальных вершин минимальна.
 11. Дан граф. Написать функцию, определяющую, является ли он связным. Связным называется граф, из каждой вершины которого можно построить путь в любую другую его вершину.
 12. Дан граф. Написать функцию, определяющую, является ли он деревом. Деревом называется связный граф без циклов.
 13. Построить каркас заданного графа алгоритмом Краскала (1956). Последовательно в каркас включаются ребра с наименьшей длиной так, чтобы получившаяся часть каркаса не содержала циклов. Для проверки наличия цикла можно использовать алгоритм Дейкстры: если рассматривается ребро $\langle a, b \rangle$, то ищем путь в построенной части каркаса из узла a в узел b . Если путь будет найден, добавление ребра приведет к образованию цикла, т. е. это ребро в каркас не включаем. Если цикла нет, то ребро включается в каркас.

1. *Абрамов, С. А.* Задачи по программированию / С. А. Абрамов, Г. Г. Гнездилова, Е. Н. Капустина, М. И. Селюн. – М. : Наука. Гл. ред. физ.-мат. лит., 1988. – 224 с.
2. *Бабенко, М. А.* Введение в теорию алгоритмов и структур данных [Электронный ресурс] / М. А. Бабенко, М. В. Левин. – Электрон. дан. — М. : МЦНМО, 2016. – 144 с.
3. *Белов, В. В.* Алгоритмы и структуры данных : учебник / В. В. Белов, В. И. Чистякова. – М. : КУРС : НИЦ ИНФРА-М, 2016. – 240 с.
4. *Вирт, Н.* Алгоритмы и структуры данных. Новая версия для Оберона + CD / пер. с англ. Ф. В. Ткачева. – М. : ДМК Пресс, 2010. – 272 с.
5. *Кнут, Д. Э.* Искусство программирования. Т. 1: Основные алгоритмы = The Art of Computer Programming. Vol. 1. Fundamental Algorithms / под ред. С. Г. Тригуб, Ю. Г. Гордиенко, И. В. Красикова. – М. : Вильямс, 2002. – Т. 1. – 720 с.
6. *Кнут, Д. Э.* Искусство программирования. Т. 2. Получисленные алгоритмы = The Art of Computer Programming. Vol. 2: Seminumerical Algorithms / под ред. Л. Ф. Козаченко, В. Т. Тертышного, И. В. Красикова. – М. : Вильямс, 2001. – Т. 2. – 832 с.
7. *Кнут, Д. Э.* Искусство программирования. Т. 3: Сортировка и поиск = The Art of Computer Programming. Vol. 3. Sorting and Searching / под ред. В. Т. Тертышного, И. В. Красикова. – 2-е изд. – М. : Вильямс, 2007. – Т. 3. – 832 с.
8. *Колдаев, В. Д.* Структуры и алгоритмы обработки данных : учеб. пособие. – М. : ИЦ РИОР : НИЦ ИНФРА-М, 2014. – 296 с.
9. *Культин, Н. Б.* C/C++ в задачах и примерах. – 2-е изд., перераб. и доп. – СПб. : БХВ-Петербург, 2009. – 368 с.
10. *Культин, Н. Б.* Microsoft Visual C# в задачах и примерах (+ CD-ROM). – М. : БХВ-Петербург, 2012. – 314 с.

-
11. *Меньшиков, Ф. В.* Олимпиадные задачи по программированию (+CD). – СПб. : Питер, 2006. – 315 с.
 12. *Москвитина, О. А.* Сборник примеров и задач по программированию [Электронный ресурс] : учеб. пособие / О. А. Москвитина, В. С. Новичков, А. Н. Пылькин. – М. : Горячая линия – Телеком, 2014. – 245 с.
 13. *Новиков, Ф. А.* Дискретная математика для программистов : учебник. – 3-е изд. – СПб. : Питер, 2009. – 384 с.
 14. *Прага, С.* Язык программирования С. Лекции и упражнения : пер. с англ. – 6-е изд. – М. : ООО «И. Д. Вильямс», 2015. – 928 с.
 15. *Троелсен, Э.* Язык программирования C# 5.0 и платформа .NET 4.5. – М. : Вильямс, 2015. – 633 с.
 16. *Тюкачев, Н. А.* C#. Алгоритмы и структуры данных [Электронный ресурс] / Н. А. Тюкачев, В. Г. Хлебостроев. – СПб. : Лань, 2017. — 232 с.

*Анастасия Александровна АНДРИАНОВА,
Линар Наилевич ИСМАГИЛОВ,
Татьяна Маратовна МУХТАРОВА*

**АЛГОРИТМИЗАЦИЯ И ПРОГРАММИРОВАНИЕ
ПРАКТИКУМ**

Учебное пособие

Издание третье, стереотипное

Зав. редакцией
литературы по информационным технологиям
и системам связи *О. Е. Гайнутдинова*

ЛР № 065466 от 21.10.97
Гигиенический сертификат 78.01.10.953.П.1028
от 14.04.2016 г., выдан ЦГСЭН в СПб

Издательство «ЛАНЬ»
lan@lanbook.ru; www.lanbook.com;
196105, Санкт-Петербург, пр. Юрия Гагарина, д. 1, лит. А.
Тел.: (812) 412-92-72, 336-25-09.
Бесплатный звонок по России: 8-800-700-40-71

Подписано в печать 31.07.24.
Бумага офсетная. Гарнитура Школьная. Формат 60×90^{1/16}.
Печать офсетная/цифровая. Усл. п. л. 15,00. Тираж 30 экз.

Заказ № 342-24-1.

Отпечатано в полном соответствии
с качеством предоставленного оригинал-макета
в АО «Т8 Издательские Технологии».
109316, г. Москва, Волгоградский пр., д. 42, к. 5.